MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Módulo 8 - Tarea 1



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- 1. Entendiendo qué pide la tarea
- 2. Entorno Android
  - 2.1 Instalación de Genymotion
  - 2.2 Instalación de ADB
  - 2.3 Probando una app
  - 2.4 Instalación de MobSF
  - 2.5 Ejemplo de Análisis Dinámico con MobSF
  - 2.6 Conexión de Burp Suite con Genymotion
    - A) Crear un listener en el puerto 8080
    - B) Instalar el certificado CA en Android
    - C) Instalar el certificado de Burp Suite en el Android de Genymotion
    - D) Poner el proxy global del emulador apuntando a Burp Suite
    - E) Probamos la interceptación del tráfico
  - 2.7 Wireshark
  - 2.8 Instalación de Frida
  - 2.9 Instalación de jadx
  - 2.10 Instalación de apktool
  - 2.11 Instalación de Ghidra
  - 2.12 Instalación de Objection
  - 2.12 Instalación de Drozer
  - 2.13 Instalación de Mara Framework
- 3. Entorno iOS
  - 3.1 Esquema de análisis en iOS
  - 3.2 Análisis estático inicial: MobSF
    - Cosas relevantes a revisar
  - 3.3 Análisis profundo: Ghidra
  - 3.4 Burp Suite
- 4. Conclusiones
  - Notas para mejorar el ejercicio

1. Entendiendo qué pide la tarea

El objetivo de la práctica es **preparar un entorno controlado para el análisis de aplicaciones móviles Android e iOS, permitiendo realizar análisis estático y, en el caso de Android, también análisis dinámico mediante emulación.** Según el módulo:

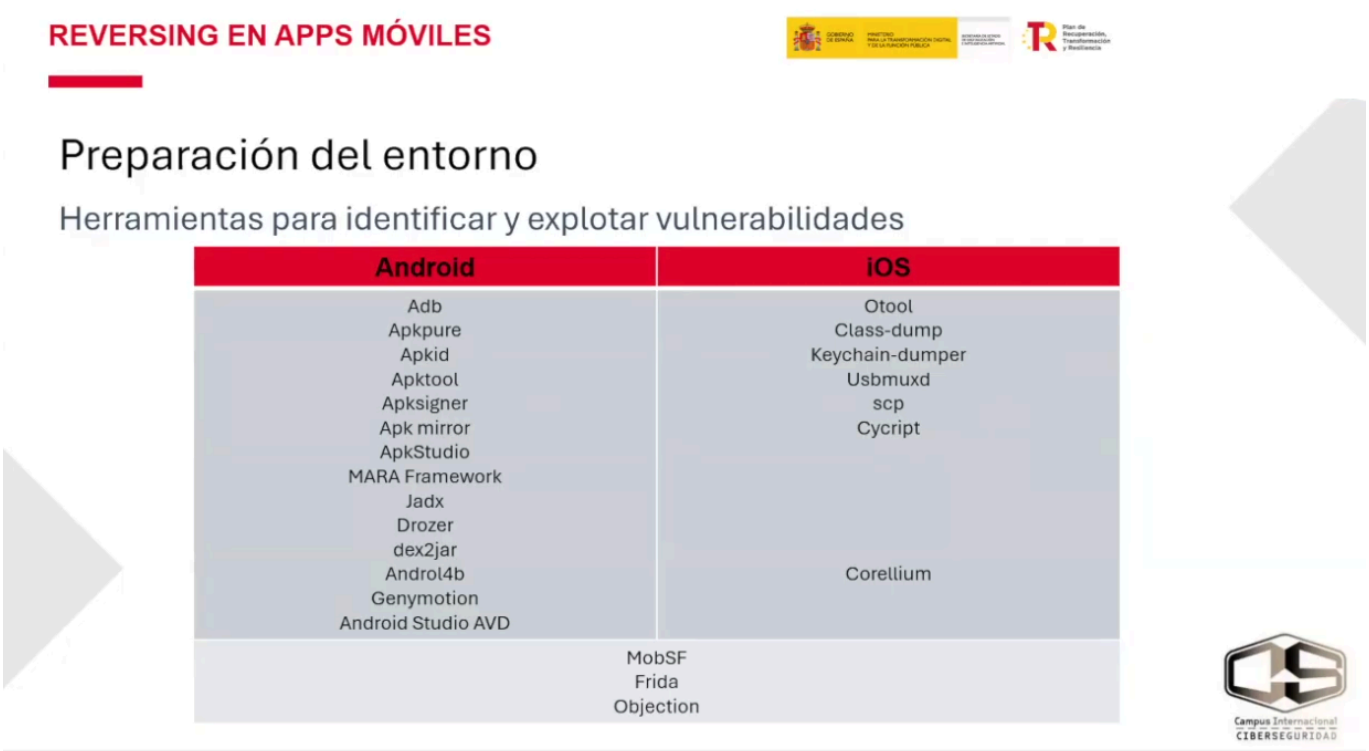
- Android será la plataforma principal para análisis dinámico.
- Mientras que en iOS el trabajo se centrará en archivos `.ipa` ya descifrados y su análisis estático.

Es importante señalar que el objetivo de este ejercicio no es realizar un análisis de malware en sentido estricto, sino preparar y validar el entorno de trabajo necesario para poder llevar a cabo ese tipo de análisis posteriormente. El foco del laboratorio se sitúa en la instalación, configuración y comprobación de las herramientas y de la infraestructura de análisis para aplicaciones móviles Android e iOS, para disponer de un entorno funcional sobre el que realizar tareas de reversing, análisis estático, análisis dinámico y, en su caso, análisis de malware.



donde:

- La diapositiva proporcionada en clase resume las capacidades necesarias para preparar un entorno de reversing móvil:
  - En Android se destaca la necesidad de un emulador/gestor de dispositivos y ADB.
  - En iOS se señalan herramientas orientadas a instalación, gestión del dispositivo y escenarios con jailbreak.
  - Además, se identifican herramientas comunes de interceptación de tráfico, como Burp Suite, Caido y OWASP ZAP.



donde:

- Utilidades para Android:
  - Comunicación con el dispositivo: ADB.
  - Desempaquetado y análisis de APK: Apktool, Jadx, dex2jar.
  - Firma y verificación: Apksigner.
  - Detección de ofuscación o empaquetado: Apkid.
  - Análisis de superficie de ataque y explotación: Drozer, Andro4Lab, MARA Framework.
  - Obtención de muestras o prueba en emulador: APKPure, APKMirror, Genymotion, Android Studio AVD.
  - Entornos de análisis: ApkStudio.
- Utilidades para iOS:
  - Inspección de binarios Mach-O: otool.
  - Volcado de clases: class-dump.
  - Extracción de secretos/dispositivo: keychain-dumper.
  - Conectividad y transferencia: usbmuxd, scp.
  - Instrumentación: Cycript.
  - Virtualización avanzada: Corellium.
- Utilidades comunes:
  - MobSF.
  - Frida.
  - Objection.

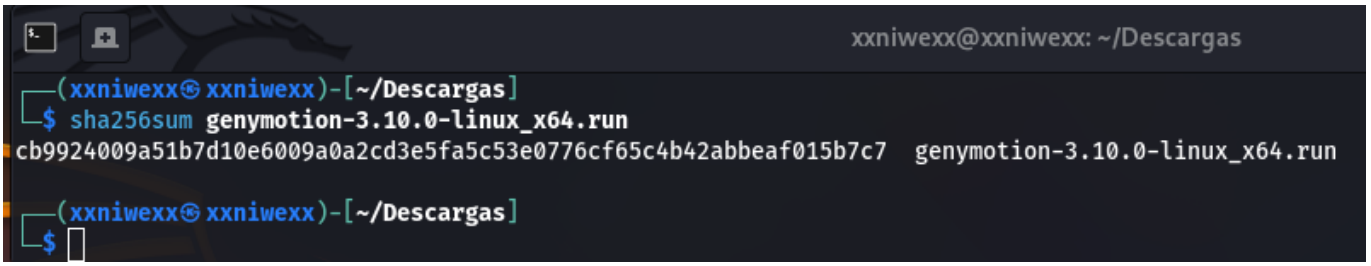
2. Entorno Android

En el sistema anfitrión Kali Linux real instalaremos:

- `VirtualBox` para usar máquinas virtuales.
- `Genymotion` como emulador Android para ejecutar aplicaciones en entorno controlado.
- `ADB` para listar dispositivos, instalar APKs y comunicarse con el emulador desde consola.
- `MobSF` como framework central para análisis estático y apoyo al análisis de seguridad de APK e IPA.
- `jadx`, como herramienta para decompilar `.apk` y `.dex` y mostrar su código en un formato cercano a Java.
- `apktool` como herramienta para desempaquetar y reconstruir aplicaciones Android `.apk`.
- `Ghidra` para realizar análisis estático avanzado de binarios nativos, tanto en bibliotecas `.so` de Android como en binarios `Mach-O` de iOS.
- `Wireshark` para captura y análisis de tráfico de red.
- `Burp Suite` como opción para la interceptación y manipulación del tráfico HTTP/HTTPS.
- `Frida` como herramienta de instrumentación dinámica que permite observar y modificar en tiempo real el comportamiento de una aplicación mientras se está ejecutando.
- `Objection` es una herramienta basada en Frida que sirve para simplificar la instrumentación dinámica de aplicaciones móviles, permitiendo explorar y manipular su comportamiento en tiempo de ejecución mediante comandos de alto nivel.
- `Drozer` es una herramienta de análisis de seguridad para aplicaciones Android que permite interactuar con una app instalada.

2.1 Instalación de Genymotion

Instalamos `Genymotion Desktop` en un host Kali. Para ello **descargamos la aplicación:** https://www.genymotion.com/product-desktop/ y comprobamos que el fichero descargado sea legítimo:



Marcamos el fichero descargado como ejecutable:

```
chmod +x genymotion-3.10.0-linux_x64.run
```

Genymotion recomienda instalación local, no global: La instalación global con `sudo` en `/opt/genymotion` no la recomiendan porque puede dar problemas de permisos. Para ello ejecutamos:

```
./genymotion-*-linux_x64.run -d "$HOME/genymotion"
```

```
└─$ chmod +x genymotion-3.10.0-linux_x64.run

┌──(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ ./genymotion-*-linux_x64.run -d "$HOME/genymotion"
Installing for current user only. To install for all users, restart this installer as root.

Installing to folder [/home/xxniwexx/genymotion/genymotion]. Are you sure [y/n] ? y


- Extracting files ..................................... OK (Extract into: [/home/xxniwexx/genymotion/genymotion])
- Installing launcher icon ............................. OK

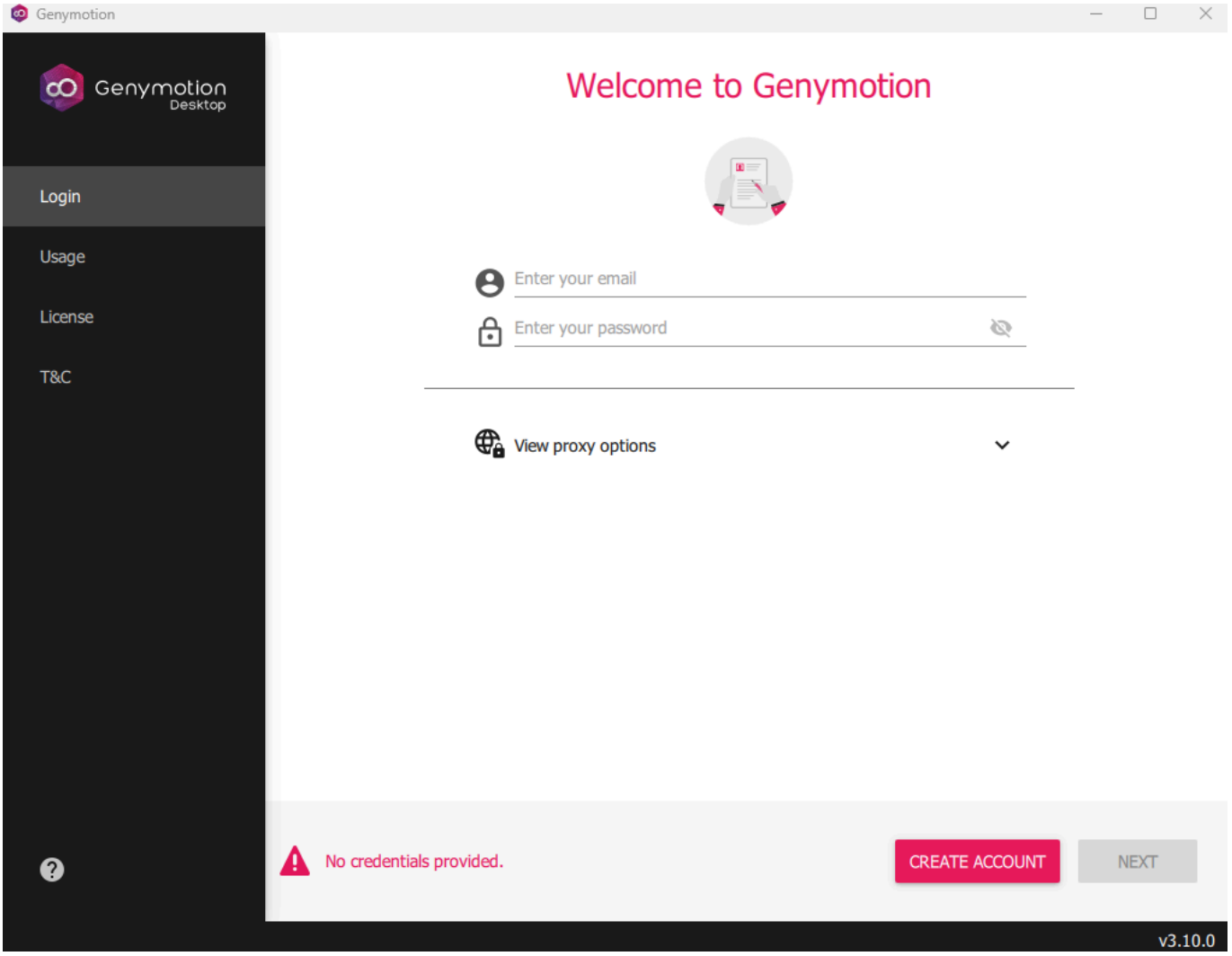
Installation done successfully.

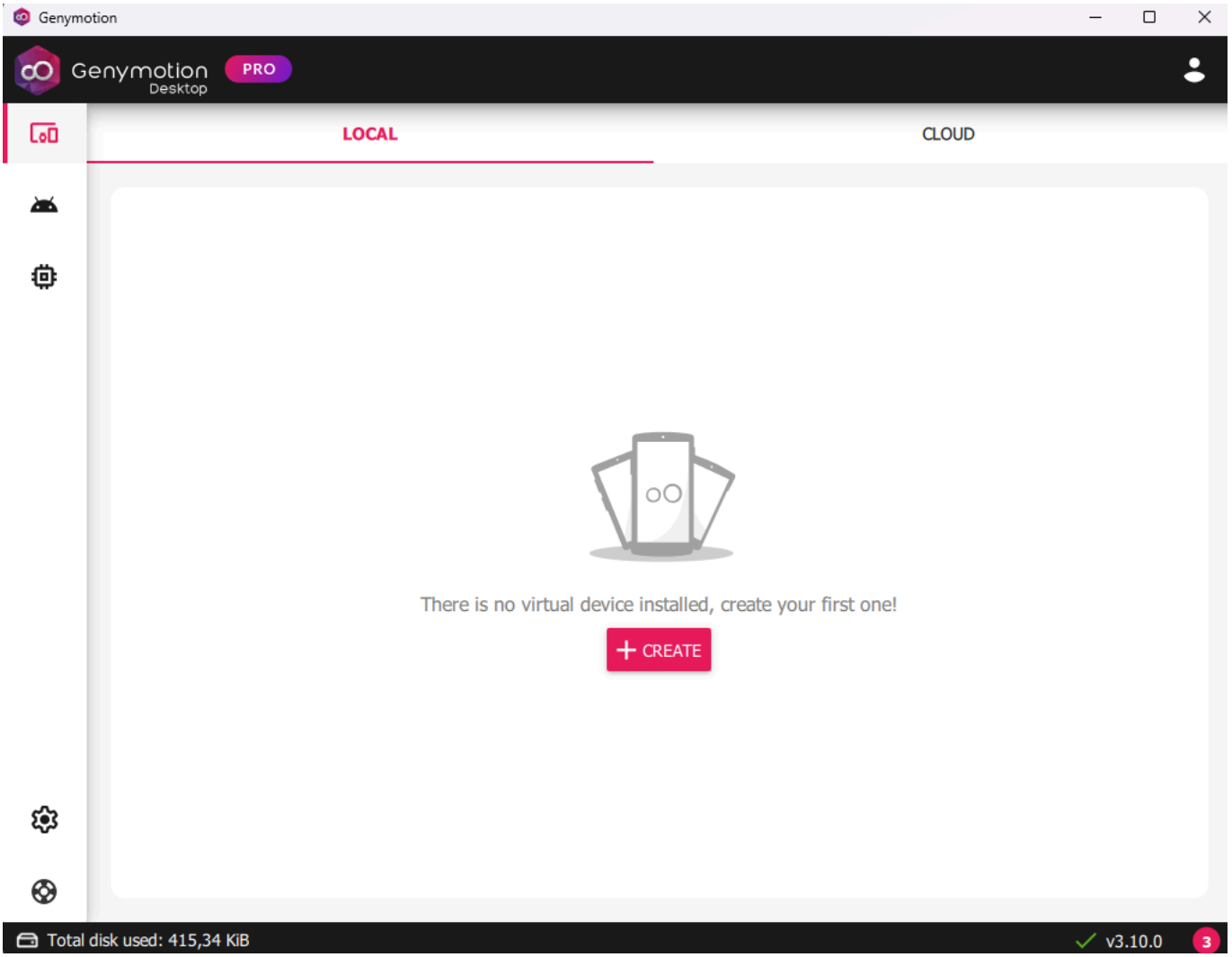
You can now use these tools from [/home/xxniwexx/genymotion/genymotion]:
 - genymotion
 - genymotion-shell
 - gmtool
```

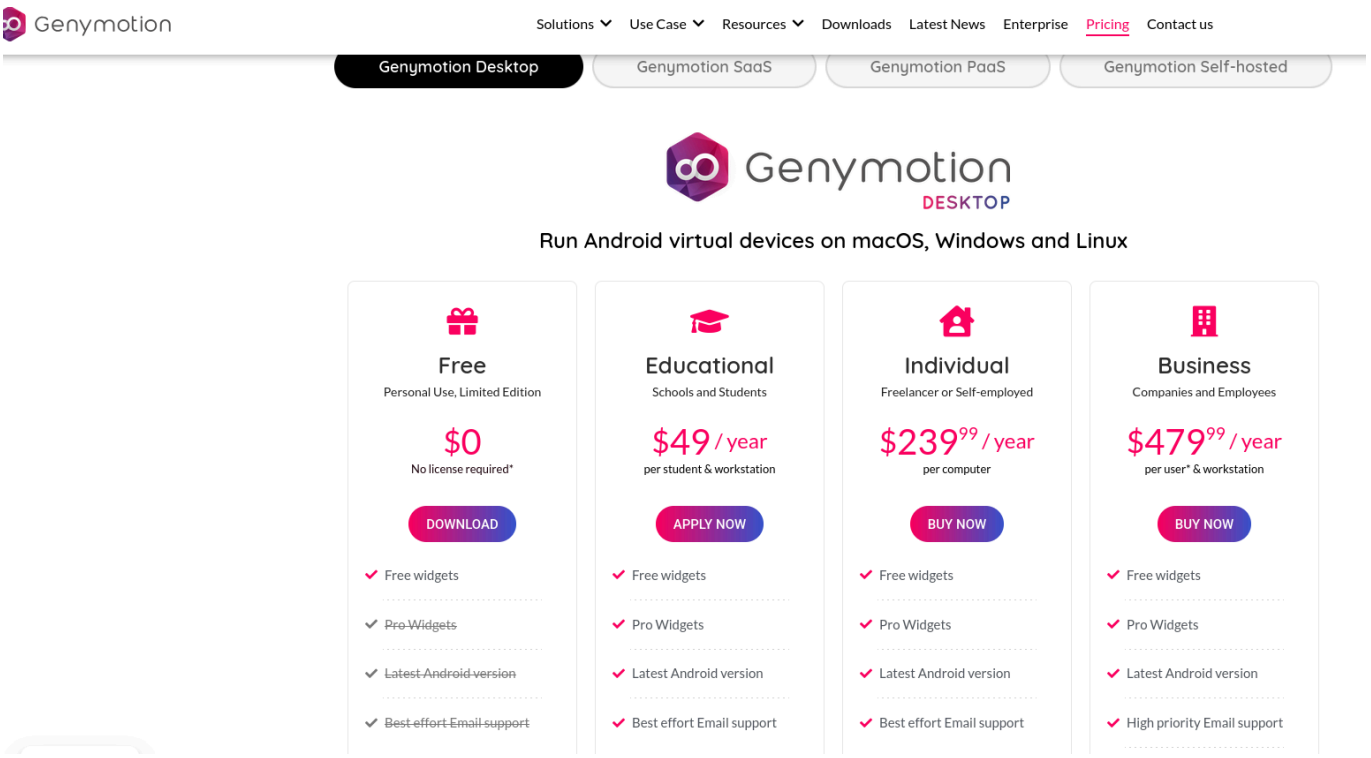
Arrancamos Genymotion:

```
~/genymotion/genymotion/genymotion
```

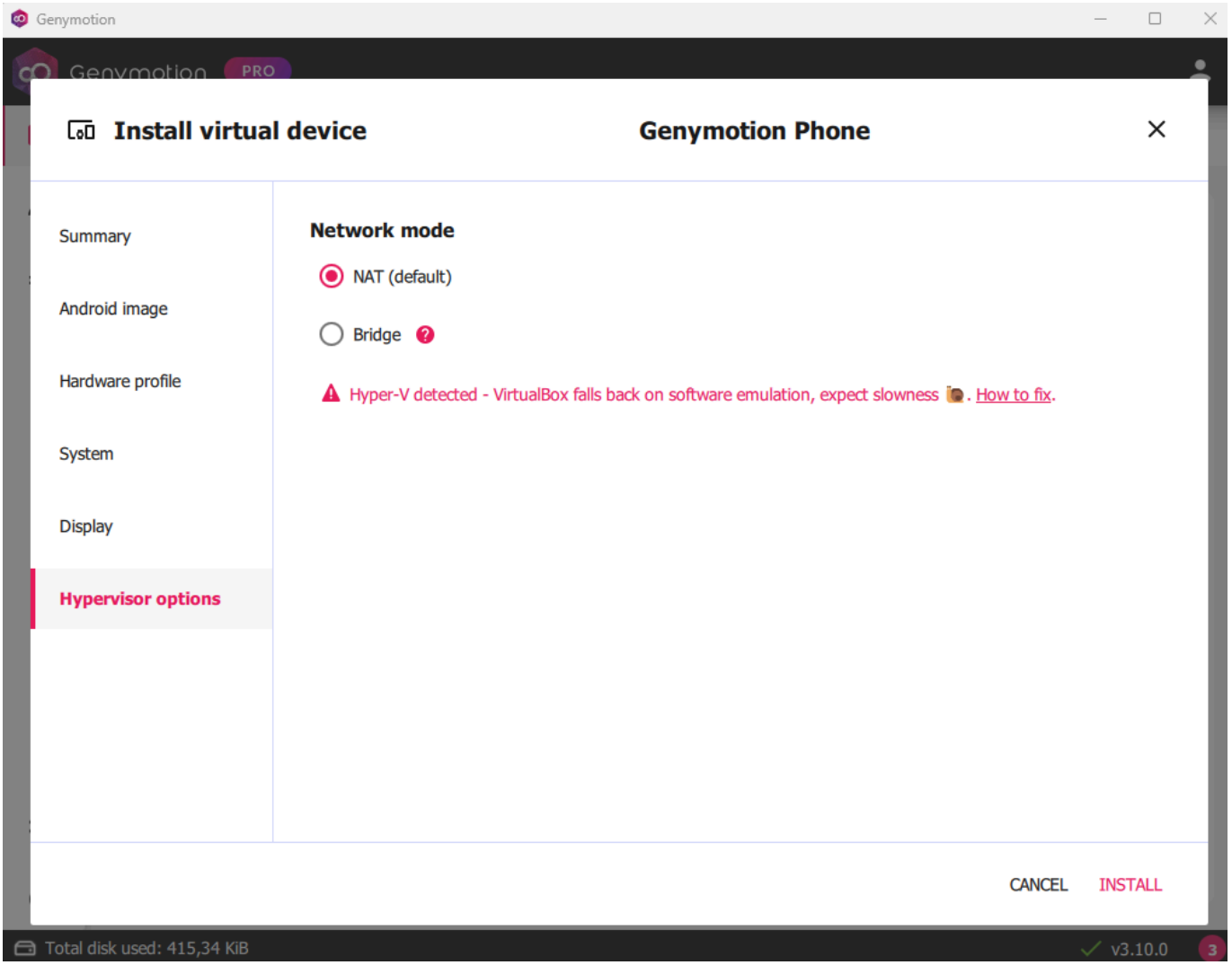
Abrimos Genymotion e iniciamos sesión: Si no tenemos cuenta, la tenemos que crear desde la propia app, confirmando el correo y entrando con las credenciales. La edición `Personal Use - Free` se activa tras descargar, instalar, iniciar `Genymotion`, crear la cuenta, validar el email y elegir `personal use`:




Genymotion
Genymotion Desktop
PRO
LOCAL
CLOUD
There is no virtual device installed, create your first one!
CREATE
Total disk used: 415,34 KiB
v3.10.0
3

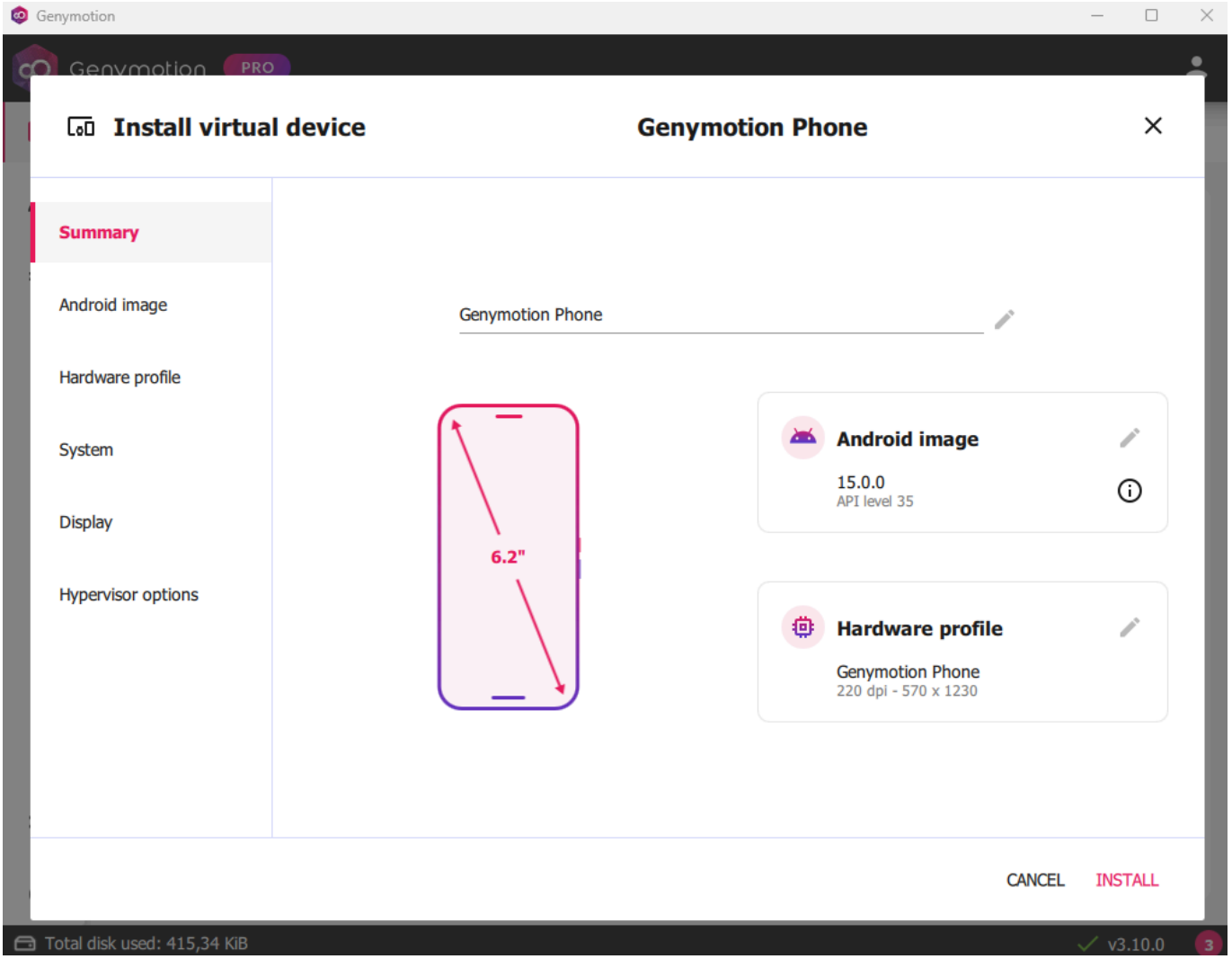

Genymotion
Solutions
Use Case
Resources
Downloads
Latest News
Enterprise
Pricing
Contact us
Genymotion Desktop
Genymotion SaaS
Genymotion PaaS
Genymotion Self-hosted
Genymotion
DESKTOP
Run Android virtual devices on macOS, Windows and Linux
Free
Personal Use, Limited Edition
$0
No license required*
DOWNLOAD
Free widgets
Pro Widgets
Latest Android version
Best effort Email support
Educational
Schools and Students
$49 / year
per student & workstation
APPLY NOW
Free widgets
Pro Widgets
Latest Android version
Best effort Email support
Individual
Freelancer or Self-employed
$239.99 / year
per computer
BUY NOW
Free widgets
Pro Widgets
Latest Android version
Best effort Email support
Business
Companies and Employees
$479.99 / year
per user* & workstation
BUY NOW
Free widgets
Pro Widgets
Latest Android version
High priority Email support

Creamos un dispositivo virtual: Ya dentro de Genymotion, pulsamos `Create`, elegimos una plantilla de Android y la arrancamos:

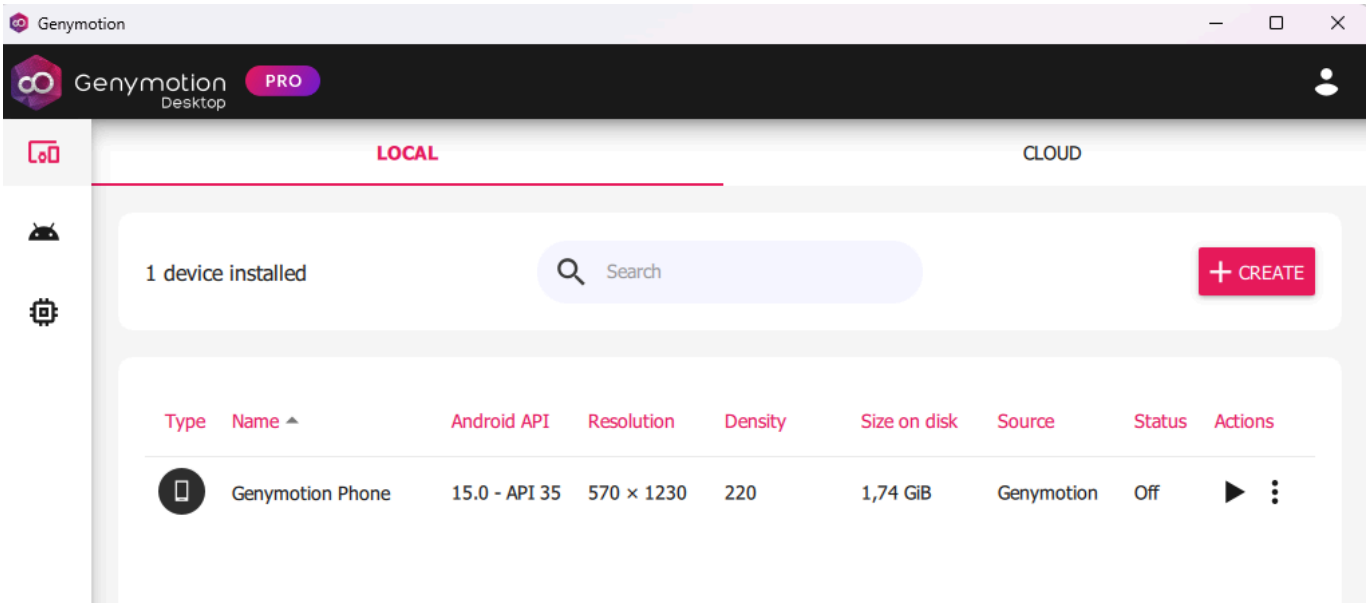


donde:

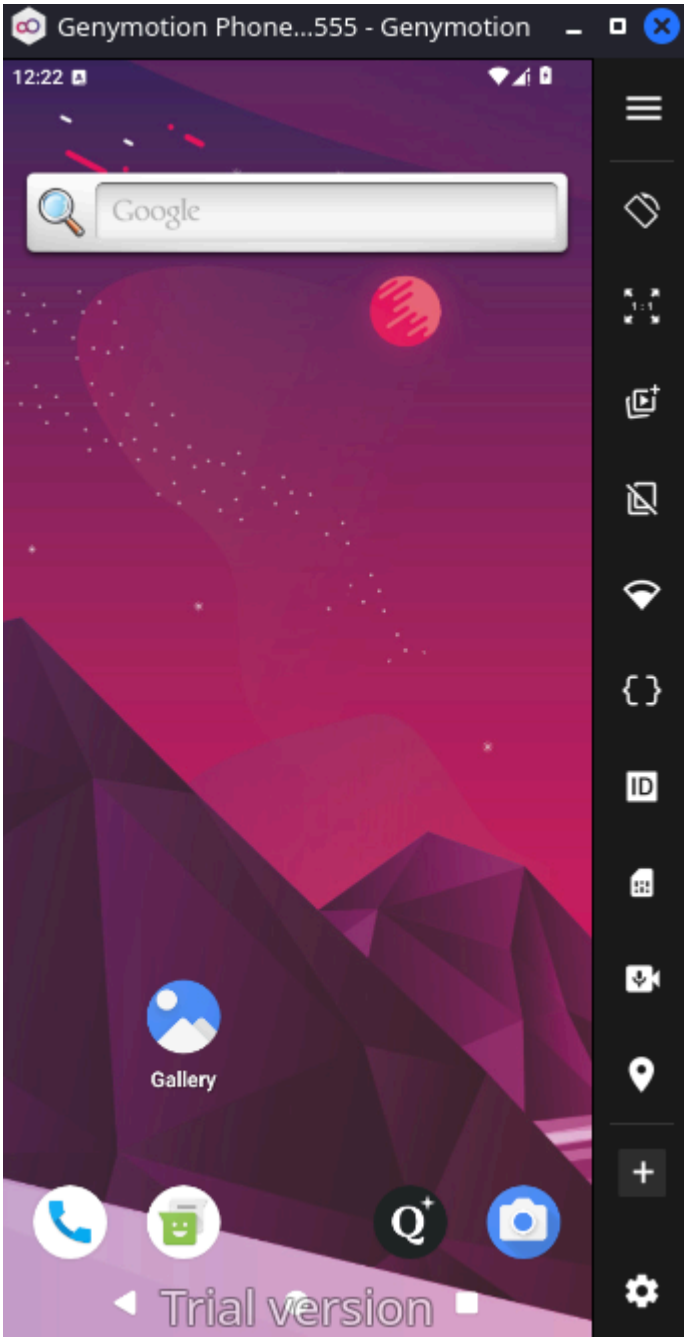
- Mantenemos `NAT (default)` seleccionado porque:
  - Da salida a Internet al emulador sin tener que tocar apenas la red.
  - Aísla mejor el dispositivo virtual del resto de la red local.
  - Evita problemas de configuración que sí pueden aparecer con Bridge, como conflictos de IP, DHCP, adaptadores o visibilidad en la red.
- Pulsamos `INSTALL`: El aviso de Hyper-V detected no impide crear el dispositivo. Implica que irá más lento porque VirtualBox no está usando aceleración por hardware completa.



Finaliza el proceso de creación de un dispositivo Android 15.0:



Arrancamos el dispositivo creado, pulsando el botón de `play`:



El siguiente paso es conectar este dispositivo con el host Kali usando ADB. Para ello, cuando el dispositivo android arranque, comprobaremos si ADB lo ve.

2.2 Instalación de ADB

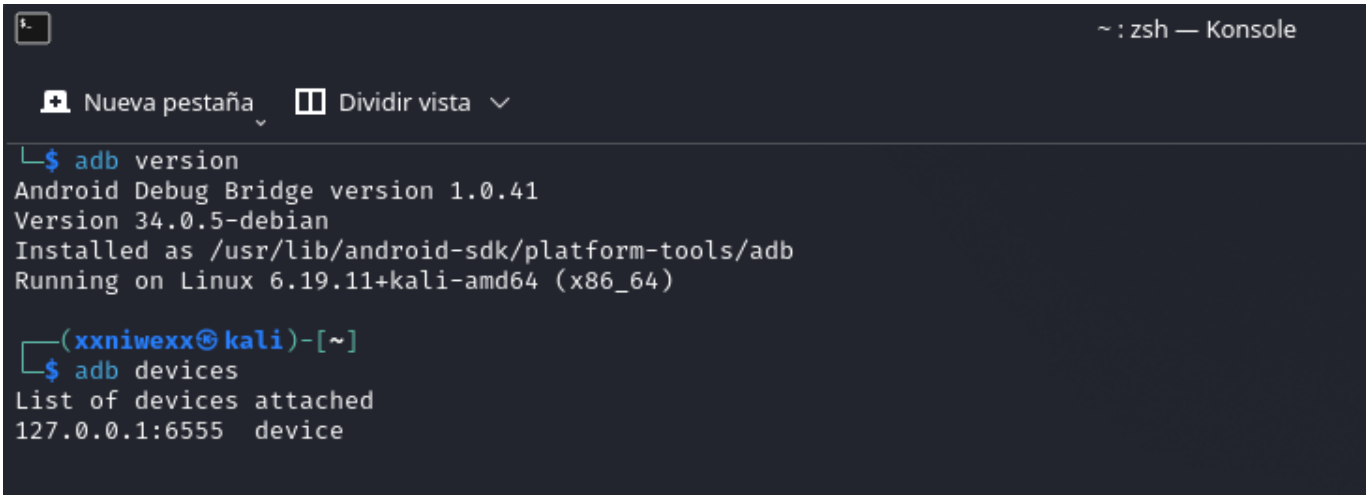
```
sudo apt install adb
Instalando:
  adb

Instalando dependencias:
  android-libbase  android-libboringssl  android-libcutils  android-liblog
android-libziparchive  android-udev-rules

Resumen:
  Actualizando: 0, Instalando 7, Eliminando: 0, no actualizando: 0
  Tamaño de la descarga: 1.198 kB
  Espacio necesario: 3.764 kB / 53,0 GB disponible

¿Continuar? [S/n] S
```

Comprobamos que adb ha sido instalado en el host Kali: Mostramos la versión y los dispositivos que encuentra:



donde:

- `127.0.0.1:6555 device`: Encuentra el dispositivo android que antes creamos.

2.3 Probando una app

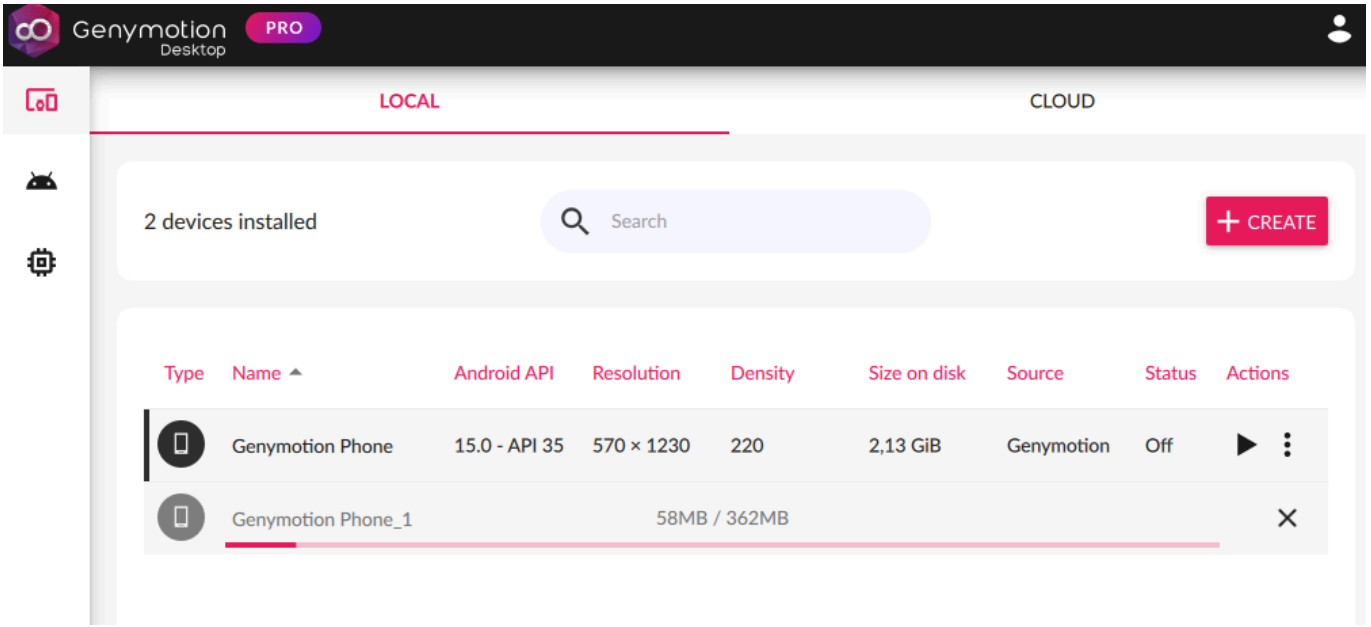
Descargamos la aplicación que aporta el temario del módulo para hacer prácticas con ella.

```
└─$ adb install diva-beta.apk
Performing Streamed Install
adb: failed to install diva-beta.apk: Failure
[INSTALL_FAILED_DEPRECATED_SDK_VERSION: App package must target at least
SDK version 24, but found 23]
```

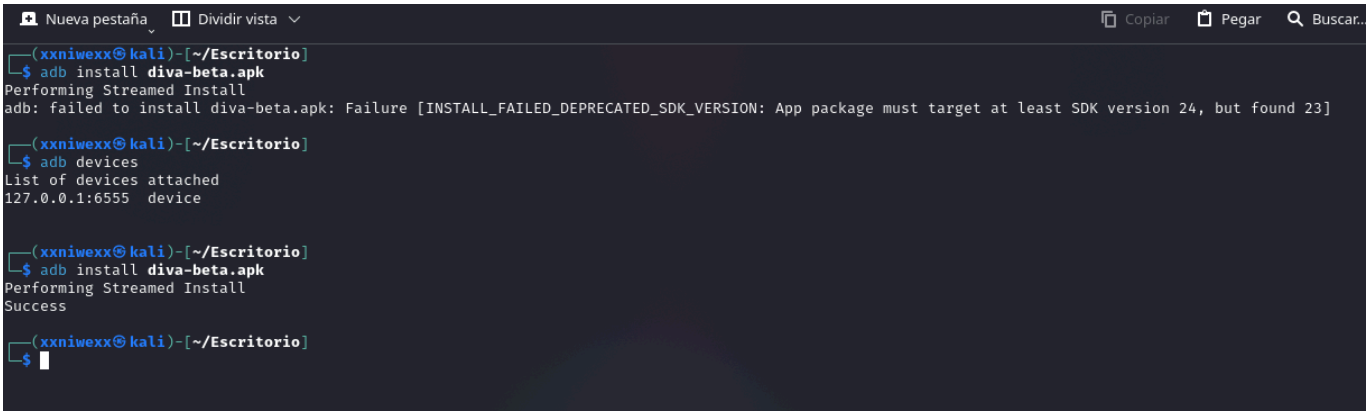
donde:

- Comprobamos que necesitamos instalar un dispositivo android más antiguo para que esta aplicación pueda arrancar.

Instalamos y arrancamos otro dispositivo Android 7.0:



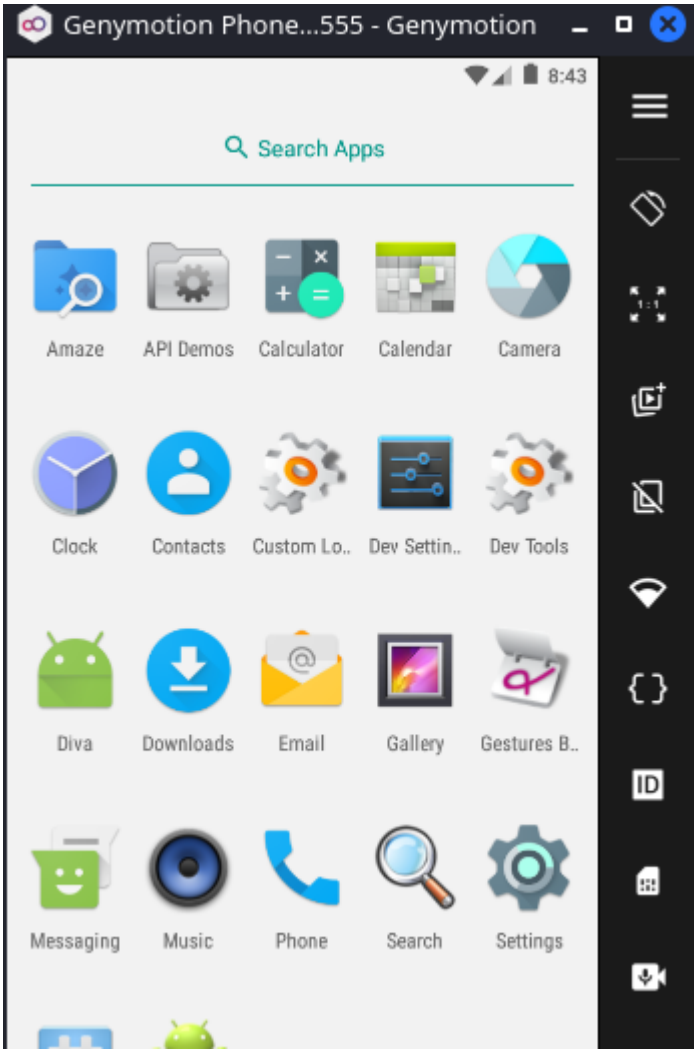
Instalamos con adb la aplicación `diva-beta.apk`.



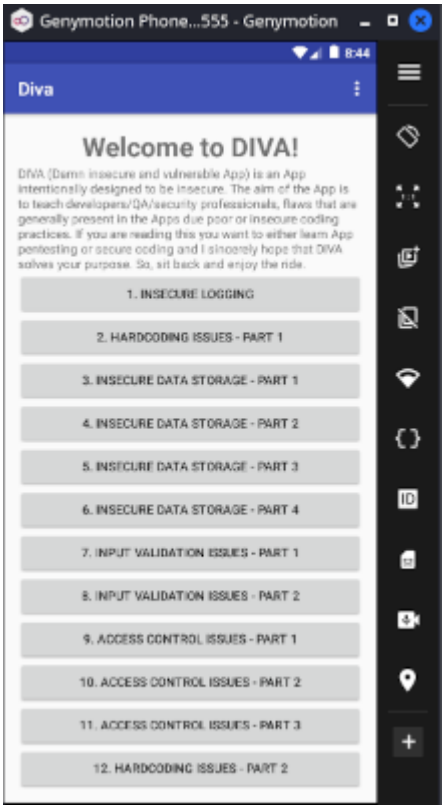
donde:

- Comprobamos los dispositivos que `adb` encuentra conectados: `127.0.0.1:6555 device`.

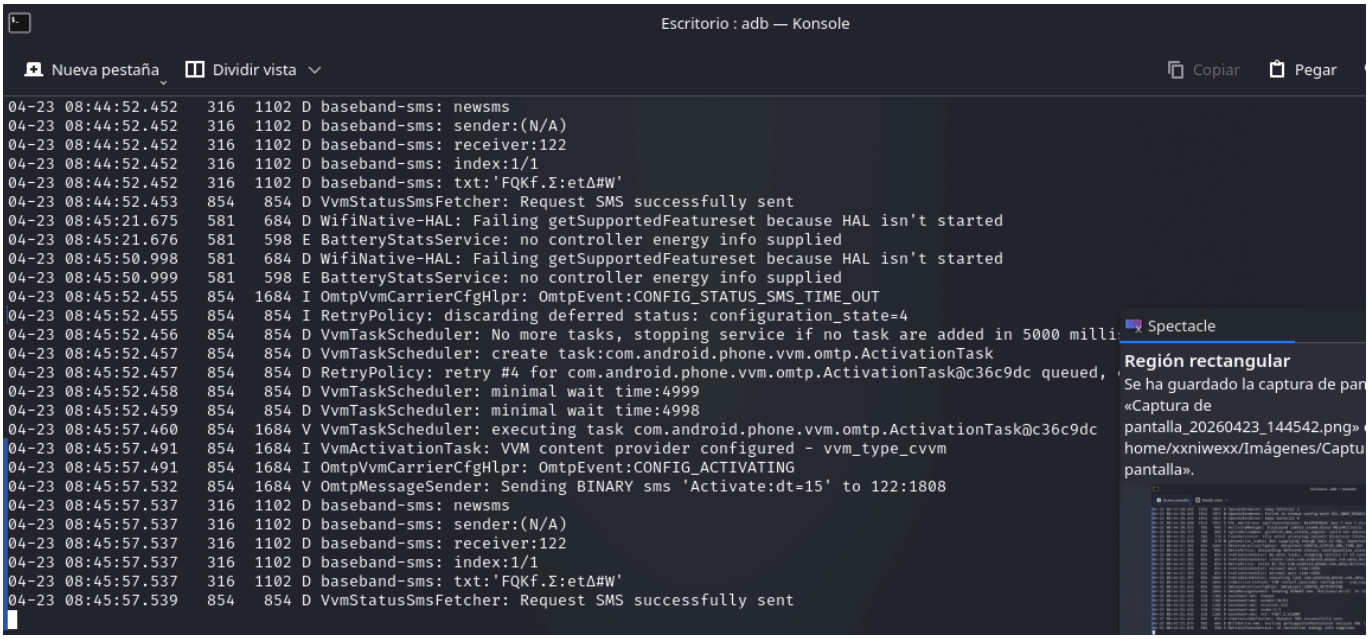
Arrancamos el dispositivo android 7 y buscamos la aplicación `diva-beta.apk`:



Ejecutamos `diva-beta.apk`:



Miramos los logs con `adb logcat`: En una terminal de comandos ejecutamos `adb logcat`:



donde:

- `logcat` sirve para ver mensajes, errores y actividad de la app durante la ejecución.

2.4 Instalación de MobSF

Actualizamos el sistema e instalamos una serie de paquetes que son necesarios:

```
sudo apt update
sudo apt install -y git python3 python3-venv python3-pip python3-dev build-
essential \
  libffi-dev libssl-dev libxml2-dev libxslt1-dev libjpeg62-turbo-dev
zlib1g-dev \
  openjdk-21-jdk
```

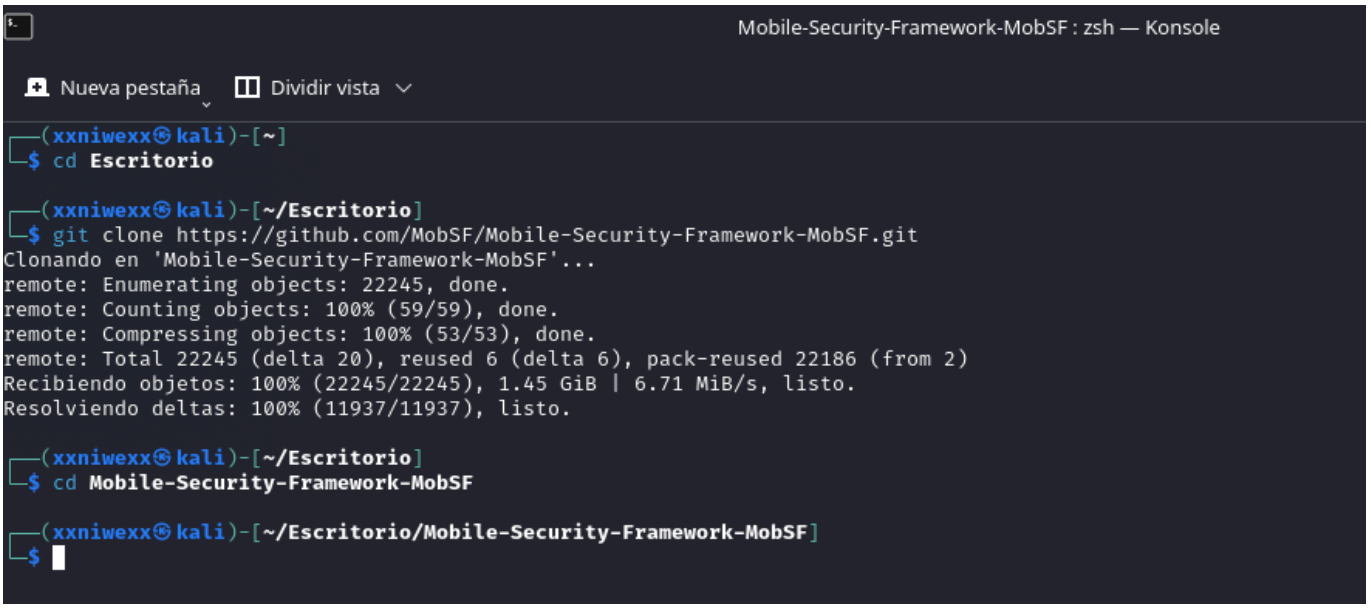
donde instalamos:

- `git` → para clonar el repositorio de `MobSF` desde GitHub.
- `python3` → lenguaje principal en el que está desarrollado `MobSF`.
- `python3-venv` → para crear un entorno virtual y aislar las dependencias de Python.
- `python3-pip` → para instalar paquetes de Python necesarios durante la instalación.
- `python3-dev` → incluye cabeceras y archivos de desarrollo para compilar módulos Python nativos.
- `build-essential` → instala compiladores y herramientas básicas de compilación necesarias para algunas dependencias.
- `libffi-dev` → soporte de desarrollo para librerías que usan interfaces entre Python y código C.
- `libssl-dev` → librerías y cabeceras de OpenSSL para funciones de cifrado y comunicaciones seguras.
- `libxml2-dev` → soporte para procesar archivos XML.
- `libxslt1-dev` → soporte para transformaciones y manejo avanzado de `XML/XSLT`.
- `libjpeg62-turbo-dev` → librerías de desarrollo para tratamiento de imágenes `JPEG`.

- `zlib1g-dev` → soporte para compresión y descompresión de datos.
- `openjdk-21-jdk` → entorno Java necesario para componentes y herramientas del análisis Android en `MobSF`.

Ahora descargamos MobSF:

```
git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
cd Mobile-Security-Framework-MobSF
```



Hacemos la instalación de MobSF en un entorno virtual python:

```
cd ~/Mobile-Security-Framework-MobSF

sudo apt install -y python3-full

python3 -m venv .venv
source .venv/bin/activate

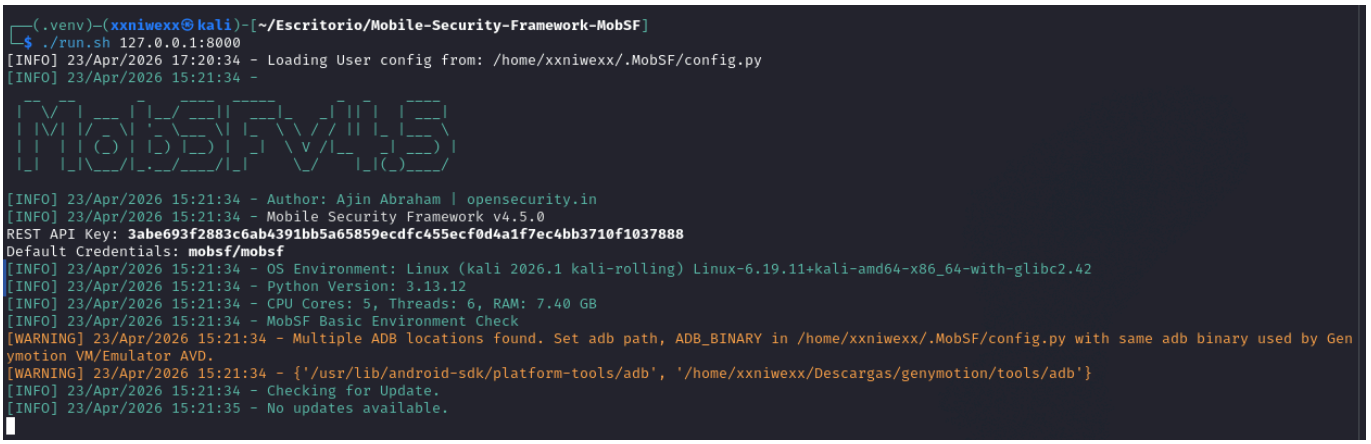
python -m pip install --upgrade pip wheel

./setup.sh

...
...
...
...
  __  __        _     _____  ______        _  _     _____
 |  \/  | ___ | |__/ ___||  ___|_   _| || |  | ___|
 | |\/| |/ _ \| '_ \___ \| |_  \ \ / / || |_ |___ \
 | |  | | (_) | |_) |__) |  _|  \ V /|__   _| ___) |
 |_|  |_|\___/|_.__/____/|_|     \_/    |_|(_)____/

[INFO] 23/Apr/2026 17:46:46 - Author: Ajin Abraham | opensecurity.in
```

```
[INFO] 23/Apr/2026 17:46:46 - Mobile Security Framework v4.5.0
REST API Key:
30de80bb81566d17f50b8b7a3c421fd7066f864347bae0b6ebf60c2a9e81123d
Default Credentials: mobsf/mobsf
[INFO] 23/Apr/2026 17:46:46 - OS Environment: Linux (kali 2026.1 kali-
rolling) Linux-6.18.12+kali-amd64-x86_64-with-glibc2.42
[INFO] 23/Apr/2026 17:46:46 - Python Version: 3.13.12
[INFO] 23/Apr/2026 17:46:46 - CPU Cores: 24, Threads: 32, RAM: 62.63 GB
[INFO] 23/Apr/2026 17:46:46 - MobSF Basic Environment Check
[WARNING] 23/Apr/2026 17:46:46 - Multiple ADB locations found. Set adb
path, ADB_BINARY in /home/xxniwexx/.MobSF/config.py with same adb binary
used by Genymotion VM/Emulator AVD.
[WARNING] 23/Apr/2026 17:46:46 - {'/usr/lib/android-sdk/platform-
tools/adb', '/home/xxniwexx/Descargas/genymotion/tools/adb'}
Roles Created Successfully!
[INFO] 23/Apr/2026 17:46:46 - Checking for Update.
[INFO] 23/Apr/2026 17:46:46 - No updates available.
Download and Install wkhtmltopdf for PDF Report Generation -
https://wkhtmltopdf.org/downloads.html
[INSTALL] Installation Complete
```



donde:

- Obtenemos una advertencia:

```
hay dos ADB distintos
/usr/lib/android-sdk/platform-tools/adb
/home/xxniwexx/Descargas/genymotion/tools/adb
```

Para que MobSF y Genymotion usen el mismo `ADB`, conviene fijarlo en la configuración de `MobSF`:
Como el `adb de Genymotion` se encuentra en: `/home/xxniwexx/Descargas/genymotion/tools/adb` vamos a fijar este `ADB` en la configuración de `MobSF`. Para ello editamos el archivo de configuración:

```
nano /home/xxniwexx/.MobSF/config.py
```

Y añadimos:

```
ADB_BINARY = "/home/xxniwexx/Descargas/genymotion/tools/adb"
```



Guardamos, cerramos y reiniciamos MobSF:

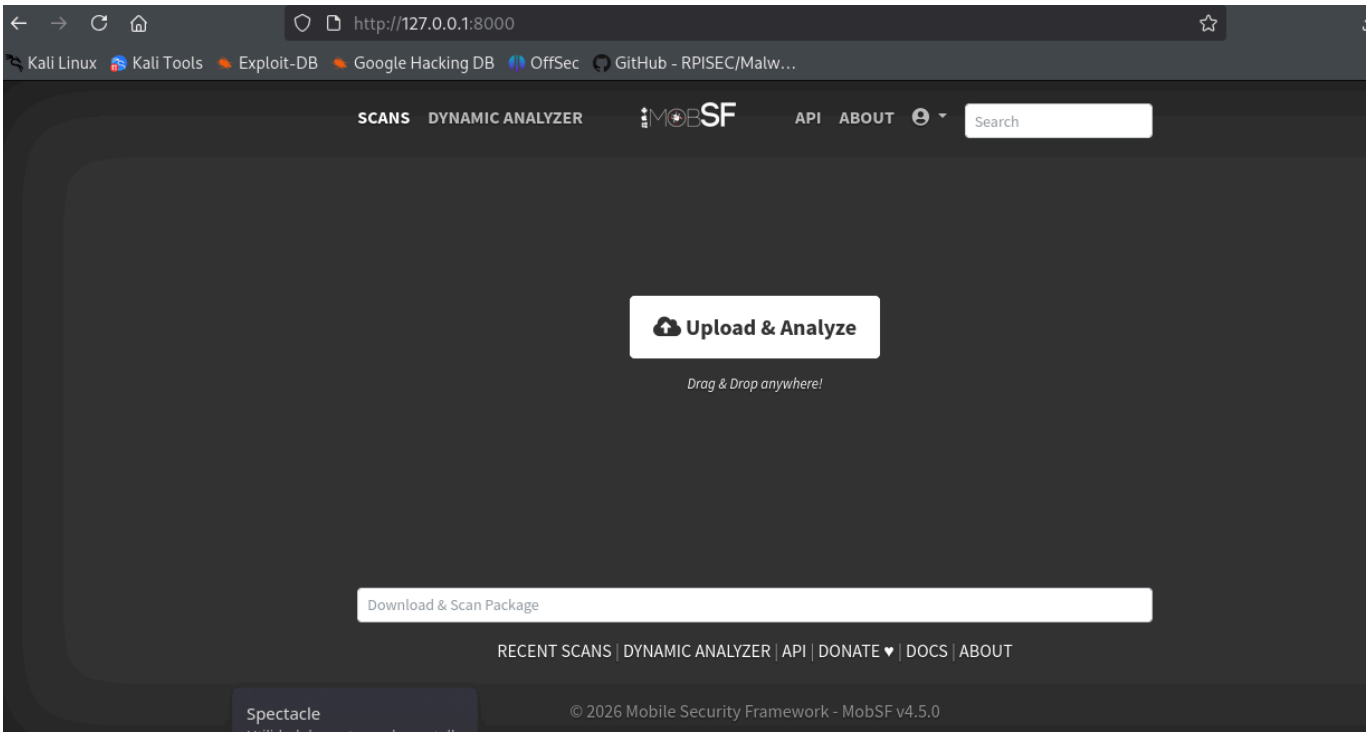
```
└─$ ./run.sh 127.0.0.1:8000
```

donde:

- Ya no obtenemos la advertencia de que encuentra dos `adb`.

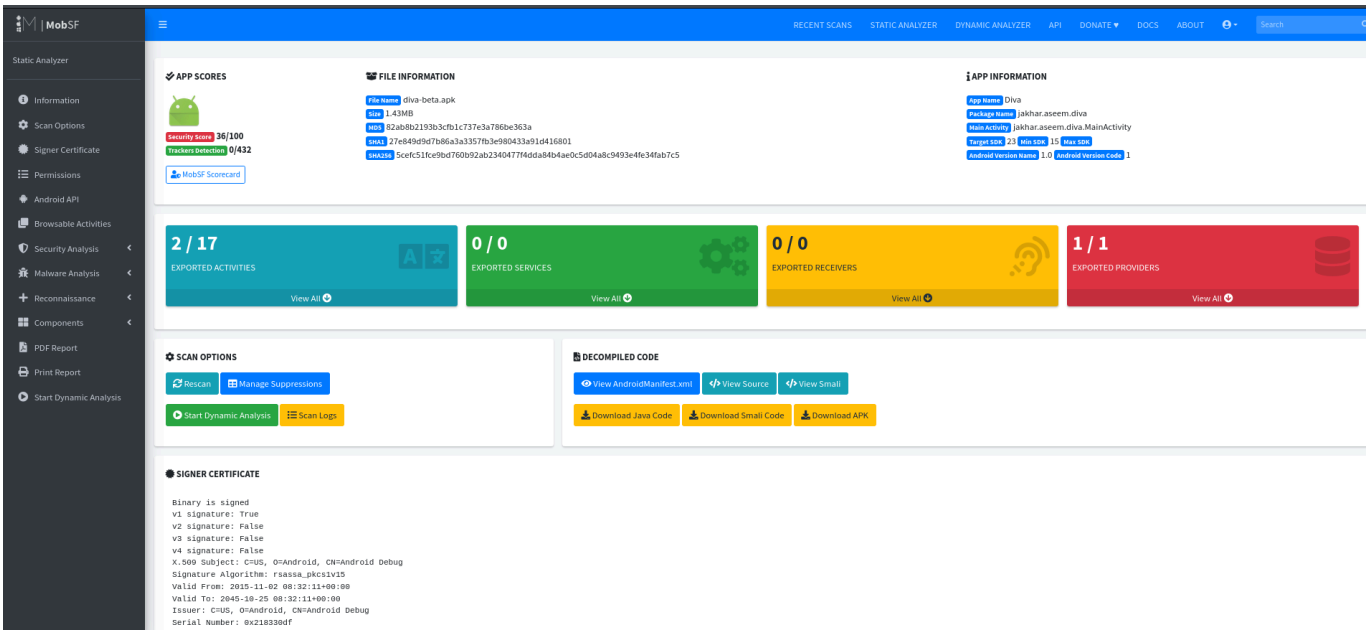
Abrimos un navegador y vamos a la url: `http://127.0.0.1:8000` Las credenciales por defecto de MobSF son:

```
usuario: mobsf
contraseña: mobsf
```

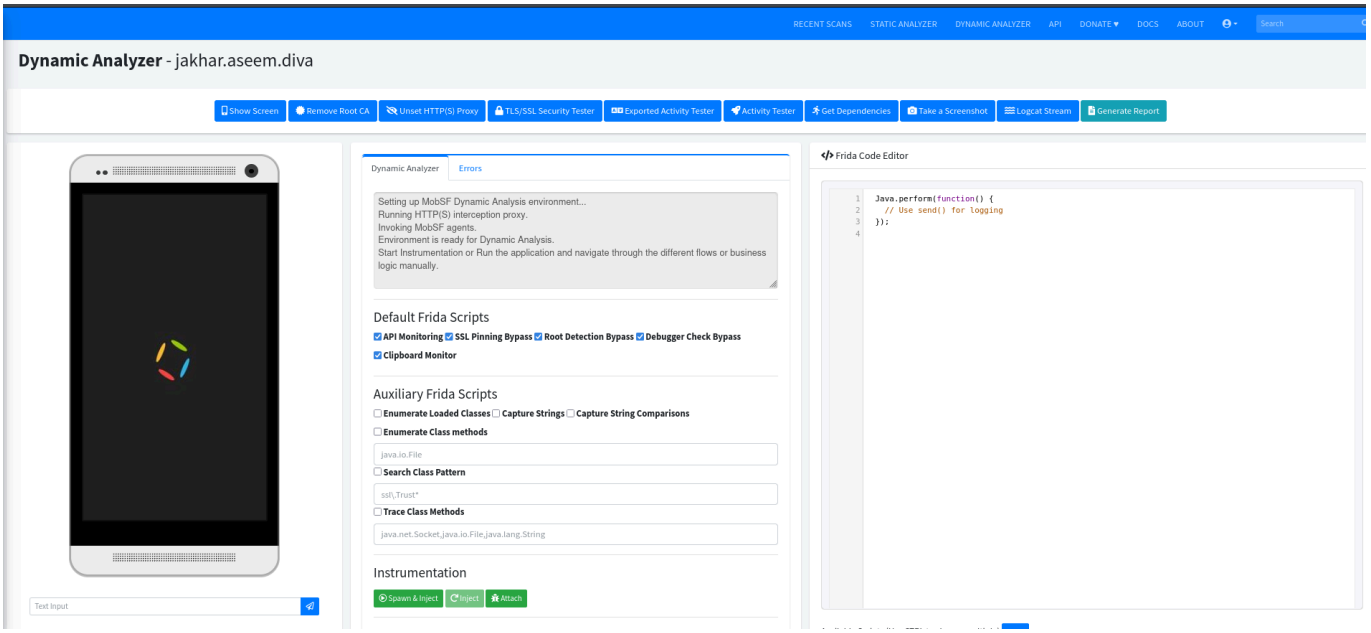


2.5 Ejemplo de Análisis Dinámico con MobSF

Instalado el Framework MobSF, ahora comprobaremos su funcionamiento con la aplicación `diva-beta.apk`.



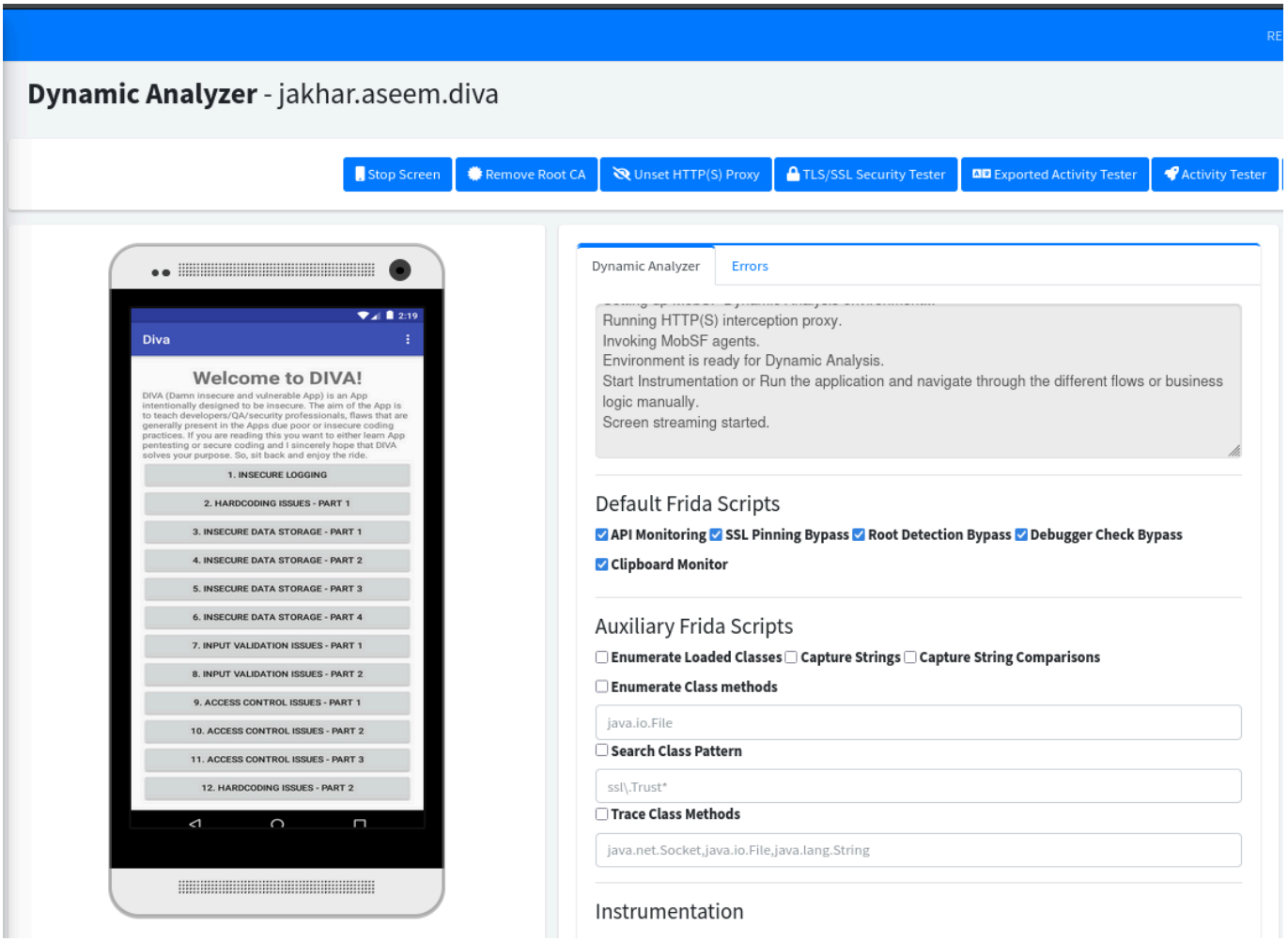
Pulsamos `Start Dynamic Analysis`:



donde:

- Ya nos aparecen las opciones que:
  - `Show Screen`.
  - `Exported Activity Tester`.
  - `Activity Tester`.
  - `Take a Screenshot`.
  - `Logcat Stream`.

Pulsamos en `Show Screen`:



Teniendo la app DIVA en la pantalla virtual, pulsamos en el framework MobSF `Logcat Stream` e interactuamos con varios menús de la app:

```
04-23 13:36:12.300 623 672 I PackageManager.DexOptimizer: Running dexopt
(dex2oat) on: /data/app/vmdl1101823754.tmp/base.apk pkg=jakhar.aseem.diva
isa=x86 vmSafeMode=false debuggable=true target-filter=interpret-only
oatDir = /data/app/vmdl1101823754.tmp/oat sharedLibraries=null\n'
04-23 13:36:45.427 623 953 I ActivityManager: START u0
{act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER]
flg=0x10200000 cmp=jakhar.aseem.diva/.MainActivity bnds=[12,514][121,704]
(has extras)} from uid 10049 on display 0\n'
04-23 13:36:45.443 623 936 I ActivityManager: Start proc
1961:jakhar.aseem.diva/u0a65 for activity
jakhar.aseem.diva/.MainActivity\n'
04-23 13:36:45.664 623 645 I ActivityManager: Displayed
jakhar.aseem.diva/.MainActivity: +232ms\n'
04-23 14:14:16.387 623 638 I ActivityManager: Force stopping
jakhar.aseem.diva appid=10065 user=-1: deletePackageX\n'
04-23 14:14:16.387 623 638 I ActivityManager: Killing
1961:jakhar.aseem.diva/u0a65 (adj 0): stop jakhar.aseem.diva\n'
04-23 14:14:16.387 623 638 W ActivityManager: Force removing
ActivityRecord{6f822cb u0 jakhar.aseem.diva/.MainActivity t5}: app died, no
saved state\n'
04-23 14:14:16.433 623 672 I ActivityManager: Force stopping
```

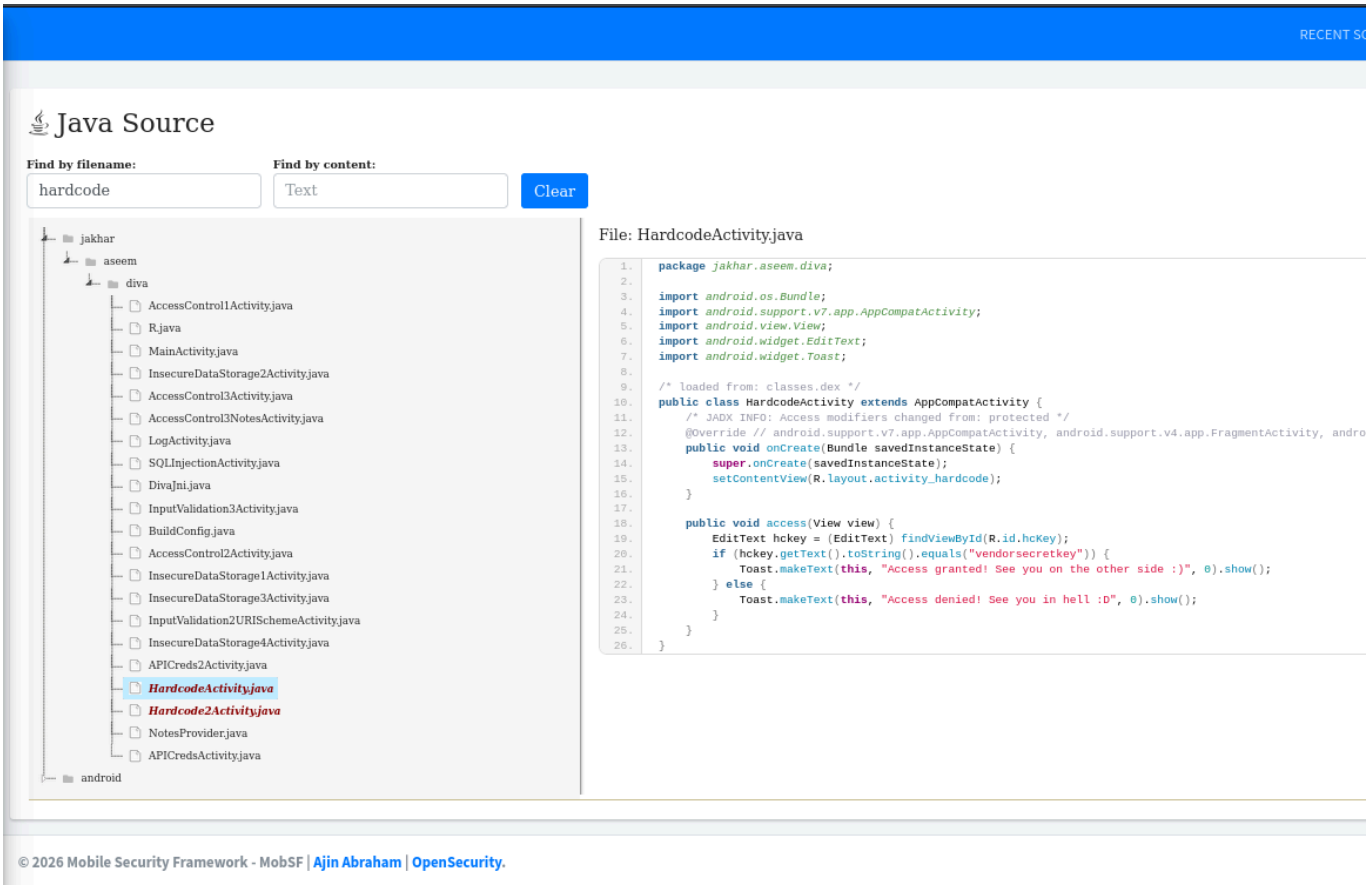
```
jakhar.aseem.diva appid=10065 user=0: pkg removed\n'
04-23 14:14:16.453 1499 2320 D VoicemailCleanupService: Cleaning up data
for package: jakhar.aseem.diva\n'
04-23 14:14:16.601 623 672 I PackageManager.DexOptimizer: Running dexopt
(dex2oat) on: /data/app/vmdl114072063.tmp/base.apk pkg=jakhar.aseem.diva
isa=x86 vmSafeMode=false debuggable=true target-filter=interpret-only
oatDir = /data/app/vmdl114072063.tmp/oat sharedLibraries=null\n'
04-23 14:18:04.602 623 1351 I ActivityManager: START u0
{act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER]
flg=0x10200000 cmp=jakhar.aseem.diva/.MainActivity bnds=[12,514][121,704]
(has extras)} from uid 10049 on display 0\n'
04-23 14:18:04.618 623 936 I ActivityManager: Start proc
2453:jakhar.aseem.diva/u0a66 for activity
jakhar.aseem.diva/.MainActivity\n'
04-23 14:18:04.797 623 645 I ActivityManager: Displayed
jakhar.aseem.diva/.MainActivity: +190ms\n'
04-23 14:21:14.332 623 954 I ActivityManager: START u0
{cmp=jakhar.aseem.diva/.HardcodeActivity} from uid 10066 on display 0\n'
04-23 14:21:14.430 623 645 I ActivityManager: Displayed
jakhar.aseem.diva/.HardcodeActivity: +93ms\n'
04-23 14:22:04.001 623 953 I ActivityManager: START u0
{cmp=jakhar.aseem.diva/.HardcodeActivity} from uid 10066 on display 0\n'
04-23 14:22:04.070 623 645 I ActivityManager: Displayed
jakhar.aseem.diva/.HardcodeActivity: +67ms\n'
```

donde vemos:

- Que el sistema realiza la optimización del APK (`dexopt / dex2oat`) durante la instalación o reinstalación de la aplicación DIVA.
- El arranque de la aplicación mediante la actividad principal `jakhar.aseem.diva/.MainActivity`, invocada con la acción `MAIN` y la categoría `LAUNCHER`.
- La creación del proceso de la app `(Start proc ... jakhar.aseem.diva)` cuando Android lanza DIVA.
- La actividad principal se muestra correctamente en pantalla `(Displayed ... MainActivity)`, lo que confirma que la app se ejecuta en el entorno virtual.
- Un evento de parada forzada y eliminación del paquete `(Force stopping, Killing, pkg removed)`, lo que indica que la aplicación fue desinstalada o reinstalada durante las pruebas. Cosa que es cierta ya que tuve algunos problemas en su instalación.
- La optimización e instalación del APK tras esa reinstalación.
- Un nuevo arranque de la actividad principal, confirmando que la aplicación vuelve a ejecutarse correctamente después de reinstalarla.
- La apertura de la actividad `jakhar.aseem.diva/.HardcodeActivity`, lo que demuestra que al interactuar con los menús de DIVA se accede a funcionalidades internas concretas.
- `HardcodeActivity` se abre en dos ocasiones, lo que confirma que la interacción del usuario queda reflejada en `Logcat` y que el entorno dinámico permite seguir la navegación entre actividades.

Resumiendo, el registro confirma que el entorno preparado con `Genymotion + ADB + MobSF` permite observar en tiempo real la instalación, ejecución, navegación y eventos relevantes de una aplicación Android.

En MobSF, hacemos una búsqueda de los ficheros de la app que tengan el nombre `hardcode`: Vemos el fichero `HardcodeActivity.java`, dentro del método access(View view):

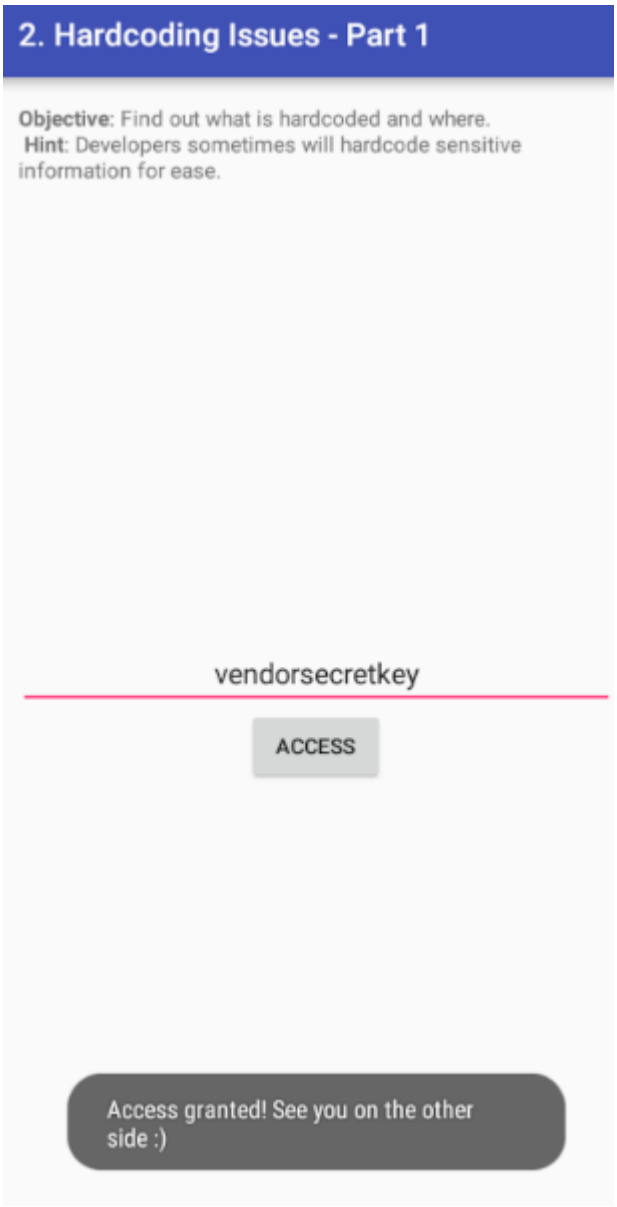


En el fichero `HardcodeActivity.java` se ve esta comparación:

```
if (hkey.getText().toString().equals("vendorsecretkey"))
```

donde:

- Toma el texto introducido por el usuario en `EditText hkey`.
- Lo convierte a cadena.
- Lo compara directamente con un valor fijo: `vendorsecretkey`.
- Lo que implica que la clave válida está hardcodeada en el código fuente: `vendorsecretkey`.
- Si el valor coincide: muestra el mensaje `Access granted! See you on the other side :)`.
- Si no coincide: muestra `Access denied! See you in hell :D`.



donde:

- Esta captura demuestra que el entorno de análisis estático funciona correctamente y que ha permitido identificar una vulnerabilidad típica: credenciales o claves hardcodeadas en la aplicación. En este caso, la key embebida es `vendorsecretkey`, localizada en la clase `HardcodeActivity`.

2.6 Conexión de Burp Suite con Genymotion

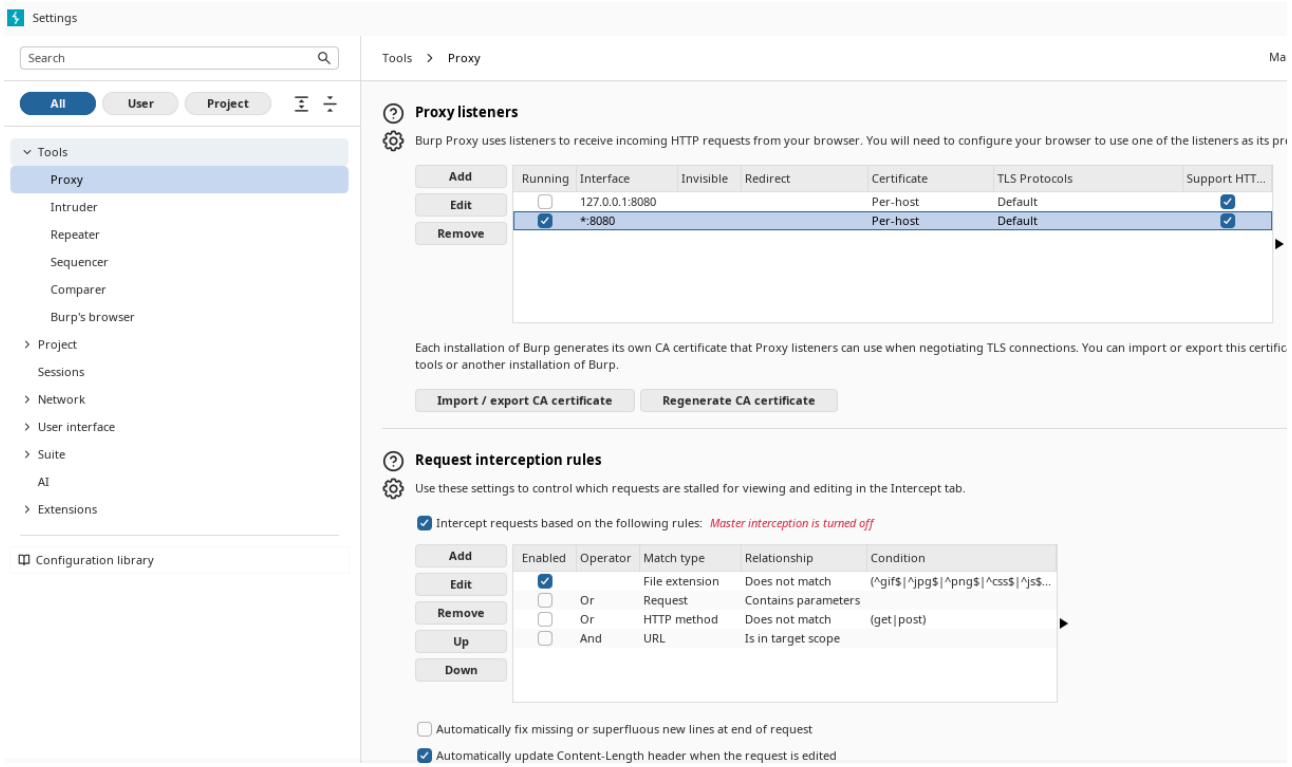
Vamos a conectar Burp Suite con Genymotion en el dispositivo `Android 15.0 - Api 35` que instalamos en el apartado anterior. La guía oficial de Genymotion recomienda:

- Crear un listener en Burp Suite para escuchar.
- Instalar el certificado CA en Android.
- Poner el proxy global del emulador apuntando a Burp.
- Además, recomiendan hacerlo por ADB en vez de tocar el proxy desde Ajustes de Android, porque es más fiable.
- También indican que podemos usar el ADB del SDK o el ADB integrado de Genymotion.

A) Crear un listener en el puerto 8080

En Burp Suite:

- Vamos a `Proxy`.
- Entramos en `Proxy Settings`.
- Creamos un listener para que escuche en `puerto 8080`, en todas las interfaces:



B) Instalar el certificado CA en Android

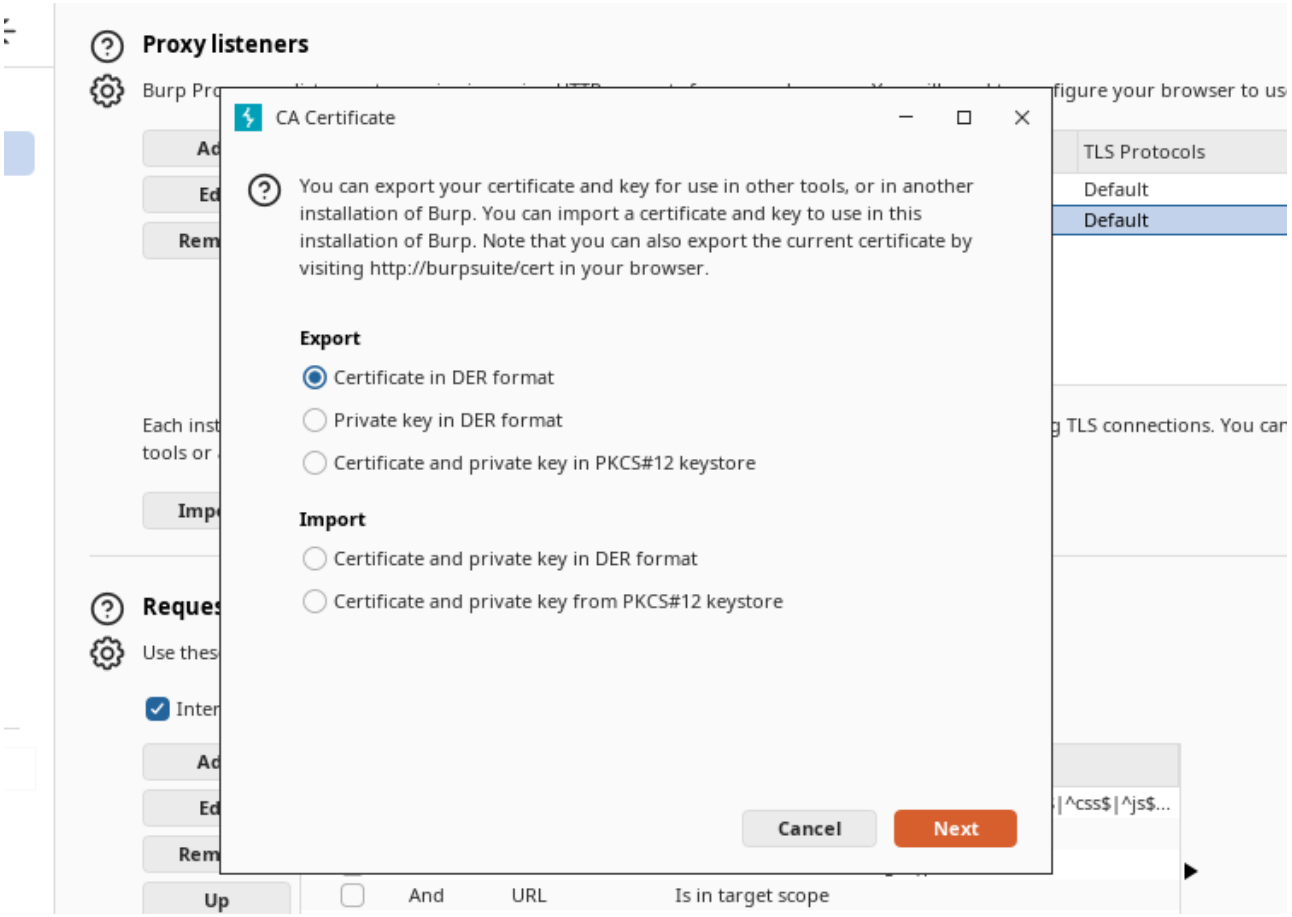
Burp Suite actúa como intermediario en las conexiones HTTPS. Cuando una app o el navegador Android acceden a una web segura:

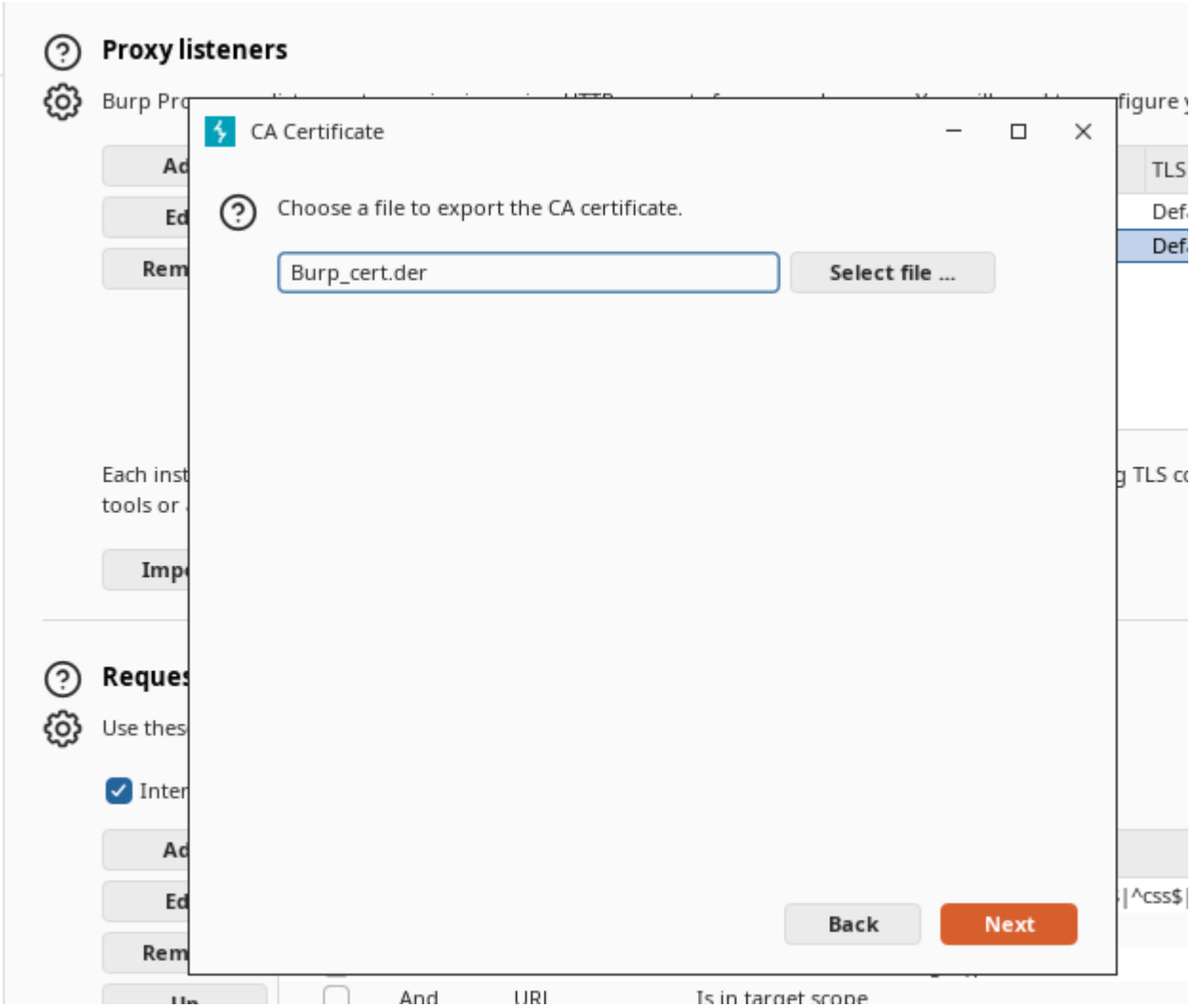
- El dispositivo espera recibir un certificado TLS válido del servidor,
- pero al interceptar Burp, quien presenta el certificado realmente es Burp Suite, no el servidor original,
- por eso Android, si no confía en la CA de Burp Suite, detecta ese certificado como no confiable y bloquea la conexión.

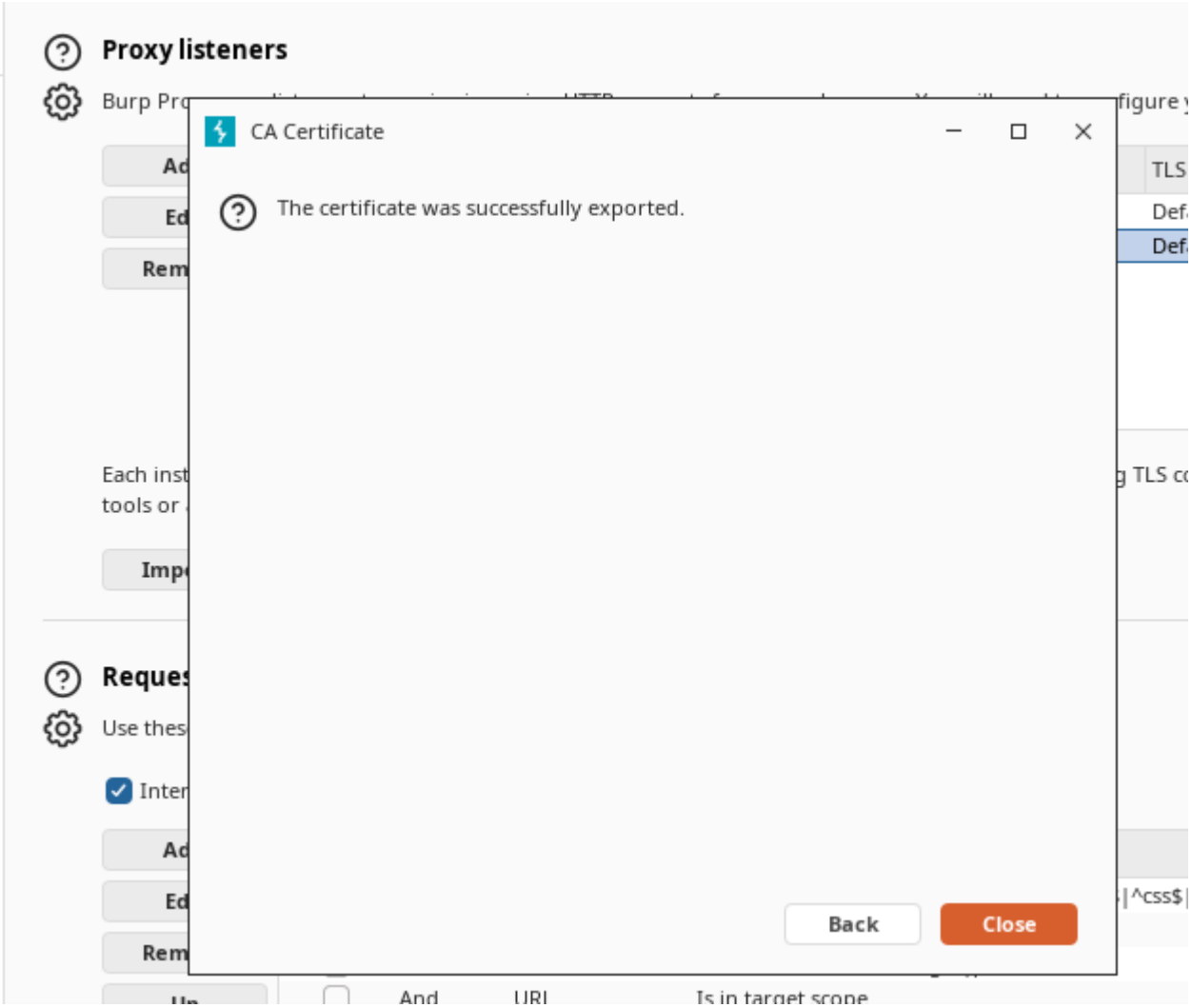
Es por ello que tenemos que instalar el certificado de usuario de Burp Suite en el dispositivo Android para que el sistema confíe en Burp Suite como autoridad certificadora y permita interceptar tráfico HTTPS sin errores de certificado.

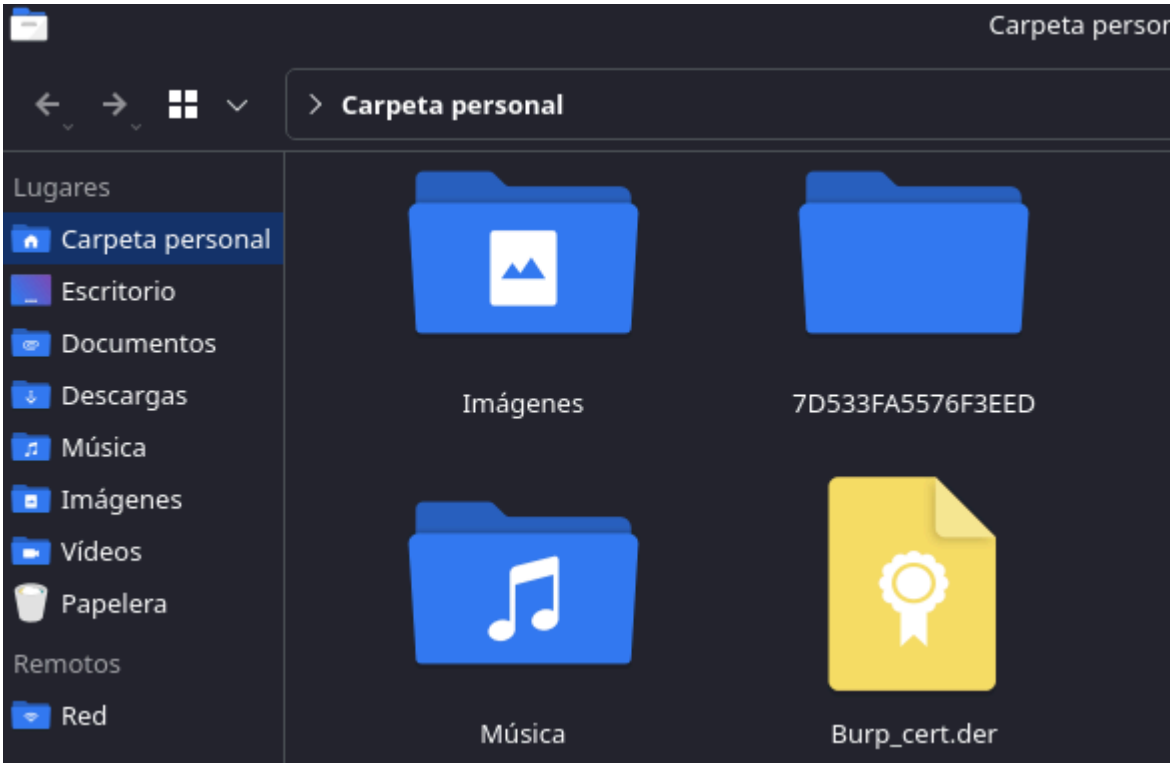
Para instalar el certificado CA en Android, en Burp Suite:

- Vamos a `Proxy`.
- Entramos en `Proxy Settings`.
- Pulsamos el botón `Import/export CA certificate` y exportamos la `CA` en formato `DER`. La guía usa como ejemplo `Burp_cert.der`:




Proxy listeners
CA Certificate
Choose a file to export the CA certificate.
Burp_cert.der
Select file ...
Back
Next

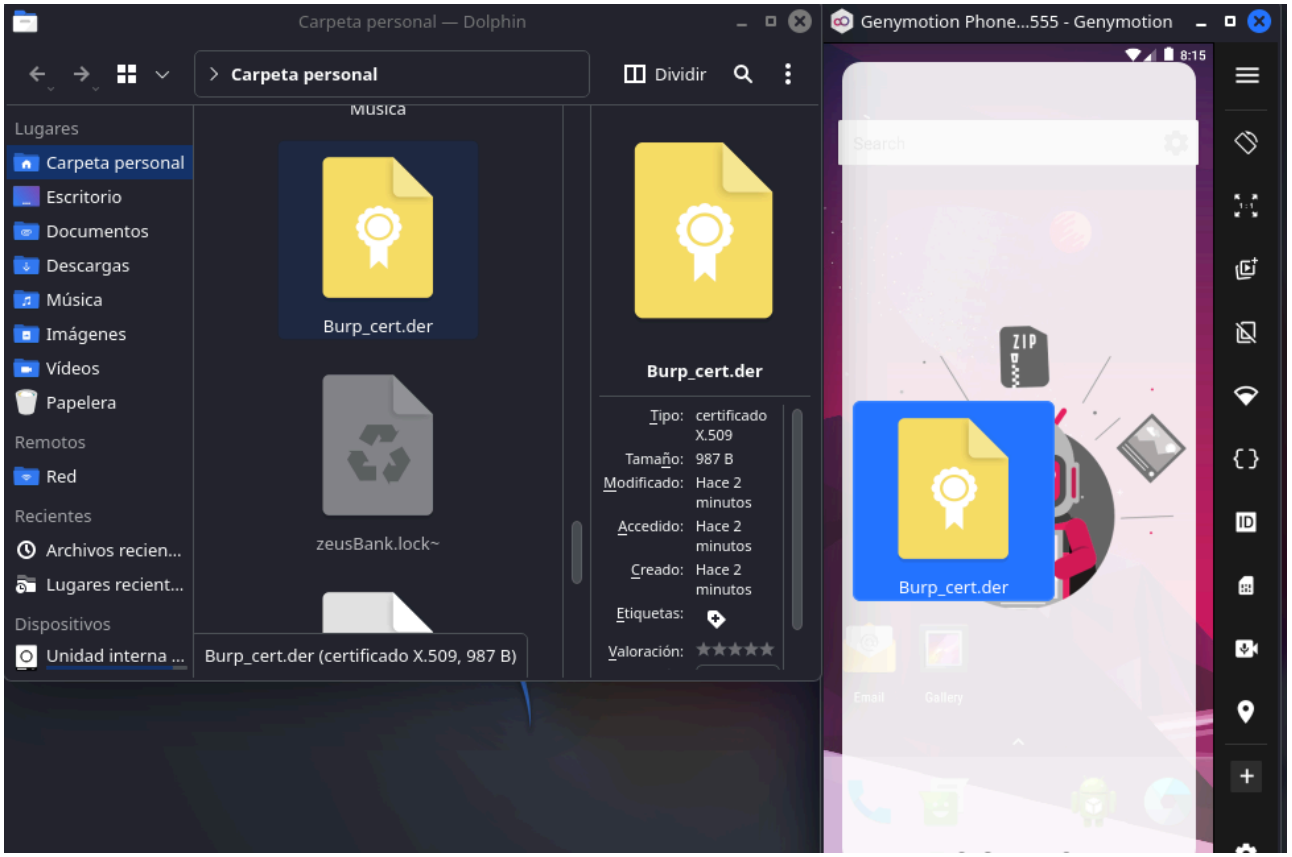

Proxy listeners
CA Certificate
The certificate was successfully exported.
Back
Close

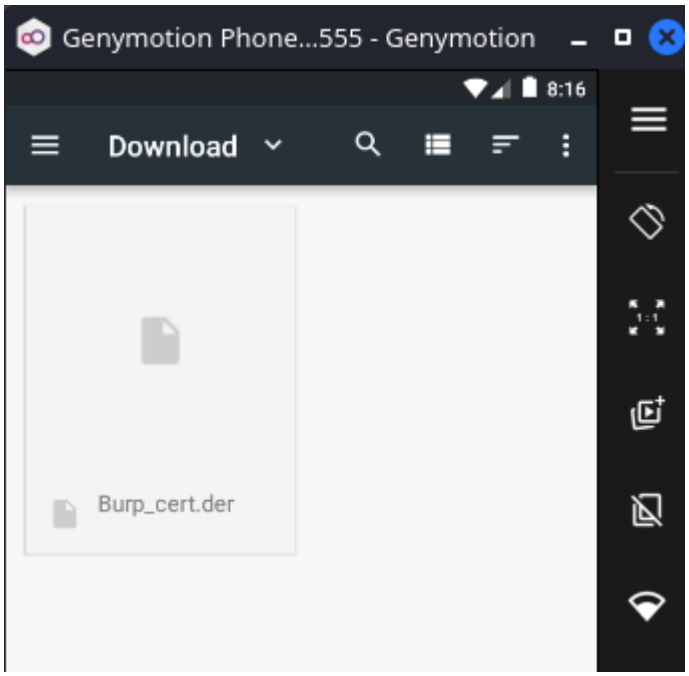

Carpeta personal
Lugares
Carpeta personal
Escritorio
Documentos
Descargas
Música
Imágenes
Vídeos
Papelera
Remotos
Red
Imágenes
7D533FA5576F3EED
Música
Burp_cert.der

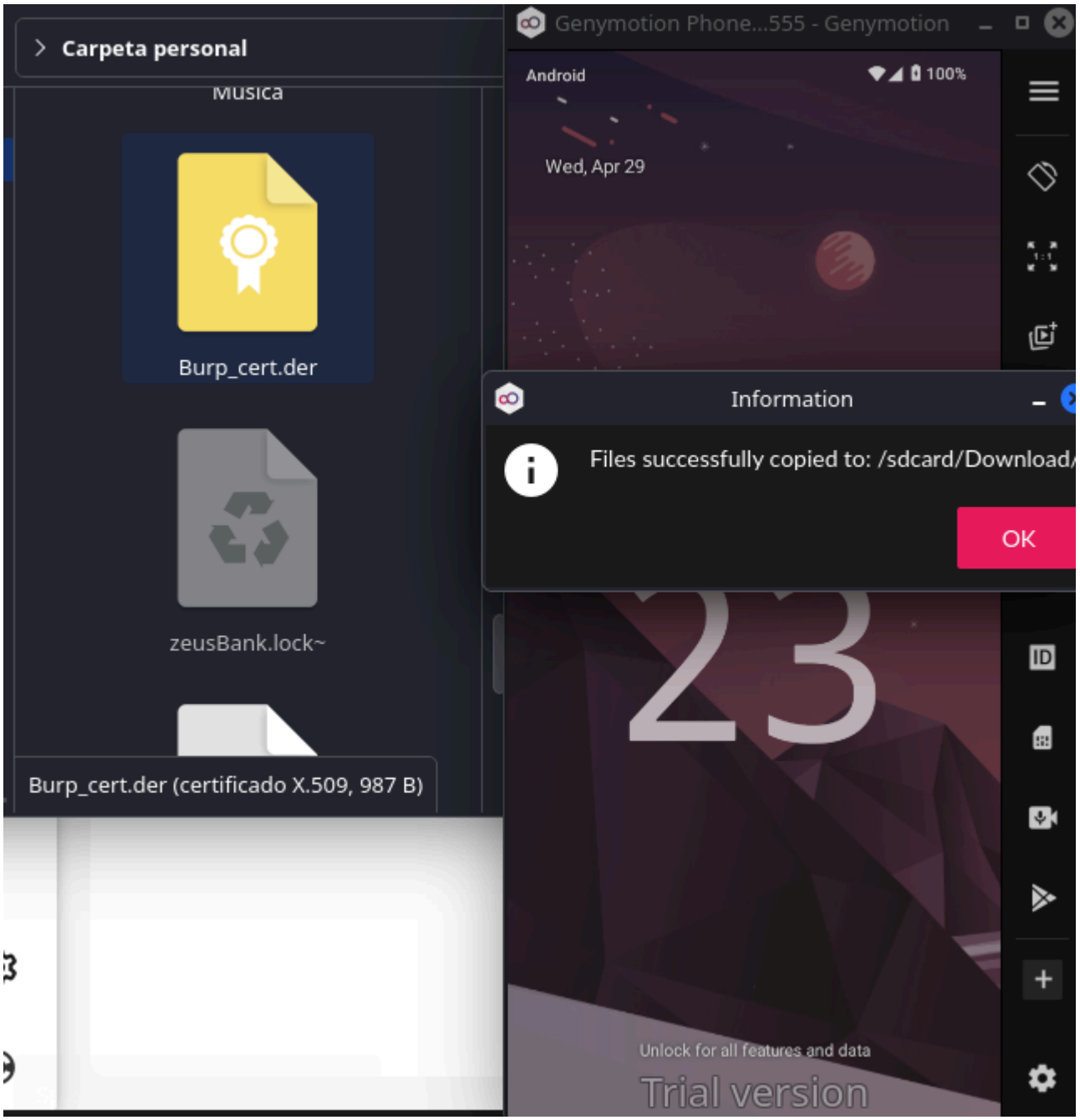
C) Instalar el certificado de Burp Suite en el Android de Genymotion

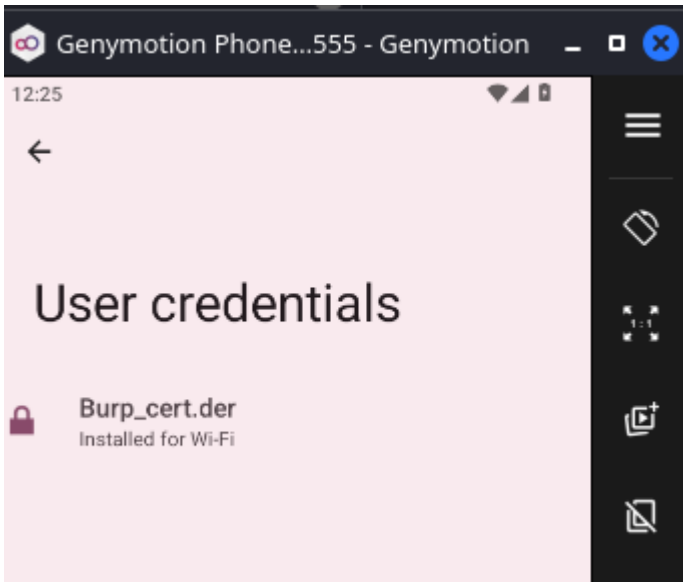
Con el dispositivo android arrancado:

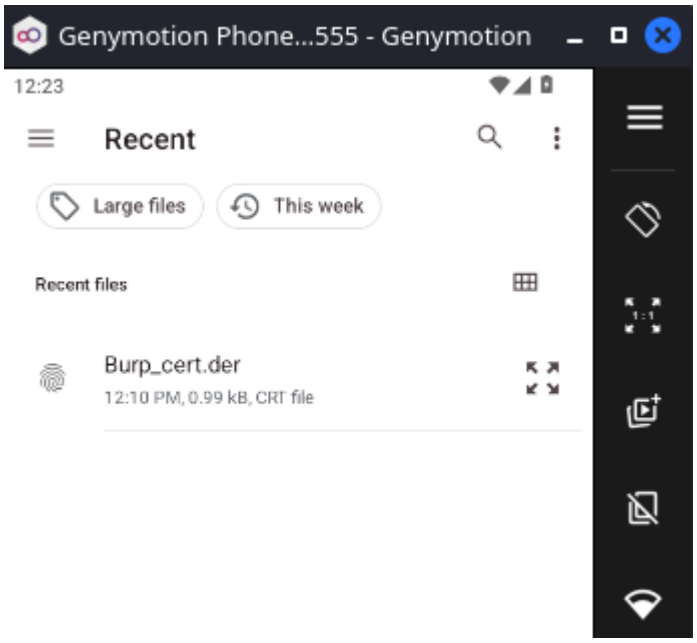
- Arrastramos `Burp_cert.der` a la ventana del dispositivo.
- En Android, buscamos `Install a certificate`.
- Entramos en `Install certificates from SD Card`.
- Elegimos `CA certificate`.
- Abrimos `/sdcard/Download/Burp_cert.der` y lo instalamos.
- Después podemos comprobarlo en `Trusted credentials`, donde debería aparecer en la pestaña `USER`.

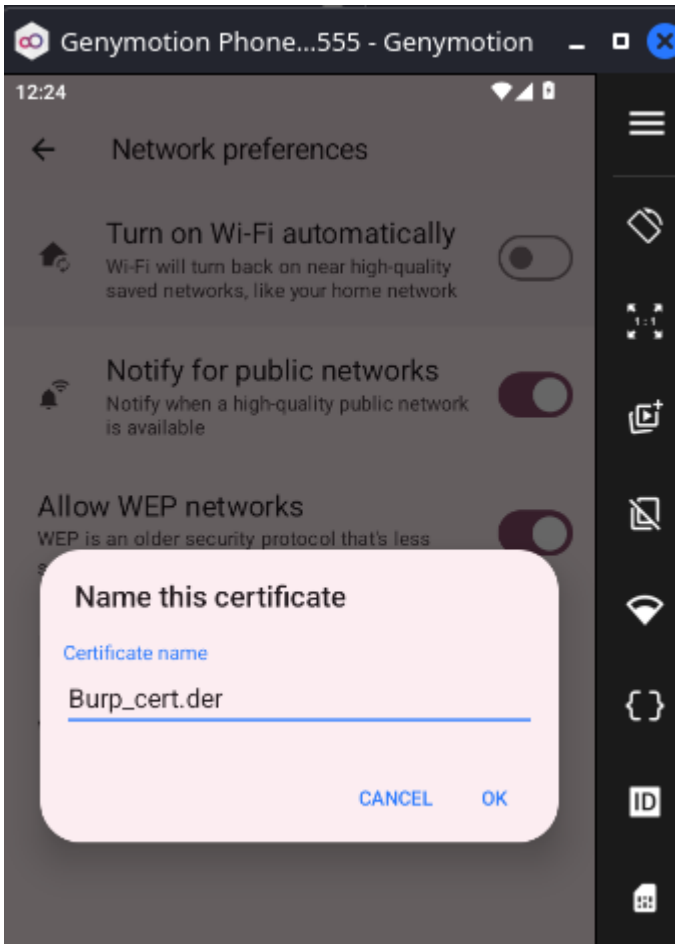



Genymotion Phone...555 - Genymotion
8:16
Download
Burp_cert.der


Carpeta personal
Música
Burp_cert.der
zeusBank.lock~
Burp_cert.der (certificado X.509, 987 B)
Genymotion Phone...555 - Genymotion
Android
100%
Wed, Apr 29
Information
Files successfully copied to: /sdcard/Download/
OK
23
Unlock for all features and data
Trial version


Genymotion Phone...555 - Genymotion
12:25
User credentials
Burp_cert.der
Installed for Wi-Fi


Genymotion Phone...555 - Genymotion
12:23
Recent
Large files
This week
Recent files
Burp_cert.der
12:10 PM, 0.99 kB, CRT file


Genymotion Phone...555 - Genymotion
12:24
Network preferences
Turn on Wi-Fi automatically
Wi-Fi will turn back on near high-quality saved networks, like your home network
Notify for public networks
Notify when a high-quality public network is available
Allow WEP networks
WEP is an older security protocol that's less
Name this certificate
Certificate name
Burp_cert.der
CANCEL
OK

D) Poner el proxy global del emulador apuntando a Burp Suite

El comando general es:

```
adb shell settings put global http_proxy <IP_DEL_HOST>:8080
```

donde:

- Genymotion explica que `<IP_DEL_HOST>` es la IP de la máquina donde corre Burp y `8080` es el puerto del listener.
- `<IP_DEL_HOST>` es la dirección que el Android virtual debe usar para llegar a la máquina donde corre Burp. Pero esa dirección no siempre es la IP LAN del host, cambia según el backend de Genymotion. Genymotion lo documenta así:
  - Con VirtualBox, el dispositivo puede usar 10.0.3.2, que es un alias de VirtualBox al loopback del host (127.0.0.1), así que un proxy local en el host se alcanza como 10.0.3.2:8080.
  - Con QEMU, no existe ese loopback alias, así que Genymotion recomienda poner el proxy global en localhost:3333 dentro del dispositivo y hacer adb reverse tcp:3333 tcp:8080 hacia el host.
- Desde ese momento, el tráfico de Internet del dispositivo debería redirigirse a Burp Suite.

Resumiendo: La dirección usada por el dispositivo Android para alcanzar Burp Suite depende del hipervisor empleado por Genymotion. Con VirtualBox puede usarse 10.0.3.2:8080, mientras que con QEMU es necesario configurar el proxy a localhost:3333 y redirigir el tráfico con adb reverse tcp:3333 tcp:8080.

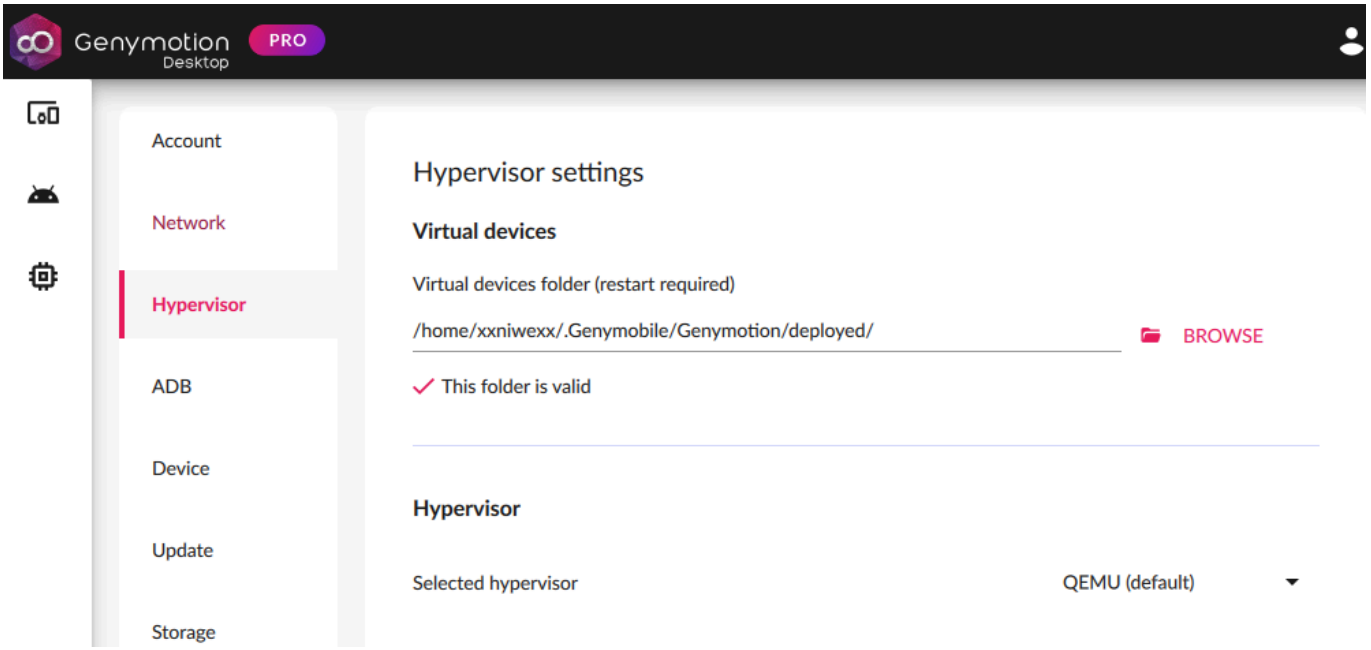
Verificamos la regla de redirección activa y el serial real del dispositivo:



donde:

- `adb reverse --list` muestra la regla `host-14 tcp:3333 tcp:8080`, lo que indica que la redirección hacia Burp Suite está configurada.
- `adb devices -l` muestra que el identificador válido del dispositivo para usar con `adb -s` es `127.0.0.1:6555`.

Verificamos el hipervisor que usa el dispositivo android 15:



donde:

- Comprobamos que el dispositivo android usa en el hipervisor: `QEMU`.
- Como usa QEMU debemos seguir el esquema: proxy del dispositivo a `localhost:3333` y `adb reverse tcp:3333 tcp:8080`.

Volvemos a configurar el acceso al proxy mediante comandos ADB:

```
/home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 shell
settings put global http_proxy :0

/home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 reverse --
remove-all

/home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 shell
settings put global http_proxy localhost:3333

/home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 reverse
tcp:3333 tcp:8080

/home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 shell
settings get global http_proxy

/home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 reverse --
list
```

```
┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ /home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 shell settings put global http_proxy :0

┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ /home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 reverse --remove-all

┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ /home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 shell settings put global http_proxy localhost:3333

┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ /home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 reverse tcp:3333 tcp:8080
3333

┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ /home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 shell settings get global http_proxy
localhost:3333

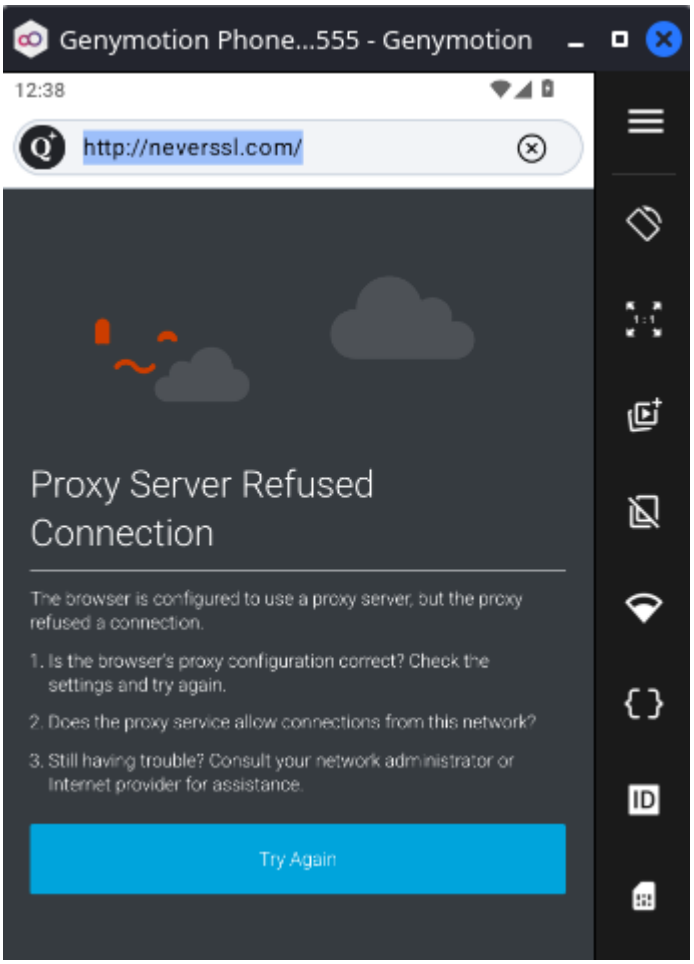
┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ /home/xxniwexx/Descargas/genymotion/tools/adb -s 127.0.0.1:6555 reverse --list
host-14 tcp:3333 tcp:8080
```

donde:

- `adb -s 127.0.0.1:6555 shell settings put global http_proxy :0`: Elimina cualquier configuración previa de proxy global en el dispositivo Android virtual.
- `adb -s 127.0.0.1:6555 reverse --remove-all`: Borra todas las reglas de redirección adb reverse que estuvieran activas.
- `adb -s 127.0.0.1:6555 shell settings put global http_proxy localhost:3333`: Configura el dispositivo para que use como proxy la dirección localhost en el puerto 3333.
- `adb -s 127.0.0.1:6555 reverse tcp:3333 tcp:8080`: Crea una redirección entre el puerto 3333 del dispositivo virtual y el puerto 8080 del host, donde está escuchando BurpSuite.
- `adb -s 127.0.0.1:6555 shell settings get global http_proxy`: Muestra la configuración actual del proxy global del dispositivo, para verificar que se ha aplicado correctamente.
- `adb -s 127.0.0.1:6555 reverse --list`: Lista las reglas adb reverse activas, para comprobar que la redirección hacia Burp Suite está configurada.

E) Probamos la interceptación del tráfico

Intentamos acceder a una web como por ejemplo: `http://neverssl.com`, pero obtenemos un error de proxy server:

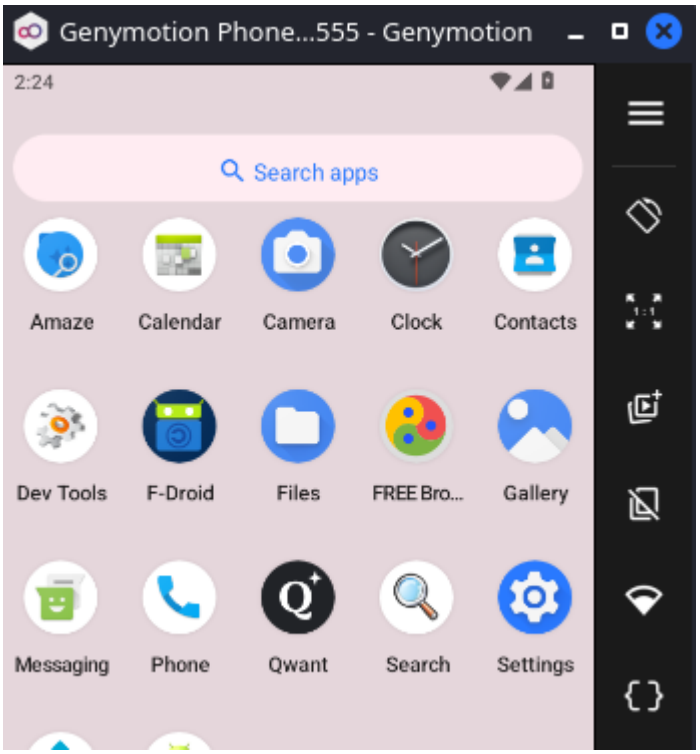


Después de seguir distintas vías pero no conseguir solventar el error del Proxy Server, vamos a instalar un nuevo Browser. Las pruebas para acceder a una web las hemos realizado en una aplicación que estaba instalada en el dispositivo android llamada `Buscador`. Por ello instalaremos un navegador gratuito:

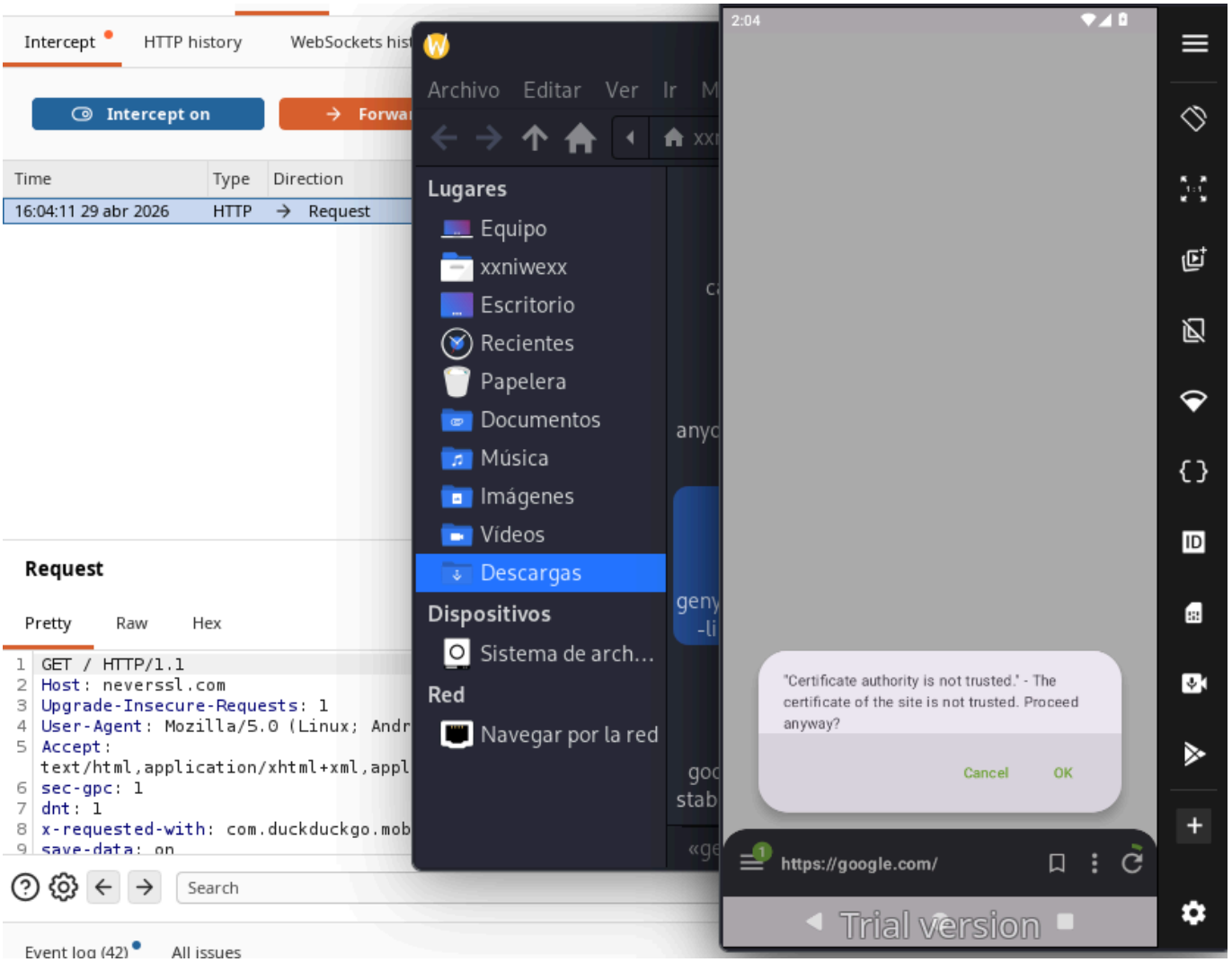
```
┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ adb shell pm list packages | grep -i -E 'firefox|chrome|browser'

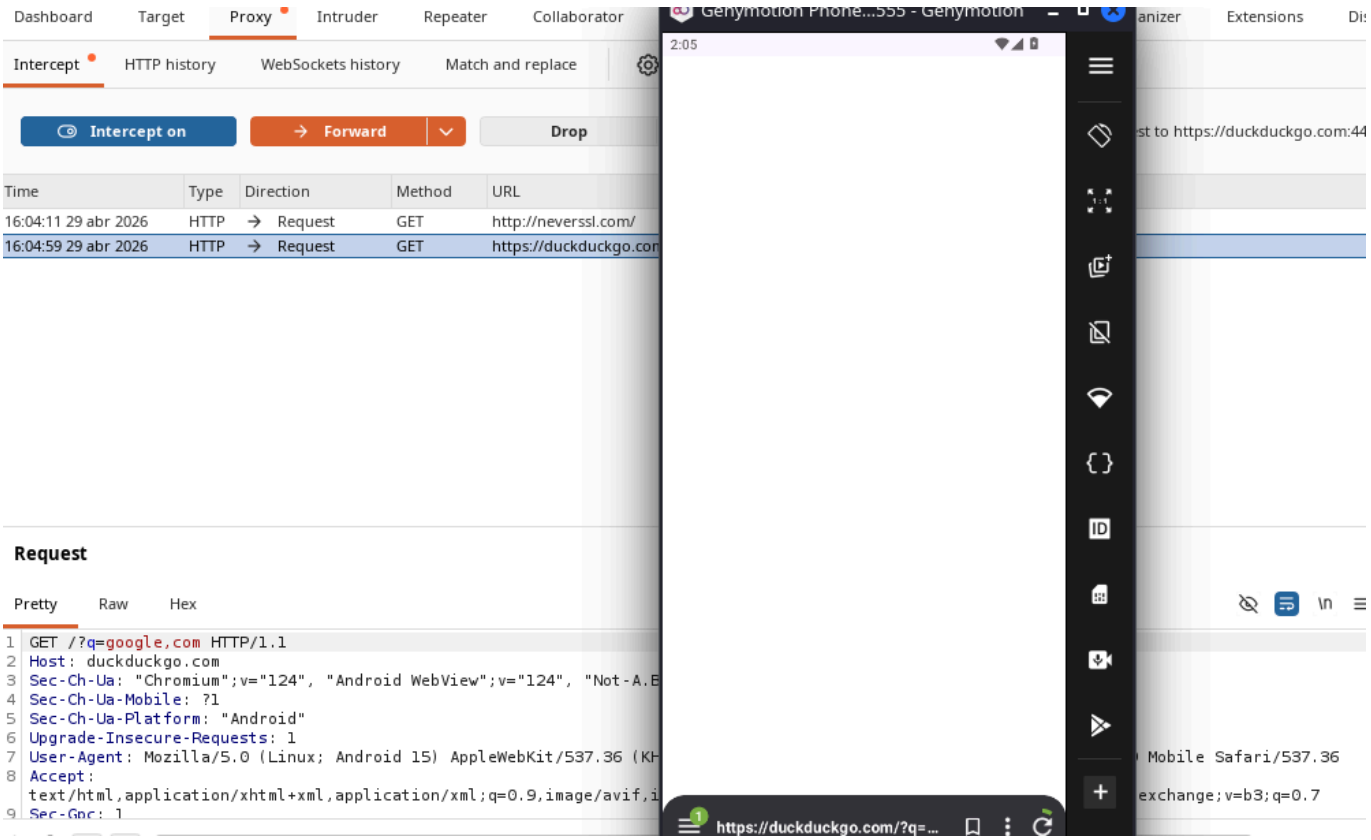
┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ adb install /home/xxniwexx/Descargas/F-Droid.apk
Performing Streamed Install
Success

┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ adb install /home/xxniwexx/Descargas/org.woheller69.browser_3500.apk
Performing Streamed Install
Success
```



Abrimos el navegador instalado y probamos a navegar por alguna web: Ya se empieza a interceptar el tráfico Burp Suite:

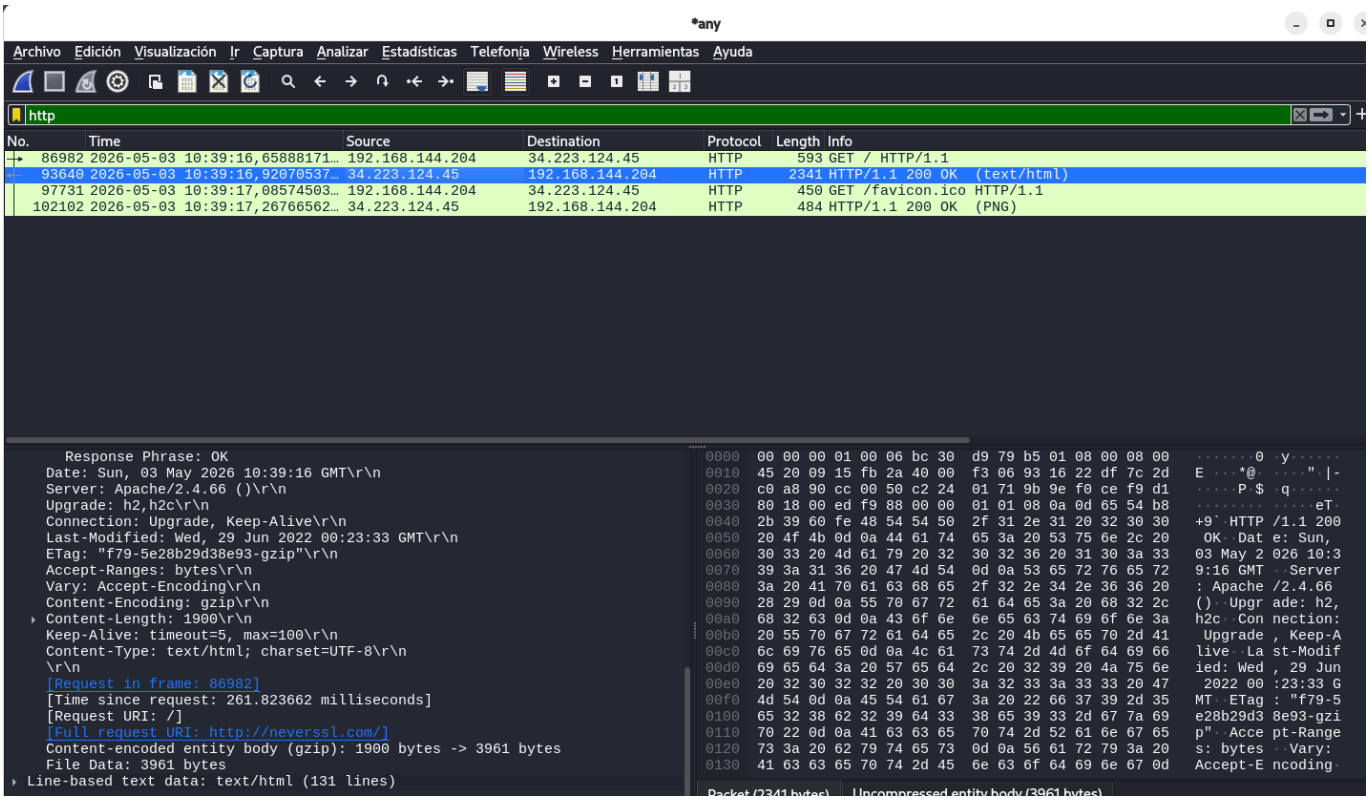




Resumiendo: Para validar la interceptación de tráfico, se configuró el proxy global del dispositivo virtual para redirigir las conexiones hacia Burp Suite. Además, se revisó el hipervisor utilizado por el dispositivo Android en Genymotion, comprobándose que estaba configurado con QEMU. A partir de ello, se reconfiguró el acceso al proxy mediante comandos `ADB`, estableciendo el proxy global en `localhost:3333` y creando una redirección con `adb reverse tcp:3333 tcp:8080`, de forma que el tráfico del emulador se reenviara correctamente al puerto `8080` de Burp Suite en el host. Tras varias pruebas, se comprobó que el navegador por defecto del emulador no gestionaba correctamente la navegación a través del proxy, mostrando el error `Proxy Server Refused Connection`. Se instaló un navegador alternativo mediante `APK` y, con él, Burp Suite comenzó a interceptar correctamente la petición GET a neverssl.com. Esto confirma que la integración `Genymotion + proxy + Burp Suite` es correcta y que la incidencia estaba asociada al navegador integrado de la imagen Android utilizada.

2.7 Wireshark

Wireshark ya viene preinstalado en el host Kali. A diferencia de Burp Suite, Wireshark no actúa como proxy ni se conecta directamente a Genymotion, sino que captura los paquetes que circulan por las interfaces de red del host por las que pasa el tráfico del emulador. Al abrir la herramienta se identifican varias interfaces disponibles para captura. Para realizar una validación general del entorno se seleccionará la interfaz `any`, ya que permite observar tráfico de todas las interfaces del host sin necesidad de determinar previamente por cuál de ellas circula la comunicación del dispositivo virtual. Una vez iniciada la captura, se arranca el dispositivo Android 15 en Genymotion y se accede desde el navegador del emulador a la web `http://neverssl.com`, generando así tráfico HTTP que puede ser observado y analizado en Wireshark.



donde:

- Vemos que Wireshark está capturando correctamente tráfico HTTP generado durante la navegación a `neverssl.com`.

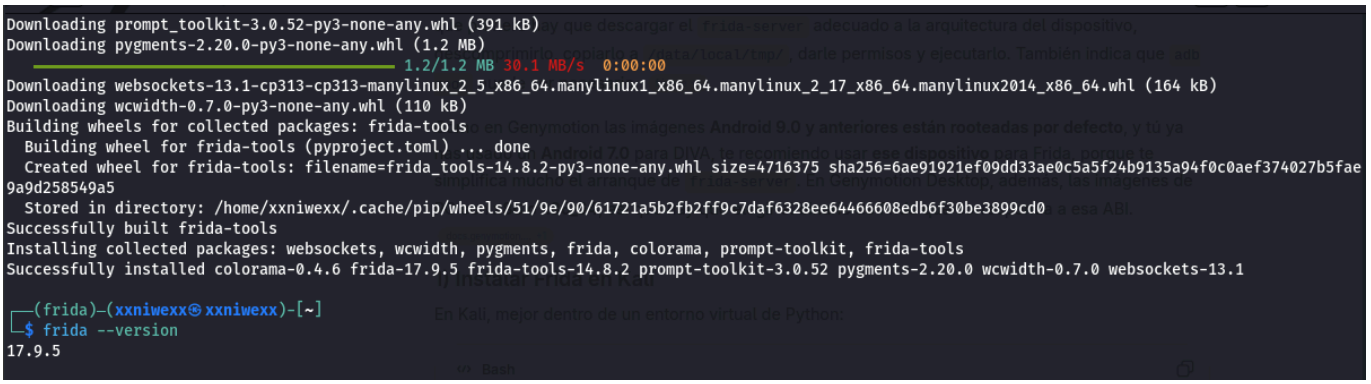
La captura de Wireshark confirma que el entorno permite monitorizar el tráfico HTTP generado por el dispositivo Android virtual. Aplicando el filtro `http`, se observan las peticiones `GET /` y `GET /favicon.ico` hacia `neverssl.com`, junto con sus respuestas `HTTP/1.1 200 OK`. En el panel de detalle se aprecia además información del protocolo, como la `URI` completa solicitada y cabeceras de la respuesta (`Server`, `Content-Type`, `Content-Encoding`). Esto valida que la infraestructura configurada permite inspeccionar tráfico de red de la aplicación o del navegador del emulador desde el host Kali.

2.8 Instalación de Frida

Instalamos Frida dentro de un entorno virtual de Python:

```
python3 -m venv ~/venvs/frida
source ~/venvs/frida/bin/activate
python -m pip install --upgrade pip
python -m pip install frida-tools
```

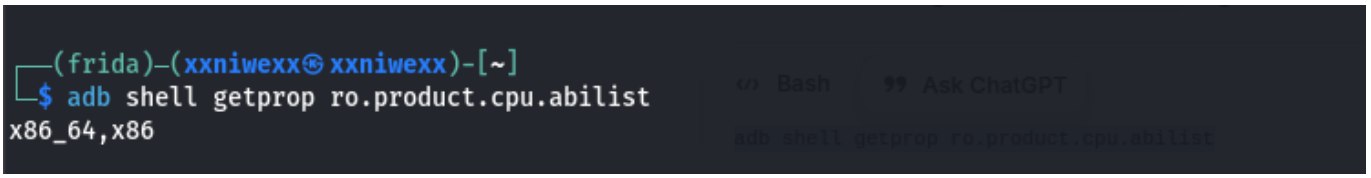
Comprobamos que se instaló correctamente:



Con un dispositivo android funcionando (elegimos android 15), comprobamos que adb lo detecta:



Comprobamos la arquitectura del Android: Frida especifica que se debe elegir el `frida-server` según la `ABI` del dispositivo.



donde:

- Obtenemos `x86_64,x86`, lo que indica que el dispositivo Android virtual soporta las arquitecturas `x86_64` y `x86`, siendo `x86_64` la `ABI` principal.

Activamos root en Genymotion: Como estamos usando un Android 15 con Genymotion Desktop, este android viene sin root por defecto, mientras que las imágenes 11.0 y anteriores ya están rooteadas.

```
adb shell getprop persist.sys.root_access
adb shell setprop persist.sys.root_access 3
adb shell getprop persist.sys.root_access
```



donde:

- La captura muestra la activación del acceso root en el dispositivo Android virtual. Inicialmente, la propiedad `persist.sys.root_access` devolvía el valor `0`, indicando que el acceso root no estaba habilitado. Tras ejecutar `adb shell setprop persist.sys.root_access 3`, el valor pasó a `3`, confirmando la activación del acceso root.

Verificar acceso root por ADB:

```
adb root
adb shell id
```



donde:

- root activado: persist.sys.root_access=3
- ADB como root: uid=0(root)

La captura muestra la validación del acceso root a través de `ADB`. Tras ejecutar `adb root`, el servicio `adbd` se reinició en modo privilegiado, tal y como indica el mensaje `restarting adbd as root`. Posteriormente, al ejecutar `adb shell id`, se obtiene `uid=0(root) gid=0(root)`, confirmando que la shell `ADB` opera con privilegios de superusuario.

Descargamos el frida-server correcto:

```
VERSION=$(frida --version)
cd ~/Descargas
wget "https://github.com/frida/frida/releases/download/$VERSION/frida-
server-$VERSION-android-x86_64.xz"
unxz -f "frida-server-$VERSION-android-x86_64.xz"
mv "frida-server-$VERSION-android-x86_64" frida-server
```

Lo subimos al Android 15 y lo arrancamos:

```
adb push ~/Descargas/frida-server /data/local/tmp/
adb shell 'chmod 755 /data/local/tmp/frida-server'
adb shell 'pkill frida-server 2>/dev/null || true'
adb shell '/data/local/tmp/frida-server >/dev/null 2>&1 &'
```

donde:

- `adb push ~/Descargas/frida-server /data/local/tmp/`: Copia el binario `frida-server` desde el host Kali al dispositivo Android virtual, dejándolo en la ruta `/data/local/tmp/`.
- `adb shell 'chmod 755 /data/local/tmp/frida-server`: Asigna permisos de ejecución al binario copiado, de forma que pueda lanzarse dentro del dispositivo.
- `adb shell 'pkill frida-server 2>/dev/null || true`: Finaliza cualquier instancia previa de `frida-server` que pudiera estar ejecutándose, evitando conflictos al arrancarlo de nuevo.
- `adb shell /data/local/tmp/frida-server >/dev/null 2>&1 &`: Intenta arrancar `frida-server` en segundo plano, redirigiendo su salida estándar y de error para que no se muestre en pantalla.



Comprobamos que Frida detecta correctamente el dispositivo Android y puede enumerar los procesos en ejecución:

```
└─$ frida-ps -U
 PID  Name
----  ------------------------------------------------------------------------
----------------------
2911  FREE Browser
1874  Files
1547  Settings
3424  adbd
2600  android.ext.services
 231  android.hardware.atrace@1.0-service
 500  android.hardware.audio.service
```

```
  501  android.hardware.authsecret@1.0-service
  668  android.hardware.biometrics.fingerprint@2.1-service
  502  android.hardware.bluetooth@1.1-service.sim
  504  android.hardware.camera.provider@2.4-service_64
  522  android.hardware.cas-service.example
  506  android.hardware.gatekeeper@1.0-service.software
  507  android.hardware.gnss@1.0-service
  512  android.hardware.graphics.allocator@2.0-service
  516  android.hardware.graphics.composer3-service.ranchu
  514  android.hardware.health-service.genymotion
  517  android.hardware.identity-service.example
  232  android.hardware.keymaster@4.1-service
  518  android.hardware.lights-service.example
  657  android.hardware.media.omx@1.0-service
  519  android.hardware.memtrack-service.example
  520  android.hardware.power-service.example
  521  android.hardware.power.stats-service.example
  515  android.hardware.sensors@1.0-service
  497  android.hidl.allocator@1.0-service
 2281  android.process.acore
 2029  android.process.media
  228  android.system.suspend-service
  527  audioserver
 1753  batteryd
  638  cameraserver
 1522  com.android.bluetooth
...
...
...
```

donde:

- `frida-ps -U` nos está devolviendo la lista de procesos del dispositivo Android, lo que confirma:
  - `Frida Tools` está bien instalado en Kali.
  - `frida-server` está ejecutándose dentro del emulador.
  - La conexión host Kali ↔ dispositivo Android 15 de Genymotion funciona correctamente.

2.9 Instalación de jadx

El proyecto indica que el bundle de Linux usa el JRE del sistema y que Java 11+ es suficiente.

Descargamos jadx:

```
java -version
sudo apt update
sudo apt install -y unzip

cd ~/Descargas
wget https://github.com/skylot/jadx/releases/download/v1.5.5/jadx-1.5.5.zip

mkdir -p ~/tools/jadx
unzip jadx-1.5.5.zip -d ~/tools/jadx
```

Lo añadimos al PATH:

```
echo 'export PATH="$HOME/tools/jadx/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Marcamos el fichero como ejecutable:

```
─$ chmod +x ~/tools/jadx/bin/jadx


└─$ chmod +x ~/tools/jadx/bin/jadx-gui
```

Mostramos la versión de jadx:

```
└─$ ~/tools/jadx/bin/jadx --version
1.5.5
```

Mostramos la ayuda de `jadx-gui`:

```
└─$ ~/tools/jadx/bin/jadx-gui --help

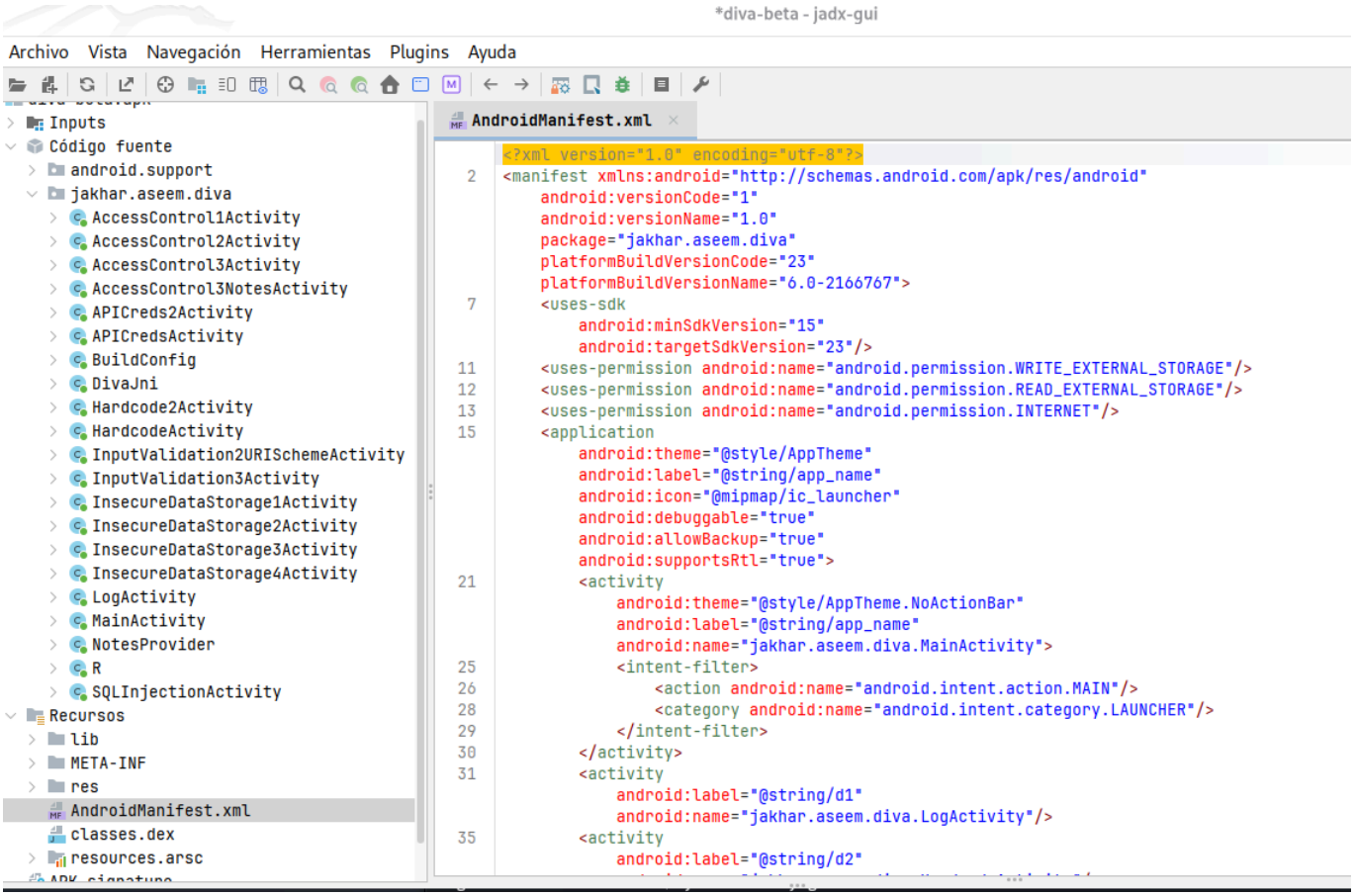
jadx - dex to java decompiler, version: 1.5.5

usage: jadx [command] [options] <input files> (.apk, .dex, .jar, .class,
.smali, .zip, .aar, .arsc, .aab, .xapk, .apkm, .jadx.kts)
commands (use '<command> --help' for command options):
  plugins     - manage jadx plugins
```

```
options:
  -sc, --select-class                          - GUI: Open the selected
class and show the decompiled code
  -d, --output-dir                             - output directory
  -ds, --output-dir-src                        - output directory for
sources
  -dr, --output-dir-res                        - output directory for
resources
  -r, --no-res                                 - do not decode resources
...
```

Abrimos jadx-gui con la app de DIVA:

```
~/tools/jadx/bin/jadx-gui ~/Escritorio/diva-beta.apk
```



Vemos como esta herramienta sirve para decompilar APK/Dex a Java y decodificar AndroidManifest.xml y recursos.

2.10 Instalación de apktool

Instalamos apktool:

```
sudo apt install -y apktool
```

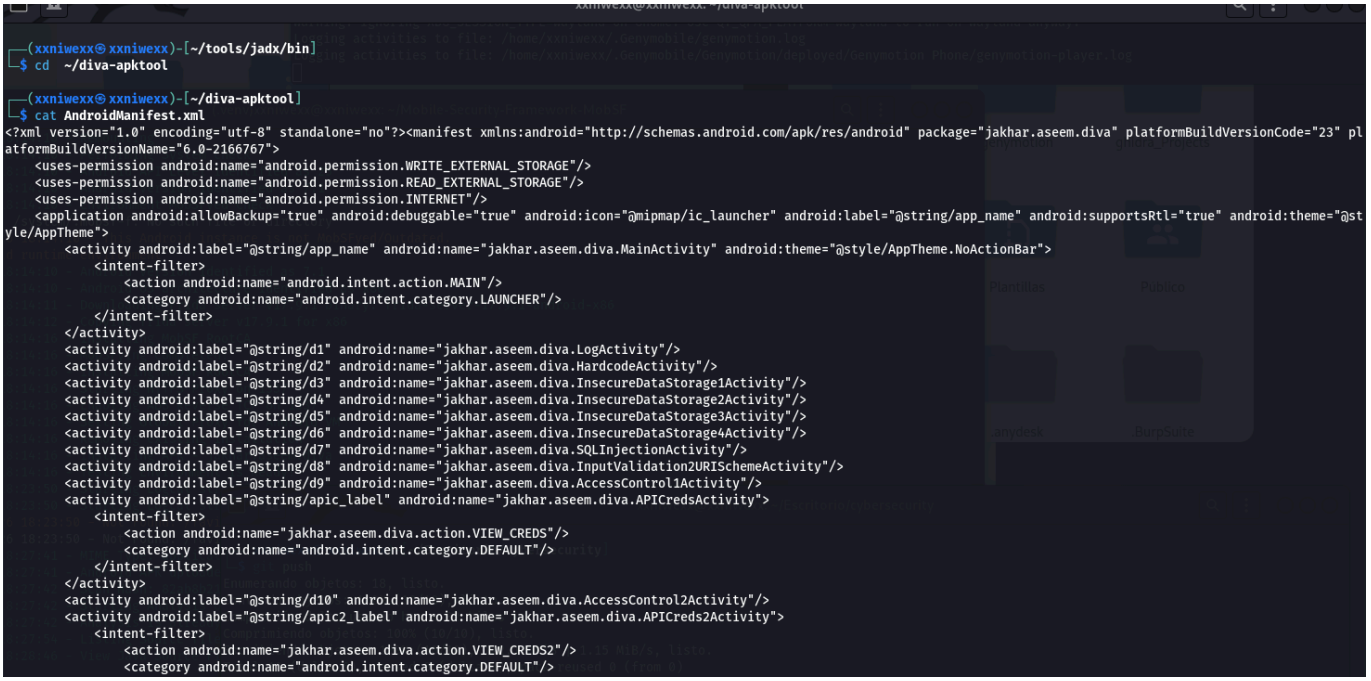
Comprobamos que se instaló correctamente:

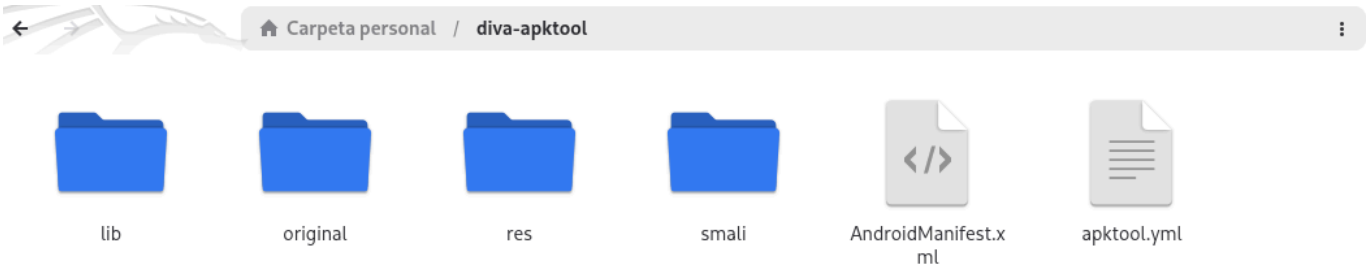
```
└─$ apktool --version
2.7.0-dirty
```

Probamos apktool con la app de DIVA:

```
└─$ apktool d ~/Escritorio/diva-beta.apk -o ~/diva-apktool
I: Using Apktool 2.7.0-dirty on diva-beta.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file:
/home/xxniwexx/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
```

Mostramos el contenido generado por apktool:





Comprobamos cómo esta herramienta desempaqueta y reconstruye esta app.

2.11 Instalación de Ghidra

Usamos Ghidra como herramienta de análisis estático avanzado. Con Ghidra podemos trabajar con partes del APK, como por ejemplo `classes.dex` para ver bytecode Dalvik/ART, librerías nativas `.so` dentro de lib y otros binarios embebidos.

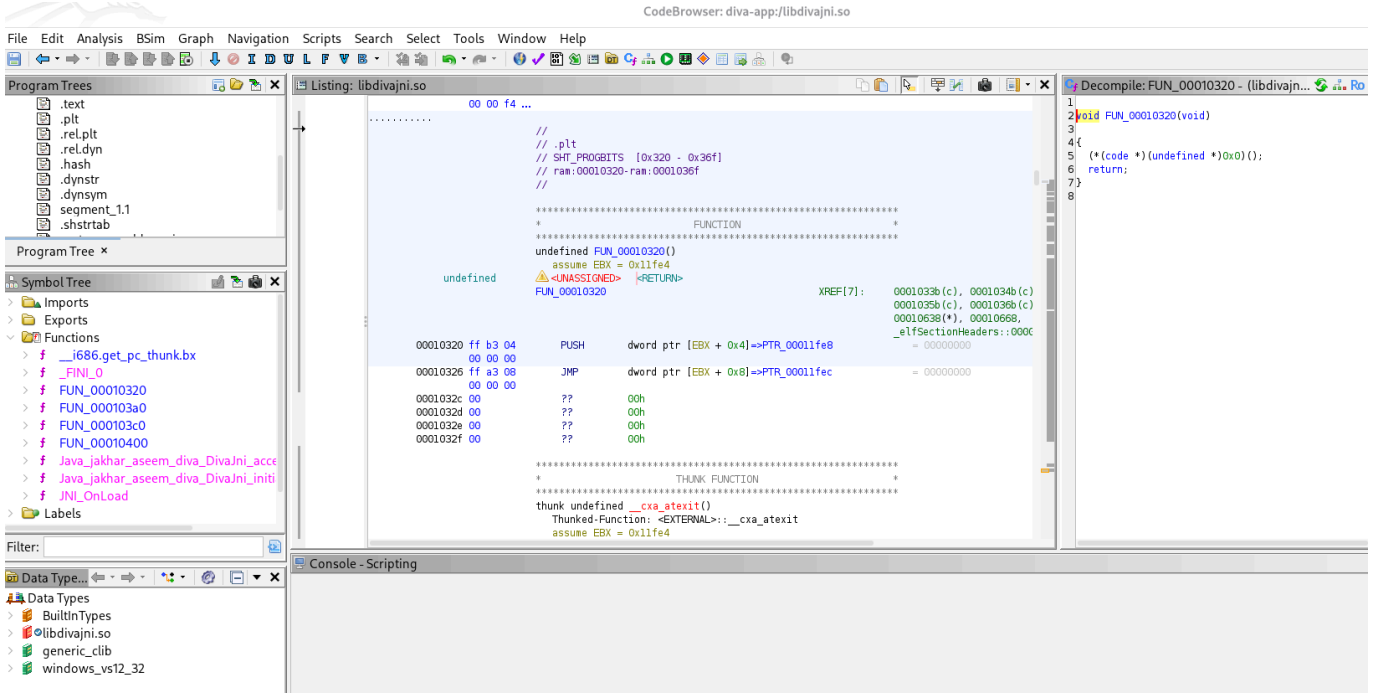
Instalamos Ghidra: Requiere tener Java JDK instalado.

```
sudo apt update && sudo apt upgrade -y
sudo apt install default-jdk -y
flatpak install flathub org.ghidra_sre.Ghidra
./ghidraRun
```

Para probar en acción ghidra, vamos a abrir la librería nativa: `lib/x86/*.so`: `libdivajni.so` que es una librería JNI propia de la app y que fue anteriormente generada por la herramienta `apktool`:



Abrimos esta librería con Ghidra:



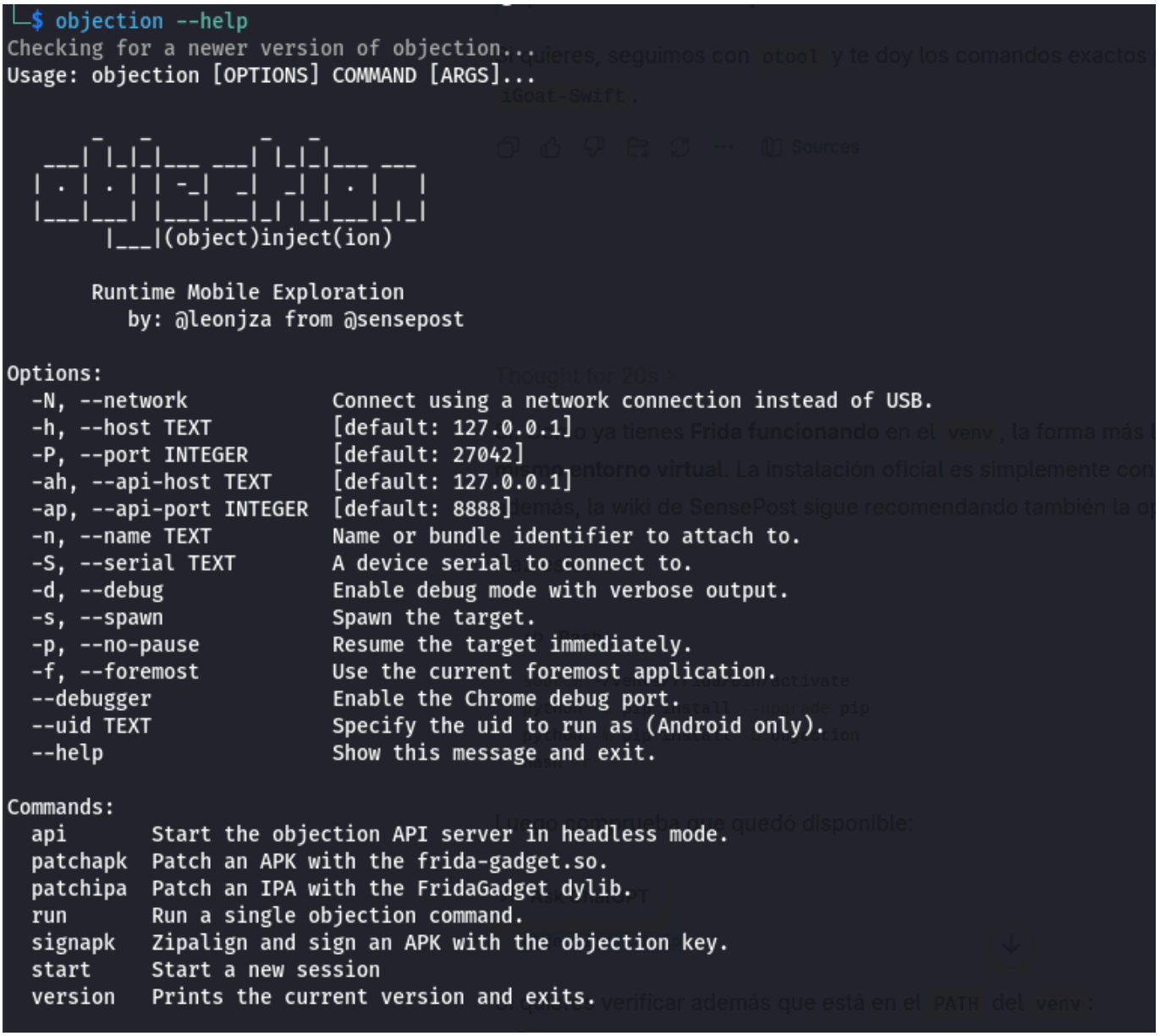
donde:

- Vemos la librería nativa `lib/x86/libdivajni.so` extraída del APK mediante `apktool`.
- Se verifica:
  - La carga correcta del binario ELF.
  - El listado de símbolos, funciones JNI (JNI_OnLoad, Java_*) y el decompilado asociado.

2.12 Instalación de Objection

Como ya tenemos Frida funcionando en el entorno virtual venv, **vamos a instalar Objection en ese mismo entorno virtual:**

```
source ~/venvs/frida/bin/activate
python -m pip install --upgrade pip
python -m pip install -U objection
hash -r
objection --help
```



Verificamos que está en el PATH del venv:

```
which objection
/home/xxniwexx/venvs/frida/bin/objection
```

Como ya tenemos `frida-server` funcionando en el Android virtual, el siguiente paso es **probar que Objection arranca correctamente:**

```
objection -f start
```



donde:

- Se confirma que Objection ya está instalado correctamente en nuestro entorno virtual frida.

Vamos a probar Objection contra el Android 7, validando que puede engancharse a una app, a Diva-Beta:

```
└─$ adb -s 127.0.0.1:6555 shell pm list packages | grep -i diva
package:jakhar.aseem.diva
```

donde:

- En el dispositivo android 7, se comprueba que DIVA está instalada y su paquete es: `jakhar.aseem.diva`.

Abrimos DIVA en el emulador y en la terminal del `venv` de Frida, ejecutamos:

```
└─$ objection -n jakhar.aseem.diva start
Frida server or gadget is not running on the target!
```

donde:

- Obtenemos un error: En el Android 7 no está corriendo `frida-server`.
- Tenemos que instalar el `frida-server` que coincida con este dispositivo android 7.

Comprobamos la arquitectura ABI del dispositivo Android virtual:

```
└─$ adb -s 127.0.0.1:6555 shell getprop ro.product.cpu.abilist
x86
```

donde:

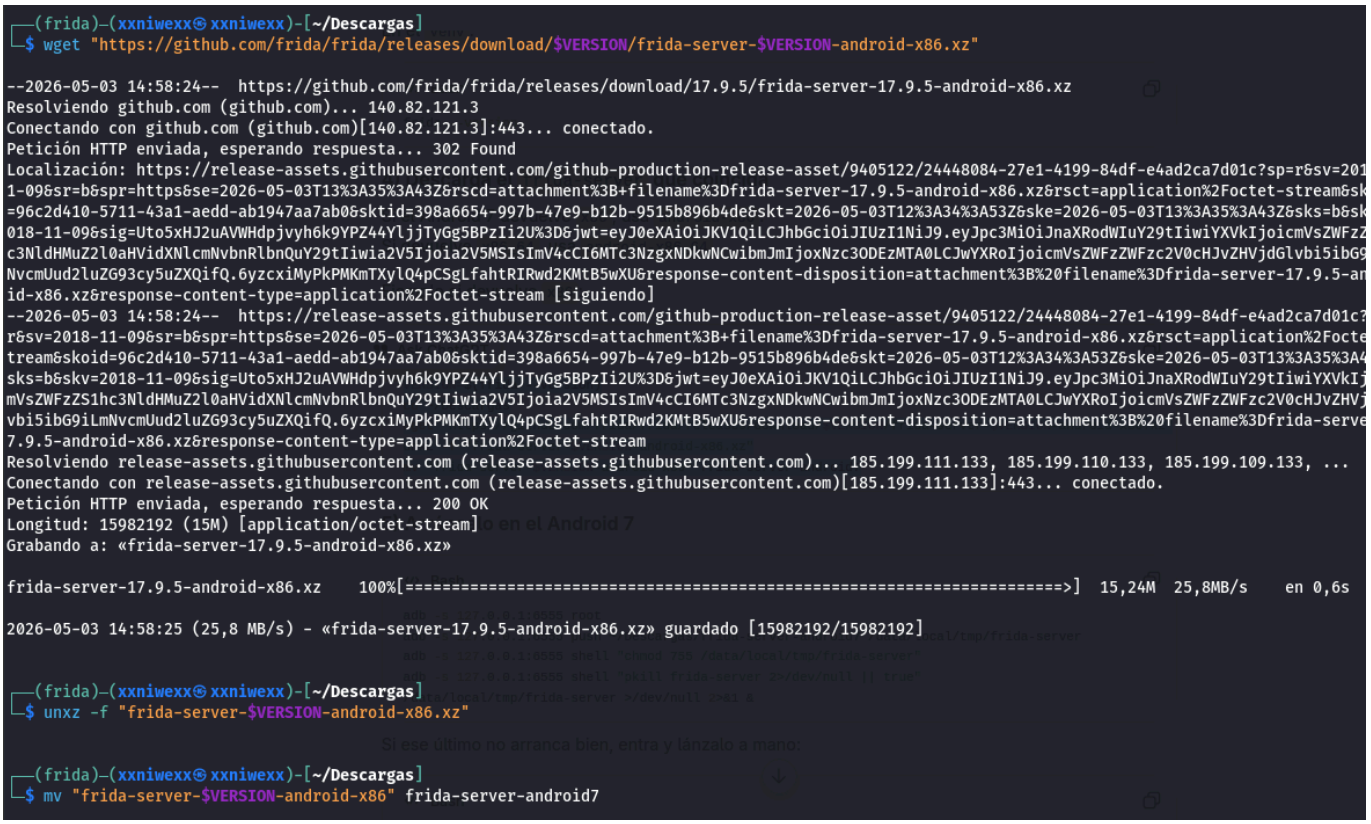
- El resultado `x86` indica que el emulador utiliza una arquitectura Intel de 32 bits.
- Por ello, para herramientas nativas como Frida, debe seleccionarse la versión correspondiente a `android-x86`.

Ejecutamos los siguientes comandos para **descargar la versión de frida-server compatible** con la arquitectura `android-x86` del dispositivo y renombrarla para su posterior despliegue:

```
VERSION=$(frida --version)
cd ~/Descargas

wget "https://github.com/frida/frida/releases/download/$VERSION/frida-
server-$VERSION-android-x86.xz"

unxz -f "frida-server-$VERSION-android-x86.xz"
mv "frida-server-$VERSION-android-x86" frida-server-android7
```



Copiamos y arrancamos `frida-server` en el dispositivo Android 7:

```
adb -s 127.0.0.1:6555 root
adb -s 127.0.0.1:6555 push \
    ~/Descargas/frida-server-android7 \
    /data/local/tmp/frida-server

adb -s 127.0.0.1:6555 shell "chmod 755 /data/local/tmp/frida-server"
adb -s 127.0.0.1:6555 shell "pkill frida-server 2>/dev/null || true"
adb -s 127.0.0.1:6555 shell "/data/local/tmp/frida-server >/dev/null 2>&1
&"
```

donde:

- `adb -s 127.0.0.1:6555 root`: Reinicia `adbd` con privilegios de superusuario para poder operar sobre el sistema del emulador.
- `adb ... push ... /data/local/tmp/frida-server`: Copia el binario de `frida-server` al dispositivo, en una ruta accesible para su ejecución.
- `adb ... shell chmod 755 ...`: Asigna permisos de ejecución al binario copiado.
- `adb ... shell "pkill frida-server ...`: Finaliza cualquier instancia previa de `frida-server` que pudiera estar en ejecución, evitando conflictos.
- `adb ... shell "/data/local/tmp/frida-server >/dev/null 2>&1 &`: Arranca `frida-server` en segundo plano dentro del dispositivo Android 7.

```
┌──(frida)─(xxniwexx㉿xxniwexx)-[~]
└─$ adb -s 127.0.0.1:6555 shell getprop ro.product.cpu.abilist
x86

┌──(frida)─(xxniwexx㉿xxniwexx)-[~]
└─$ VERSION=$(frida --version)

┌──(frida)─(xxniwexx㉿xxniwexx)-[~]
└─$ cd ~/Descargas

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ mv "frida-server-$VERSION-android-x86" frida-server-android7
mv: no se puede efectuar `stat' sobre 'frida-server-17.9.5-android-x86': No existe el fichero o el directorio

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ wget "https://github.com/frida/frida/releases/download/$VERSION/frida-server-$VERSION-android-x86.xz"
--2026-05-03 16:26:40--  https://github.com/frida/frida/releases/download/17.9.5/frida-server-17.9.5-android-x86.xz
Resolviendo github.com (github.com)... 140.82.121.3
Conectando con github.com (github.com)[140.82.121.3]:443... conectado.
Petición HTTP enviada, esperando respuesta... 302 Found
Localización: https://release-assets.githubusercontent.com/github-production-release-asset/9405122/24448084-27e1-4199-8
1-09&sr=b&spr=https&se=2026-05-03T15%3A00%3A10Z&rscd=attachment%3B+filename%3Dfrida-server-17.9.5-android-x86.xz&rsct=a
=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-05-03T13%3A59%3A39Z&ske=2026-
018-11-09&sig=H%2FglQnIMLKvpm1KtK6y0vFTZlYoeUffZOdWUNdnFk20%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJnaXF
1hc3NldHMuZ2l0aHVidXNlcmNvbnRlbnQuY29tIiwia2V5Ijoia2V5MSIsImV4cCI6MTc3NzgyMDIwMCwibmJmIjoxNzc3ODE4NDAwLCJwYXRoIjoicmVsZ
LmNvcmUud2luZG93cy5uZXQifQ._Z7GYhKZoS1R39wyAdtbo9FWBb4XzwUO9W3EULhOIR4&response-content-disposition=attachment%3B%20fil
roid-x86.xz&response-content-type=application%2Foctet-stream [siguiendo]
--2026-05-03 16:26:40--  https://release-assets.githubusercontent.com/github-production-release-asset/9405122/24448084-
r&sv=2018-11-09&sr=b&spr=https&se=2026-05-03T15%3A00%3A10Z&rscd=attachment%3B+filename%3Dfrida-server-17.9.5-android-x8
tream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sktid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-05-03T13%3A59%3A39
sks=b&skv=2018-11-09&sig=H%2FglQnIMLKvpm1KtK6y0vFTZlYoeUffZOdWUNdnFk20%3D&jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJp
icmVsZWFzZS1hc3NldHMuZ2l0aHVidXNlcmNvbnRlbnQuY29tIiwia2V5Ijoia2V5MSIsImV4cCI6MTc3NzgyMDIwMCwibmJmIjoxNzc3ODE4NDAwLCJwYX
Glvbi5ibG9iLmNvcmUud2luZG93cy5uZXQifQ._Z7GYhKZoS1R39wyAdtbo9FWBb4XzwUO9W3EULhOIR4&response-content-disposition=attachme
-17.9.5-android-x86.xz&response-content-type=application%2Foctet-stream
Resolviendo release-assets.githubusercontent.com (release-assets.githubusercontent.com)... 185.199.110.133, 185.199.111
Conectando con release-assets.githubusercontent.com (release-assets.githubusercontent.com)[185.199.110.133]:443... cone
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 15982192 (15M) [application/octet-stream]
Grabando a: «frida-server-17.9.5-android-x86.xz»

frida-server-17.9.5-android-x86.xz    100%[=====================================================================>]  1

2026-05-03 16:26:41 (24,1 MB/s) - «frida-server-17.9.5-android-x86.xz» guardado [15982192/15982192]


┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ unxz -f "frida-server-$VERSION-android-x86.xz"

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ mv "frida-server-$VERSION-android-x86" frida-server-android7
```

```
┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 root
adbd is already running as root

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 root
adbd is already running as root

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 push \
~/Descargas/frida-server-android7 \
/data/local/tmp/frida-server
/home/xxniwexx/Descargas/frida-server-android7: 1 file pushed, 0 skipped. 338.4 MB/s (53691336 bytes in 0.151s)

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 shell "chmod 755 /data/local/tmp/frida-server"

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 shell "pkill frida-server 2>/dev/null || true"

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 shell "/data/local/tmp/frida-server >/dev/null 2>&1 &"

┌──(frida)─(xxniwexx㉿xxniwexx)-[~/Descargas]
└─$ frida-ps -Uai | grep -i diva
1641  Diva                      jakhar.aseem.diva
```

Comprobamos que Frida detecta correctamente el dispositivo Android 7 y puede enumerar los procesos en ejecución:

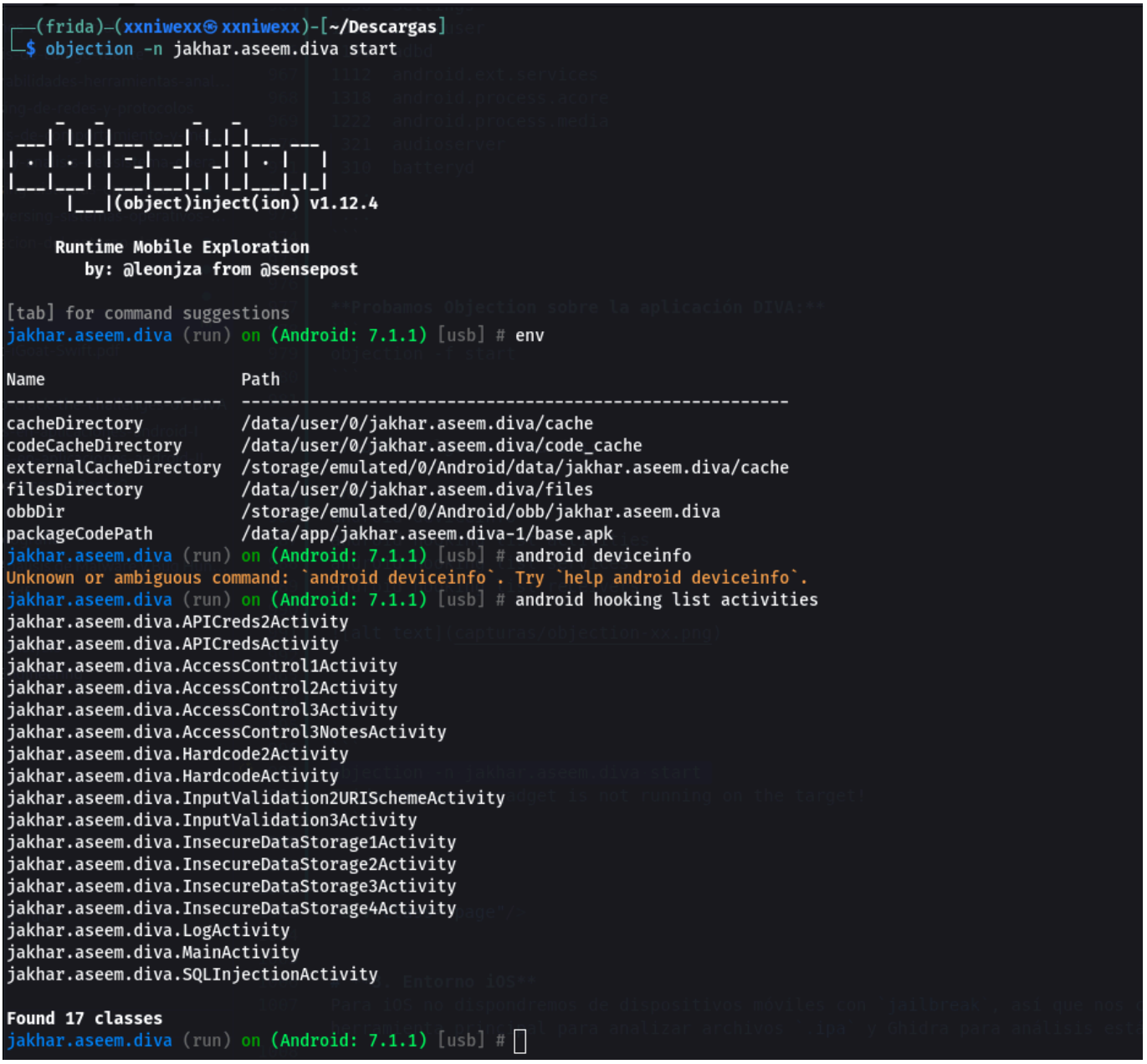
```
frida-ps -U
 PID  Name
----  --------------------------------
1388  Calendar
1127  Clock
2286  Diva
1424  Email
1467  Messaging
1362  Phone
 836  Settings
1514  Superuser
 160  adbd
1112  android.ext.services
1318  android.process.acore
1222  android.process.media
 321  audioserver
 310  batteryd
 ...
 ...
```

Probamos Objection sobre la aplicación DIVA:

```
objection -n jakhar.aseem.diva start
```

Probamos algunos comandos:

```
env
android hooking list activities
android hooking list services
android hooking list receivers
```



La captura muestra que Objection se ha adjuntado correctamente a la aplicación DIVA en ejecución sobre Android 7. Mediante los comandos `env` y `android hooking list activities` se obtienen rutas internas de la aplicación y el listado de sus actividades, confirmando que el entorno de instrumentación dinámica con Frida + Objection funciona correctamente.

2.12 Instalación de Drozer

Drozer es una herramienta de análisis de seguridad para aplicaciones Android que permite interactuar con una app instalada desde el propio dispositivo para enumerar componentes, revisar su superficie de ataque y validar configuraciones inseguras como actividades, servicios, receptores o proveedores exportados.

Para instalar Drozer en Kali, instalaremos el `cliente de drozer` con `pipx` y luego instalar el `drozer Agent` en el Android. El README oficial del proyecto indica que esta versión reescrita soporta Python 3, recomienda `pipx install drozer`, requiere `Python 3.8+` y `JDK 11+`, y el propio repositorio muestra la release 3.1.0 como la más reciente: https://github.com/ReversecLabs/drozer-agent

Instalamos Drozer en el dispositivo android 7:

```
sudo apt update
sudo apt install -y pipx default-jdk adb
pipx ensurepath
pipx install drozer
drozer --help
```

donde:

- Instalamos drozer y mostramos su ayuda:

```
┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ pipx install drozer
  installed package drozer 3.1.0, installed using Python 3.13.12
  These apps are now available
    - drozer
    - drozer-complete
    - drozer-repository
done! ✨ 🌟 ✨

┌──(xxniwexx㉿kali)-[~/Escritorio/cybersecurity]
└─$ drozer --help
unknown command: --help

usage: drozer [COMMAND]

Run `drozer [COMMAND] --help` for more usage information.

Commands:

           agent  create custom drozer Agents
         console  start the drozer Console
         exploit  generate an exploit to deploy drozer
          module  manage drozer modules
         payload  generate payloads to deploy drozer
          server  start a drozer Server
             ssl  manage drozer SSL key material
```

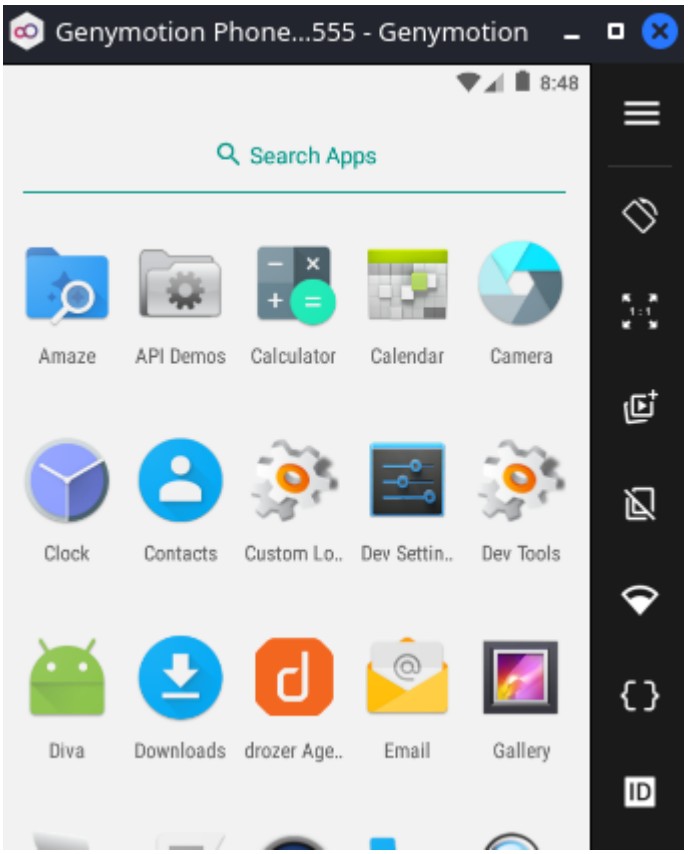
Descargamos Drozer e instalamos drozer Agent en el dispositivo Android:

```
cd ~/Descargas
wget -O drozer-agent.apk https://github.com/ReversecLabs/drozer-agent/releases/download/3.1.0/drozer-agent.apk
adb -s 127.0.0.1:6555 install -r ~/Descargas/drozer-agent.apk
```

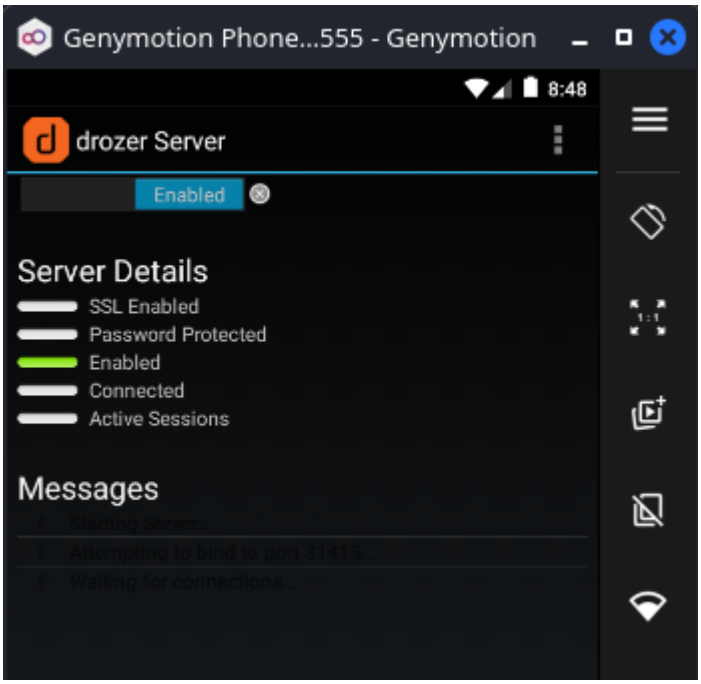
```
┌──(frida)─(xxniwexx㉿kali)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 install -r ~/Descargas/drozer-agent.apk
Performing Streamed Install
Success
```

Conectamos el cliente al agente: En el dispositivo android 7:

- Abrimos la app drozer Agent.



- Entramos en `Embedded Server`.
- Pulsamos `Enable`.



- Conectamos desde el cliente por ADB forward:

```
adb -s 127.0.0.1:6555 forward tcp:31415 tcp:31415
drozer console connect
```

```
┌──(frida)─(xxniwexx㉿kali)-[~/Descargas]
└─$ adb -s 127.0.0.1:6555 forward tcp:31415 tcp:31415
31415

┌──(frida)─(xxniwexx㉿kali)-[~/Descargas]
└─$ drozer console connect
Selecting 89d8a400f7d5f4f7 (unknown Phone 7.1.1)

           ..                    ..:.
          ..o..                  .r..
           ..a..  . ....... .  ..nd
             ro..idsnemesisand..pr
             .otectorandroidsneme.
          .,sisandprotectorandroids+.
        ..nemesisandprotectorandroidsn:.
       .emesisandprotectorandroidsnemes..
     ..isandp,..,rotecyayandro,..,idsnem.
     .isisandp..rotectorandroid..snemisis.
     ,andprotectorandroidsnemisisandprotec.
    .torandroidsnemesisandprotectorandroid.
    .snemisisandprotectorandroidsnemesisan:
    .dprotectorandroidsnemesisandprotector.

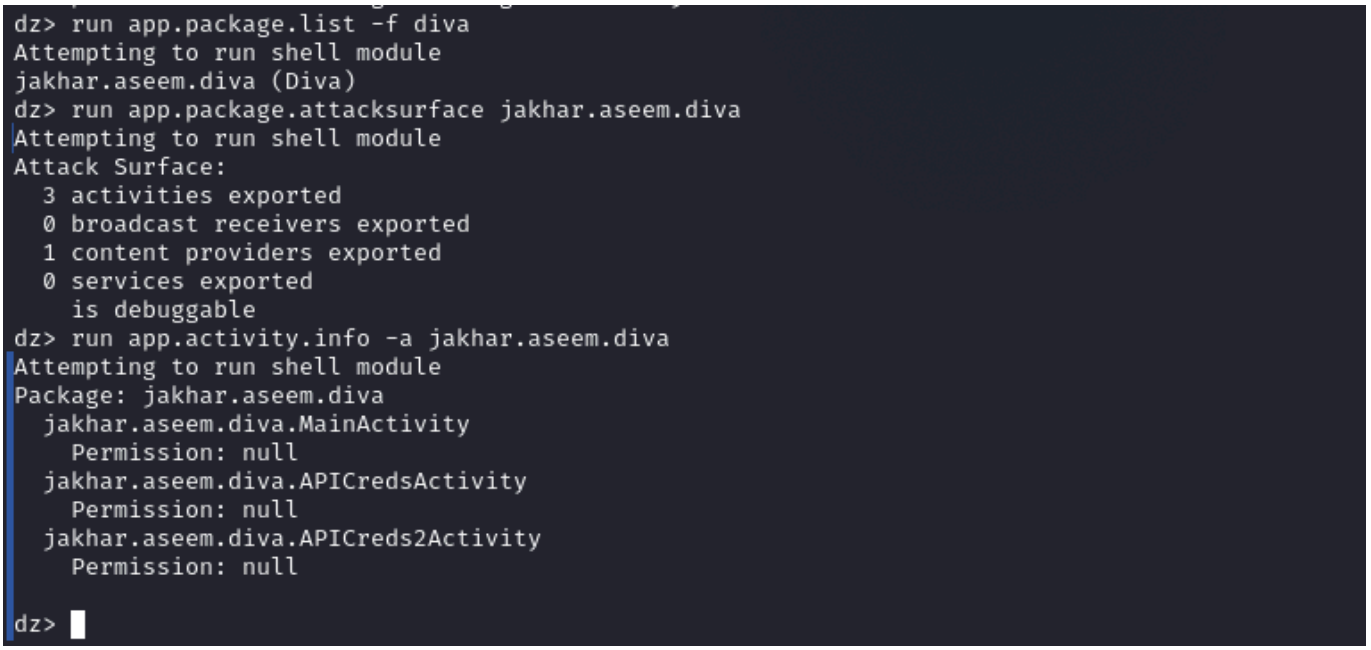
drozer Console (v3.1.0)
dz>
```

donde:

- Hemos entrado en el prompt de drozer.

Hacemos una pequeña validación con Drozer de la app DIVA. Dentro de `drozer console connect`:

```
run app.package.list -f diva
run app.package.attacksurface jakhar.aseem.diva
run app.activity.info -a jakhar.aseem.diva
```



donde:

- `run app.package.list -f diva`: Comprueba que drozer detecta la app instalada.
- `run app.package.attacksurface jakhar.aseem.diva`: Resumen de la superficie de ataque de la app:
  - 3 activities exportadas.
  - 0 broadcast receivers exportados.
  - 1 content provider exportado.
  - 0 services exportados.
  - Además, la app aparece como `debuggable`.
- `run app.activity.info -a jakhar.aseem.diva`: Enumera actividades concretas, por ejemplo:
  - `jakhar.aseem.diva.MainActivity`.
  - `jakhar.aseem.diva.APICredsActivity`.
  - `jakhar.aseem.diva.APICreds2Activity`.
  - Además indicando `Permission: null`, lo que sugiere que esas activities no exigen un permiso específico para su invocación.

La captura valida que drozer funciona correctamente en el laboratorio y que el entorno permite realizar enumeración básica de superficie de ataque sobre aplicaciones Android instaladas. Para este ejercicio es suficiente para justificar que la herramienta ha quedado instalada, conectada y preparada para análisis posteriores.

2.13 Instalación de Mara Framework

Instalamos Docker en Kali:

```
└─$ sudo apt update

└─$ sudo apt install -y docker.io

└─$ sudo systemctl enable docker --now
```

Configuramos para usar docker sin sudo:

```
└─$ sudo usermod -aG docker $USER


└─$ newgrp docker
```

Descargamos la imagen de MARA:

```
└─$ docker pull xyphex/mara-framework
```

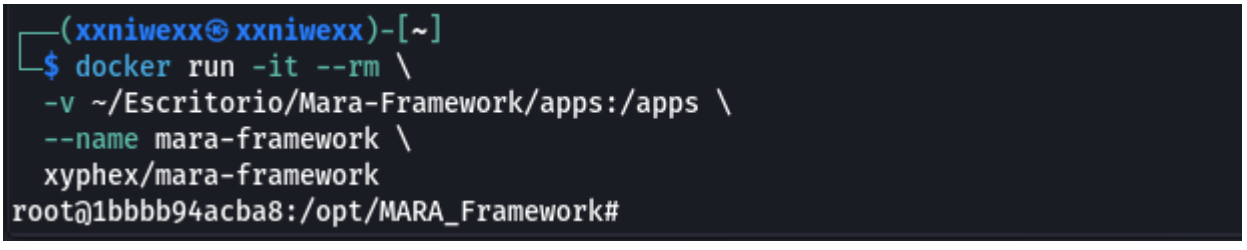
Creamos una carpeta para los APK:

```
└─$ mkdir -p ~/Escritorio/Mara-Framework/apps

└─$ cp ~/Escritorio/diva-beta.apk ~/Escritorio/Mara-Framework/apps
```

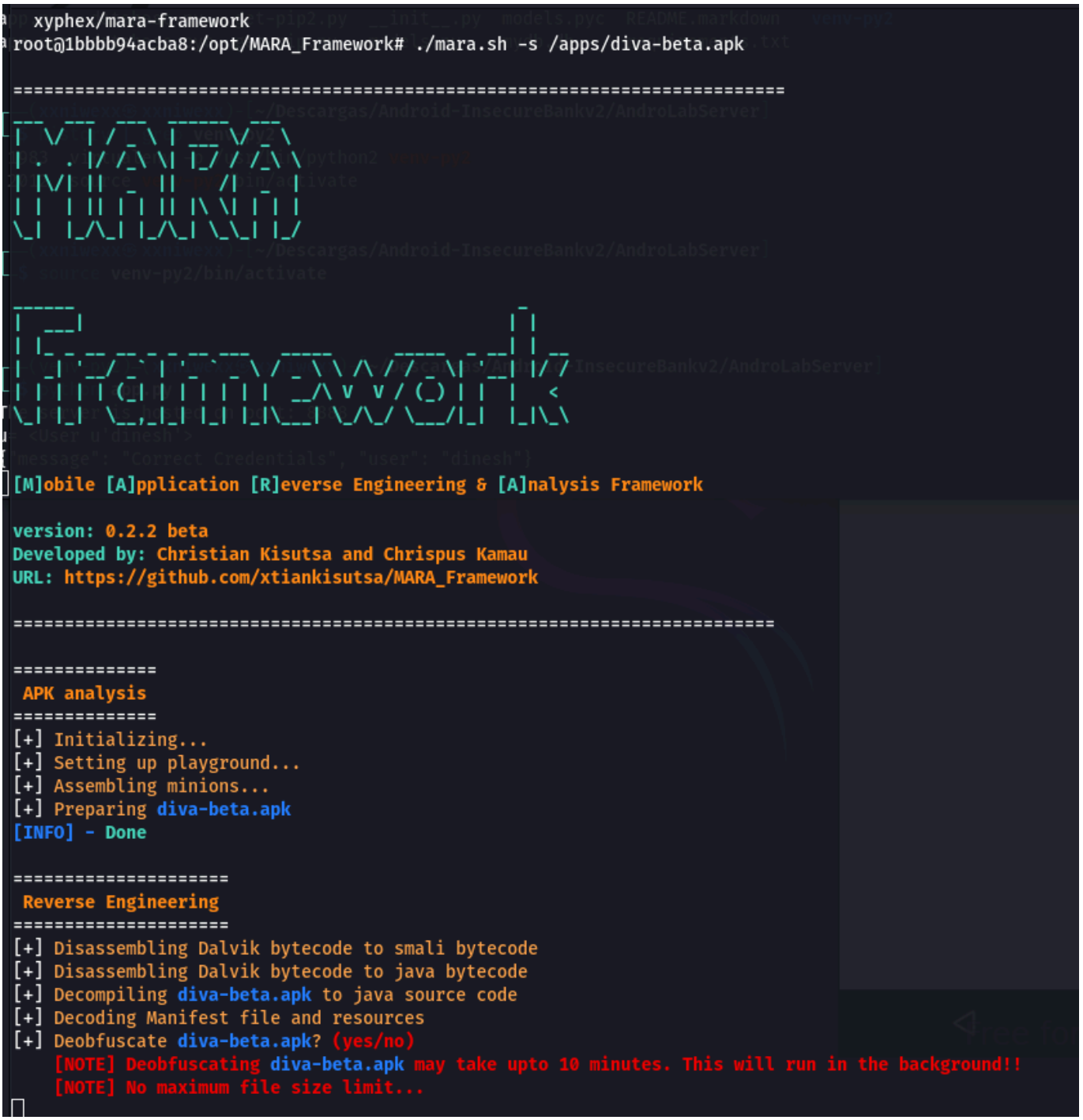
Arrancamos el contenedor:

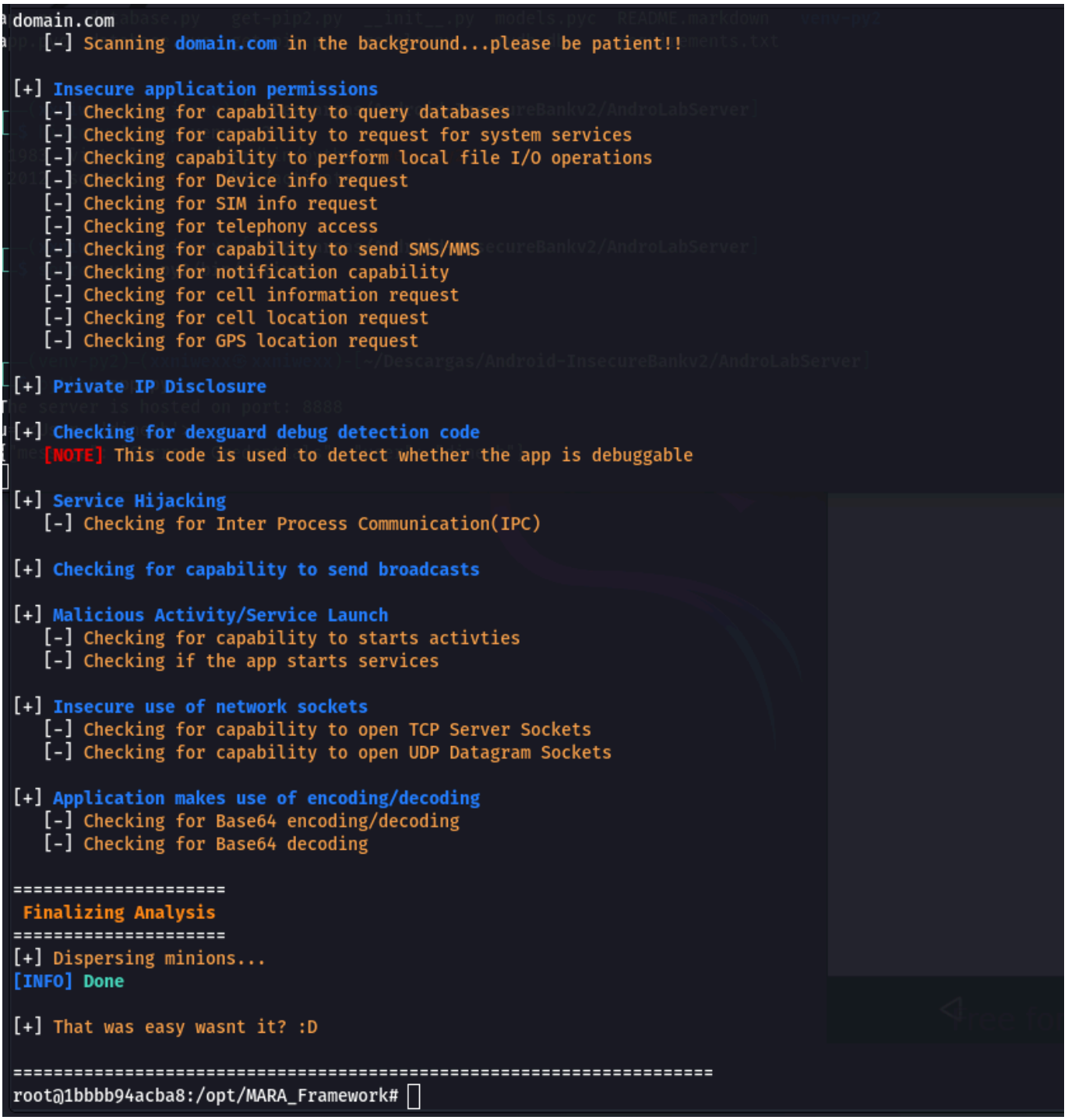
```
└─$ docker run -it --rm \
  -v ~/Escritorio/Mara-Framework/apps:/apps \
  --name mara-framework \
  xyphex/mara-framework
```



Analizamos una APK dentro del contenedor: Ya dentro del contenedor:

```
└─$ ./mara.sh -s /apps/diva-beta.apk
```





donde vemos que Mara ha realizado:

- Preparación de la APK,
- reversing,
- análisis del Manifest,
- análisis preliminar,
- checks OWASP Mobile Top 10,
- y finalización del análisis.

MARA Framework genera principalmente un reporte textual en consola, donde va mostrando las distintas fases del análisis y las comprobaciones realizadas sobre la APK. En la ejecución observada, el análisis finalizó correctamente y devolvió el prompt, aunque se detectaron errores asociados a la ausencia de utilidades auxiliares como `aapt` y `dexdump`, por lo que parte del análisis no se completó íntegramente.

3. Entorno iOS

Para iOS no dispondremos de dispositivos móviles con `jailbreak`, así que nos centraremos en análisis estático. En ese contexto, usaremos MobSF como herramienta principal para analizar archivos `.ipa` y Ghidra para análisis estático avanzado del binario.

Las apps de App Store están protegidas con FairPlay DRM, así que para obtener una versión descifrada normalmente hace falta un dispositivo físico modificado, por lo que en el laboratorio académico se parte de ficheros `.ipa` ya descifrados. Así que nos centraremos en extraer su contenido, revisar `Info.plist`, frameworks, cadenas, binario `Mach-O` y resultados automáticos de MobSF.

3.1 Esquema de análisis en iOS

Partiendo el host Kali Linux que ya tenemos preparado, usaremos:

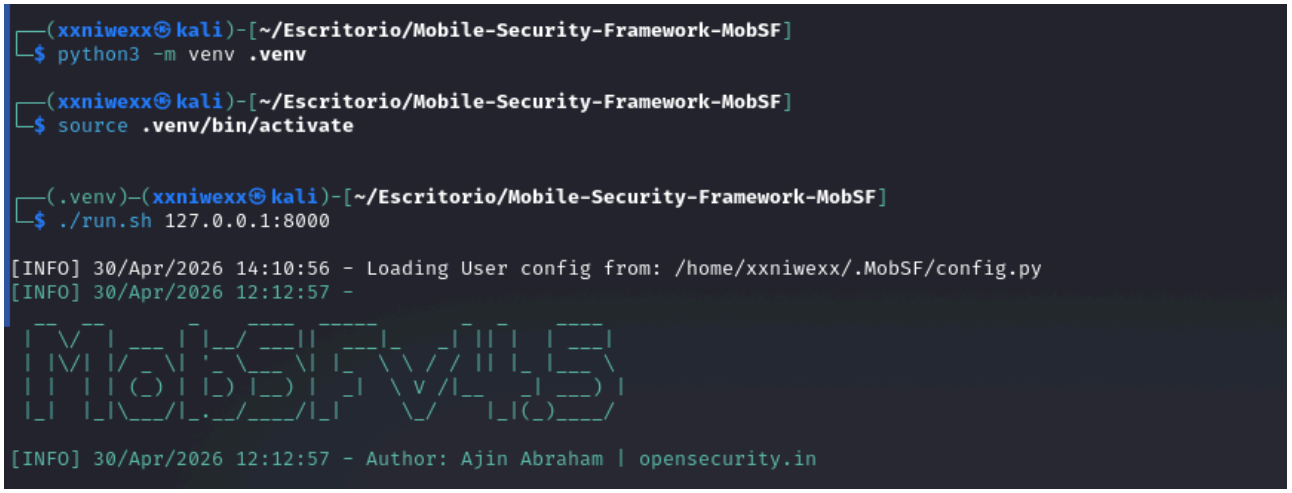
- Con MobSF → analizamos el `.ipa` completo. Nos aportará visión una general de:
  - Info.plist.
  - Permisos.
  - Strings.
  - Frameworks.
  - Configuración.
- Ghidra → analizamos el `Mach-O`. Nos aportará visión una general de:
  - Funciones.
  - Strings en contexto.
  - Referencias.
  - Imports/Exports.
  - Decompilado.

3.2 Análisis estático inicial: MobSF

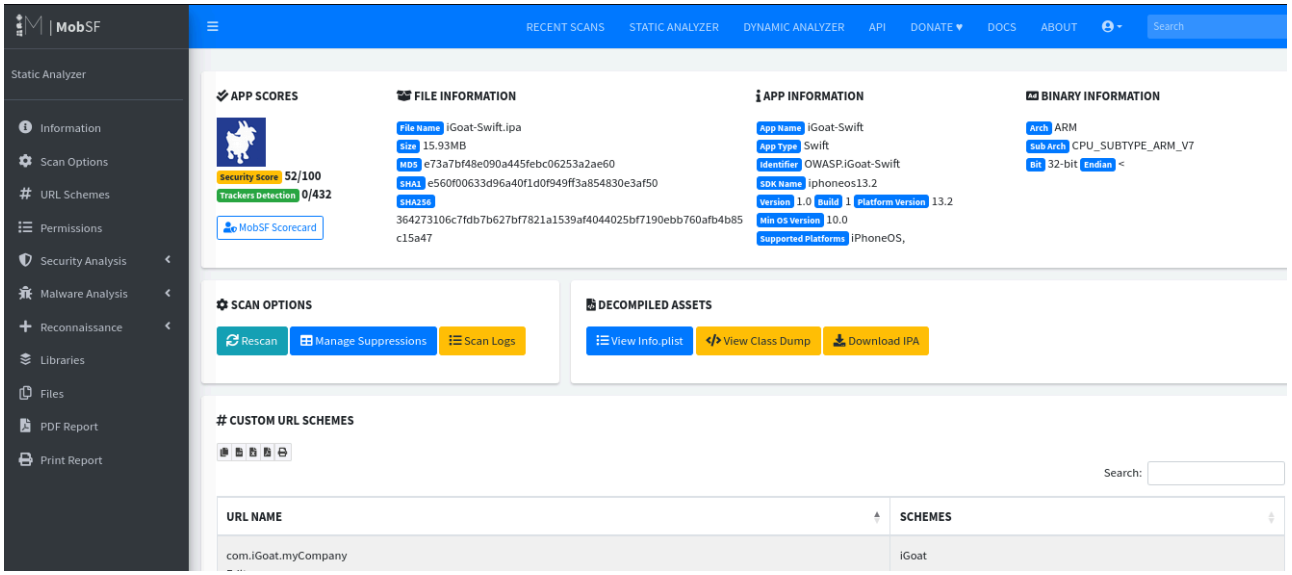
Para este apartado vamos a escoger para hacer prácticas: OWASP iGoat-Swift ya que es una app iOS vulnerable orientada a aprendizaje y su repositorio público incluye un archivo `iGoat-Swift.ipa`.

Descargamos el fichero y lo cargamos en MobSF:

- Activamos el entorno virtual python y abrimos MobSF.



- Pulsamos el botón `Upload & Analize` para cargar `iGoat-Swift.ipa`.



Cosas relevantes a revisar

- Info.plist:

```
Info.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
"http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>BuildMachineOSBuild</key>
    <string>19B2093</string>
    <key>CFBundleDevelopmentRegion</key>
    <string>en</string>
    <key>CFBundleExecutable</key>
    <string>iGoat-Swift</string>
    <key>CFBundleIcons</key>
    <dict>
        <key>CFBundlePrimaryIcon</key>
        <dict>
            <key>CFBundleIconFiles</key>
            <array>
                <string>AppIcon29x29</string>
                <string>AppIcon60x60</string>
            </array>
            <key>CFBundleIconName</key>
            <string>AppIcon</string>
        </dict>
    </dict>
    <key>CFBundleIdentifier</key>
    <string>OWASP.iGoat-Swift</string>
    <key>CFBundleInfoDictionaryVersion</key>
    <string>6.0</string>
    <key>CFBundleName</key>
    <string>iGoat-Swift</string>
```

```
    <key>CFBundlePackageType</key>
    <string>APPL</string>
    <key>CFBundleShortVersionString</key>
    <string>1.0</string>
    <key>CFBundleSupportedPlatforms</key>
    <array>
        <string>iPhoneOS</string>
    </array>
    <key>CFBundleURLTypes</key>
    <array>
        <dict>
            <key>CFBundleTypeRole</key>
            <string>Editor</string>
            <key>CFBundleURLName</key>
            <string>com.iGoat.myCompany</string>
            <key>CFBundleURLSchemes</key>
            <array>
...
...
</dict>
</plist>
```

- Strings.
- Frameworks.
- Permisos/configuración.
- Binario principal.
- Análisis estático profundo: Ghidra.

Informe del Análisis estático realizado por MobSF

3.3 Análisis profundo: Ghidra

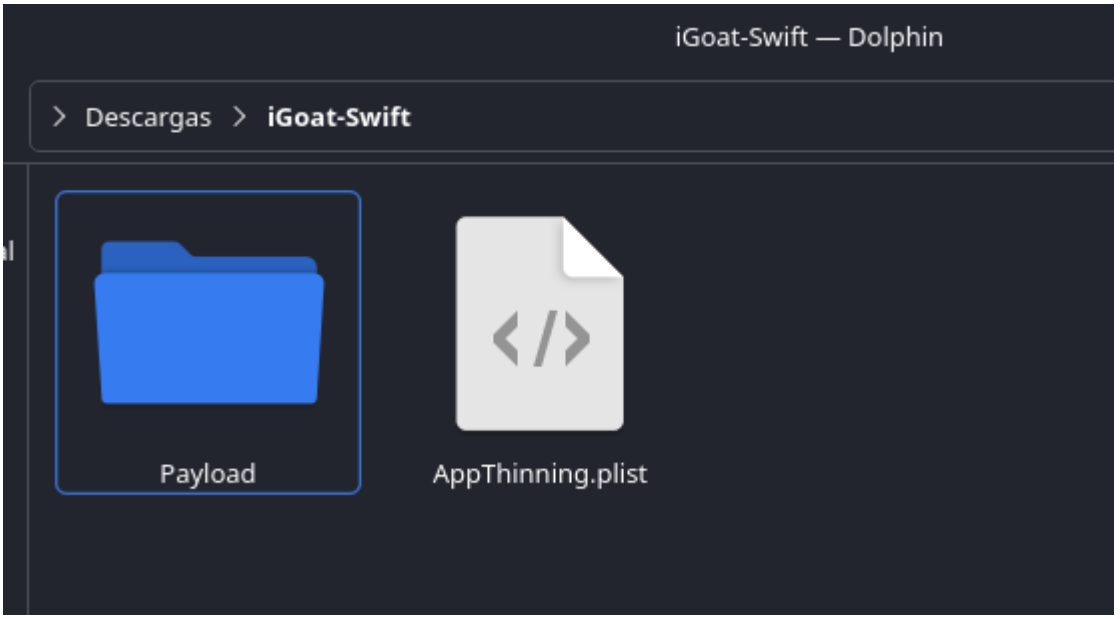
En iOS:

- El fichero `.ipa` es un contenedor parecido a un ZIP.
- Dentro de este contenedor va la app.
- Y dentro de la app está el ejecutable `Mach-O`, que es el binario que realmente se ejecuta en iPhone.

Vamos a descomprimir la aplicación:

```
cp iGoat-Swift.ipa iGoat-Swift.zip
unzip -q iGoat-Swift.zip -d iGoat-Swift
```

Obtenemos los siguientes ficheros:



Encontrar `Mach-O`: En iOS, el ejecutable real suele estar dentro de `Payload/NombreApp.app/` y normalmente no tiene extensión. Para saber dónde se encuentra realmente el ejecutable principal, necesitamos encontrar `Mach-O`:

Sabemos por `Info.plist` que el `Mach-O` principal está en:

```
Payload/iGoat-Swift.app/iGoat-Swift
```

donde:

- CFBundleExecutable.
- iGoat-Swift.
- En iOS, `CFBundleExecutable` indica el nombre del ejecutable principal de la app.

Así obtenemos que el ejecutable principal de esta app está en: `Payload/iGoat-Swift.app/iGoat-Swift`.

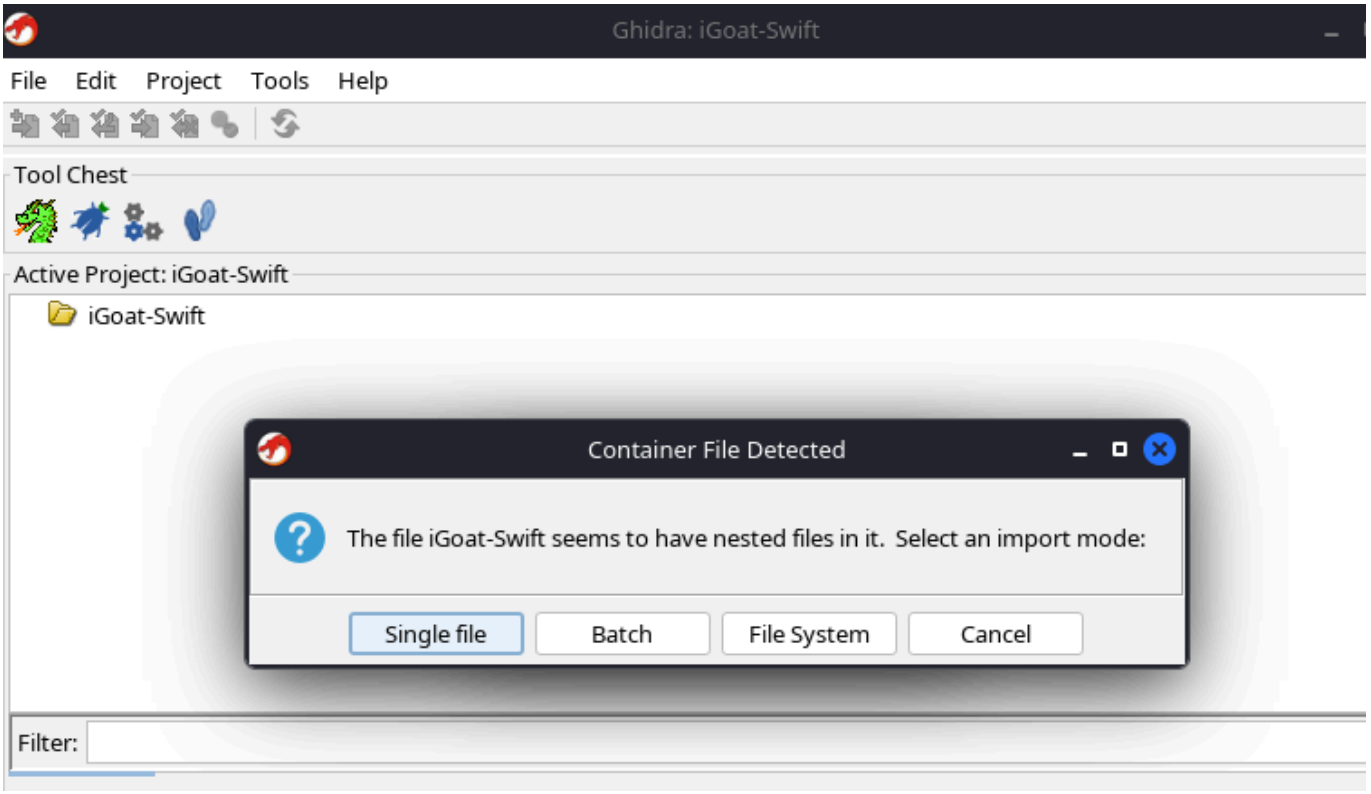
Hacemos la siguiente comprobación:

```
└─$ file Payload/iGoat-Swift.app/iGoat-Swift
Payload/iGoat-Swift.app/iGoat-Swift: Mach-O universal binary with 2
architectures: [armv7:\012- Mach-O armv7 executable, flags:
<NOUNDEFS|DYLDLINK|TWOLEVEL|WEAK_DEFINES|BINDS_TO_WEAK|PIE>] [\012-
arm64:\012- Mach-O 64-bit arm64 executable, flags:
<NOUNDEFS|DYLDLINK|TWOLEVEL|WEAK_DEFINES|BINDS_TO_WEAK|PIE>]
```

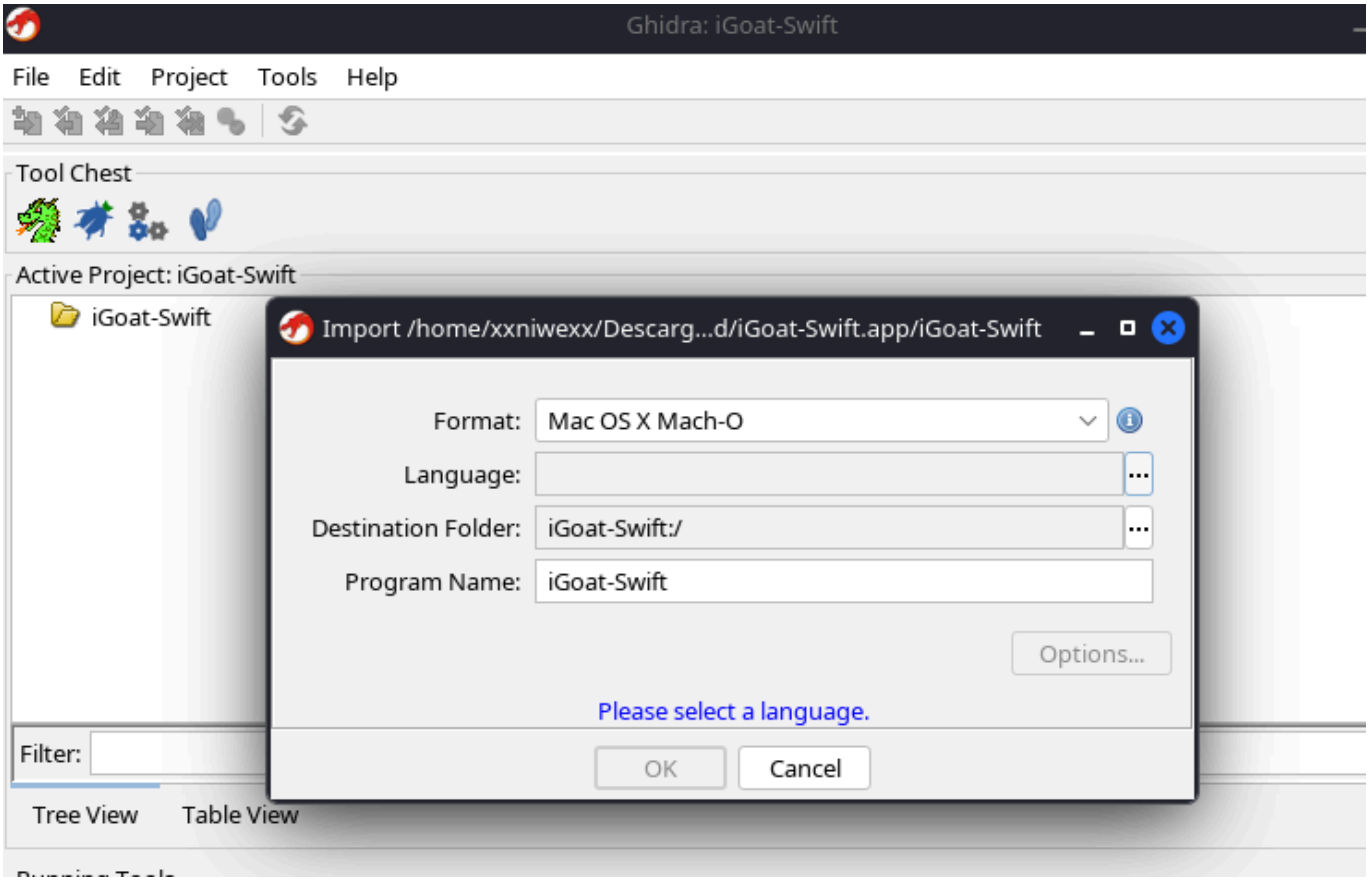
donde destacamos:

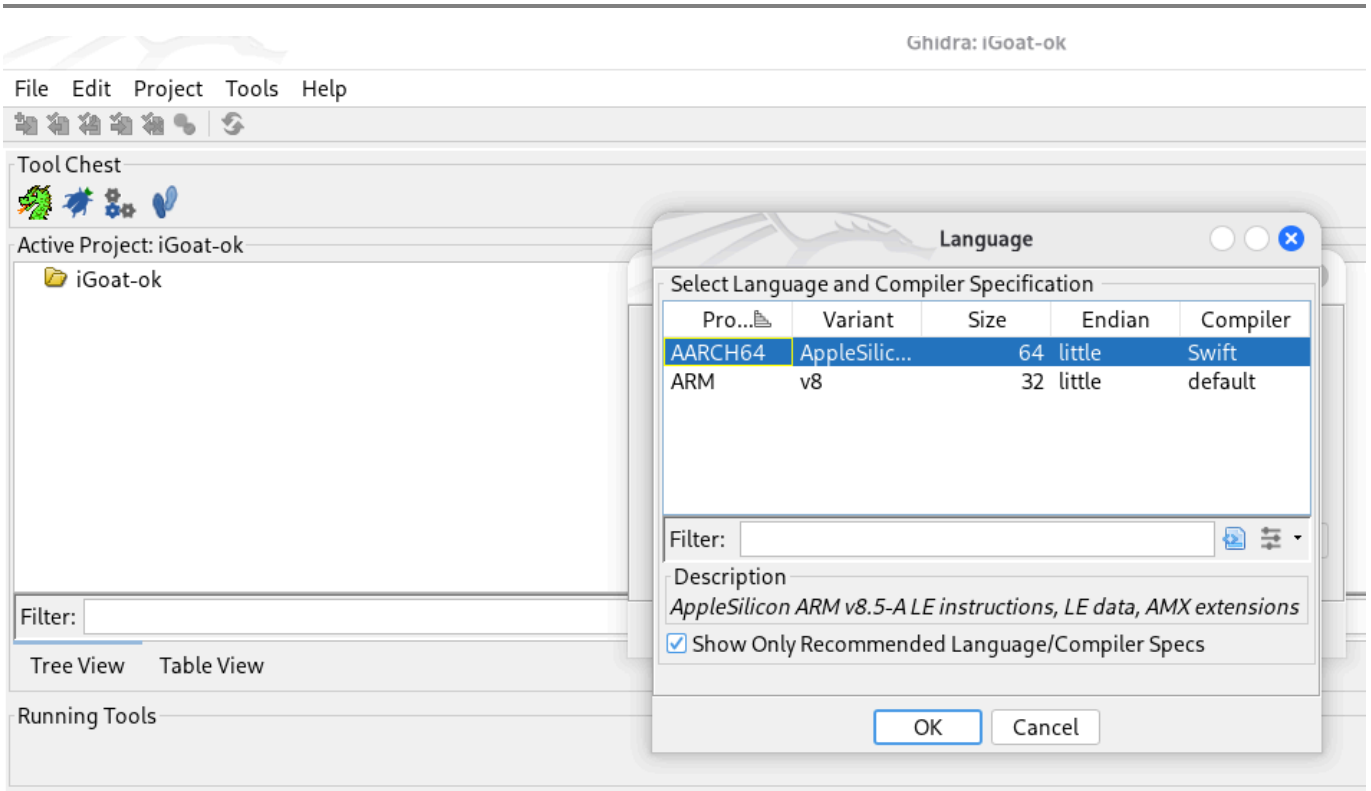
- `Payload/iGoat-Swift.app/iGoat-Swift` es efectivamente el binario ejecutable de la aplicación iOS.
- `Mach-O universal binary with 2 architectures`: Implica que el mismo fichero contiene dos versiones del ejecutable, una para cada arquitectura:
  - armv7 → 32 bits.
  - arm64 → 64 bits. Esto parece habitual para dar compatibilidad con distintos dispositivos iPhone/iPad.
- `PIE` → el binario es `Position Independent Executable`, lo que significa que soporta `ASLR`. Esto indica que el binario fue compilado como position independent executable, lo que permite `ASLR` y mejora la protección frente a ciertos ataques de corrupción de memoria.

Abrimos este ejecutable principal con Ghidra: Marcamos que para este proyecto con Ghidra lo vamos a importar como un único fichero:



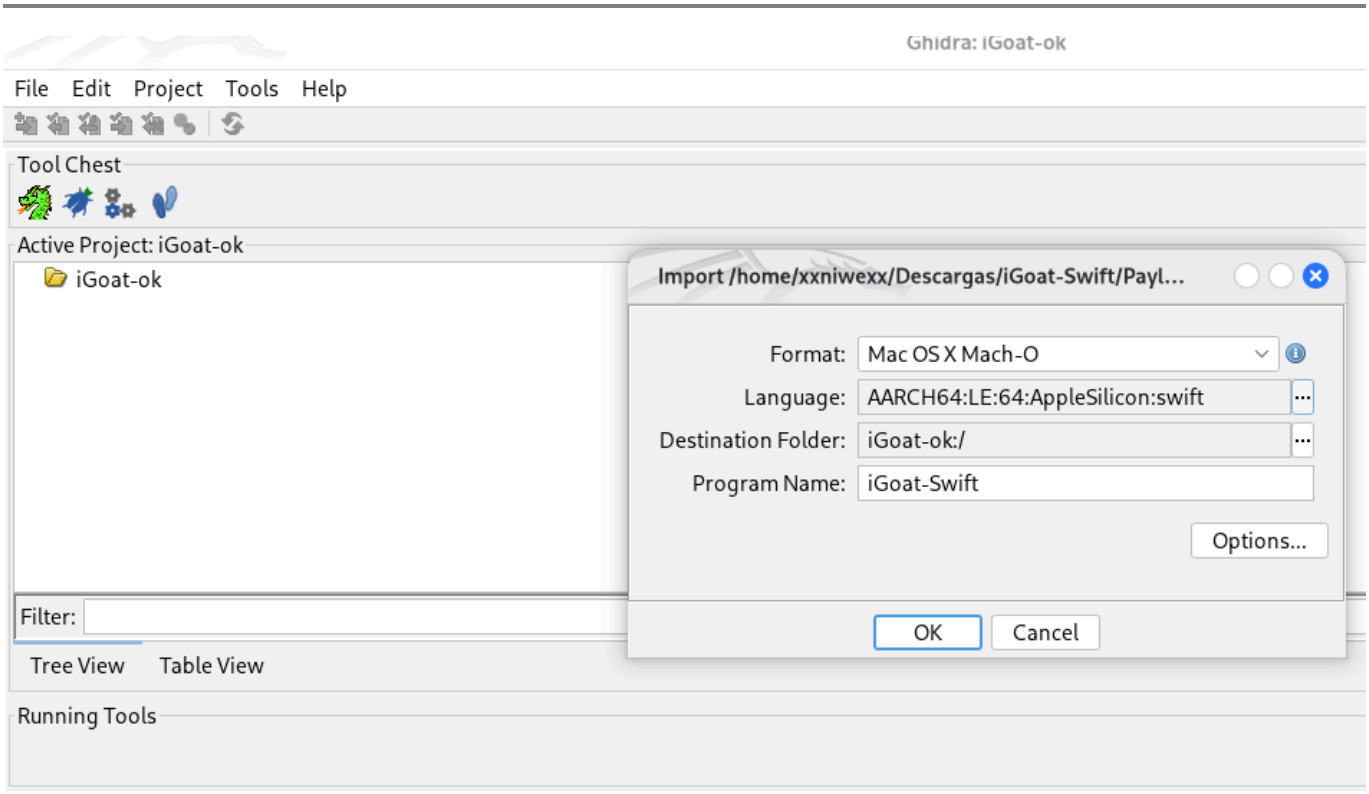
Al importar el ejecutable `iGoat-Swift` en Ghidra, hemos visto que el binario `Mach-O universal` permite analizar distintas arquitecturas. En este vamos a seleccionar el `slice arm64`, por lo que el lenguaje correcto en Ghidra es `AArch64 little endian de 64 bits`, por ejemplo `AARCH64:LE:64:v8A`

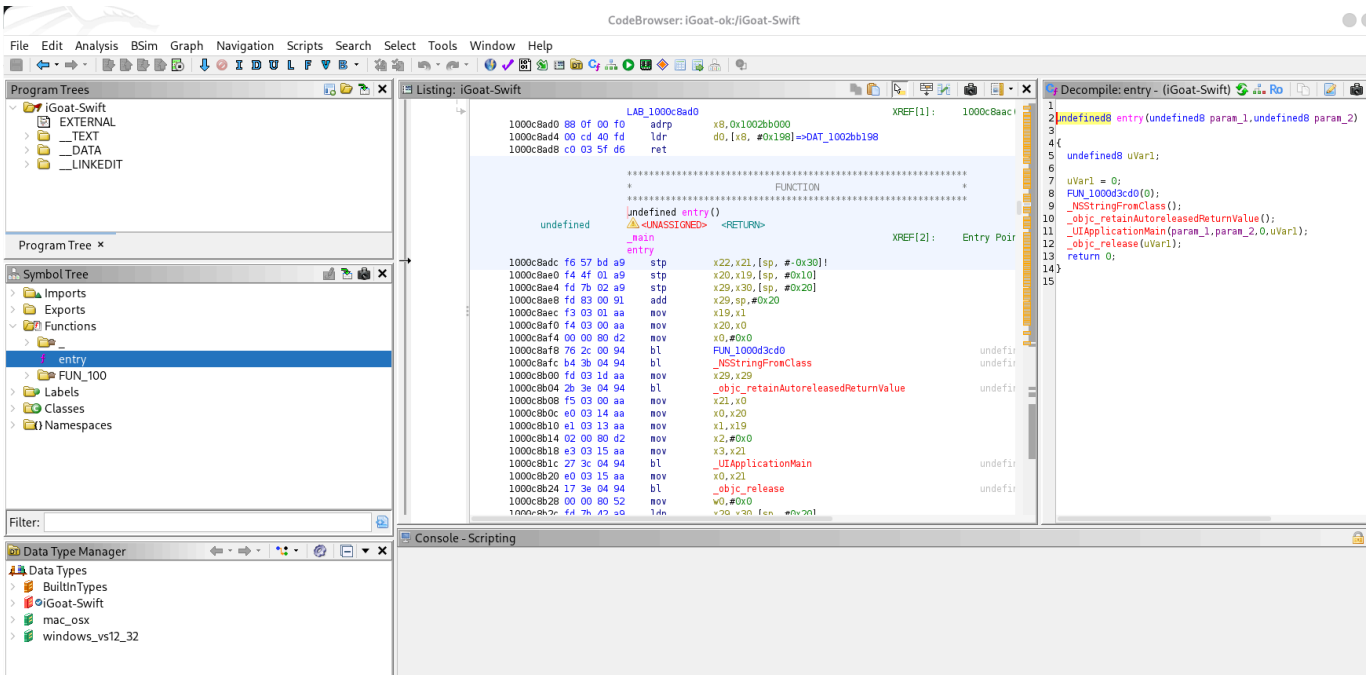




donde:

- Como el ejecutable iGoat-Swift es un `Mach-O` universal con `slices armv7` y `arm64`, para el análisis principal seleccionamos la arquitectura `arm64`. En Ghidra elegimos un lenguaje `AArch64 little endian de 64 bits`, por ejemplo `AARCH64:LE:64:v8A`. La opción `ARM:LE:32:v8` sólo sería adecuada si quisiéramos analizar el slice armv7 de 32 bits.





Una vez importado, Ghidra carga correctamente el binario y muestra sus secciones principales (`__TEXT`, `__DATA`, `__LINKEDIT`), el árbol de símbolos y la función de entrada `entry`. En el panel de decompilado se aprecian llamadas típicas del arranque de una aplicación iOS, como `UIApplicationMain` y referencias al runtime de `Objective-C`, lo que confirma que el ejecutable `Mach-O` ha sido importado y analizado correctamente.

3.4 Burp Suite

En la parte correspondiente a iOS no vamos a configurar Burp Suite, ya que el entorno preparado en este ejercicio está orientado exclusivamente a análisis estático sobre archivos `.ipa`. Para utilizar Burp Suite sería necesario disponer de un escenario de análisis dinámico, es decir, poder ejecutar la aplicación en un dispositivo físico o en un entorno iOS adecuadamente preparado, con capacidad para redirigir el tráfico a través de un proxy e instalar el certificado correspondiente. Dado que en este laboratorio no se cuenta con dispositivos con `jailbreak` ni con un entorno de ejecución iOS completo, nos hemos limitado a la revisión estática del paquete `.ipa`.

4. Conclusiones

Como conclusión, en este ejercicio se ha preparado y validado un entorno funcional de análisis para aplicaciones móviles Android e iOS. En la parte Android se configuraron correctamente herramientas como `Genymotion`, `ADB`, `MobSF`, `JADX`, `apktool`, `Ghidra`, `Frida`, `Objection`, `Wireshark` y `Burp Suite`, comprobando su funcionamiento mediante análisis estático, dinámico e interceptación de tráfico. En la parte iOS se dejó preparado un entorno orientado a análisis estático, utilizando `MobSF` para revisar el contenido del `.ipa` y Ghidra para el análisis del binario `Mach-O`.

El ejercicio prepara la infraestructura necesaria para las fases posteriores de reversing y análisis de seguridad, y que servirá como base para los análisis que se llevarán a cabo en los siguientes ejercicios.

Notas para mejorar el ejercicio

Drozer (anteriormente Mercury): Es principalmente un framework de evaluación de seguridad interactivo. Se centra en interactuar con la aplicación Android una vez instalada en el dispositivo (o emulador) para buscar vulnerabilidades en sus componentes (Activities, Services, Broadcast Receivers, Content Providers).

MARA Framework (Mobile Application Reverse engineering and Analysis): Es un framework de ingeniería inversa y análisis de aplicaciones móviles. Se enfoca más en desensamblar el APK, analizar el código fuente (SMALI), extraer información (strings, URLs, APIs) y buscar vulnerabilidades estáticas de forma más

Magisk

OWASP ZAP (Zed Attack Proxy): Es una herramienta diseñada específicamente para encontrar vulnerabilidades en aplicaciones web y realizar pruebas de penetración (pentesting). Funciona como un proxy intermedio (man-in-the-middle) que intercepta, inspecciona y modifica el tráfico HTTP/HTTPS entre tu navegador y el servidor web.